Guia 1 - Clonar el repositorio y levantar el microservicio en local

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** cc + i + ↻

Guía Paso a Paso: configuración y Despliegue de Aplicación Spring Boot

Primera parte: Hacer funcionar el proyecto en ambiente local

1. Verificación e Instalación de Prerrequisitos

Antes de comenzar con la configuración del proyecto, es fundamental asegurar que nuestro entorno de desarrollo cuente con las herramientas necesarias para compilar y gestionar el código fuente.

- **Java Development Kit (JDK 17):** Entorno de ejecución y compilación necesario para aplicaciones Spring Boot.
- **Apache Maven:** Herramienta para la gestión de dependencias y construcción del proyecto.
- **Git:** Sistema de control de versiones para clonar y gestionar repositorios.
- **Entorno de Desarrollo Integrado (IDE):** IntelliJ IDEA, VS Code o Spring Tools Suite con plugins para Java y bases de datos.

MySQL Server y Workbench / Docker: Motor de base de datos relacional e interfaz de administración.

1. Instalar y configurar Git
2. Instalar y configurar un IDE (IntelliJ IDEA o Spring Tools o VSC (plugins para Java, Spring, Maven y BD))
3. Instalar y con MySQL Server (opcional, pero útil)
4. Instalar y con MySQL Workbench (opcional pero útil)
5. Instalar y conDocker (Docker desktop, Docker componse)

Apertura de la Consola de Comandos desde el Navegador de Archivos

Para facilitar el uso de la terminal en la ubicación deseada, nos posicionamos en la carpeta de trabajo del sistema operativo y accedemos a la barra de direcciones.

Hacemos clic en la barra de direcciones o seleccionamos la opción **"Editar dirección"** mediante clic derecho para escribir el comando de apertura de la terminal.

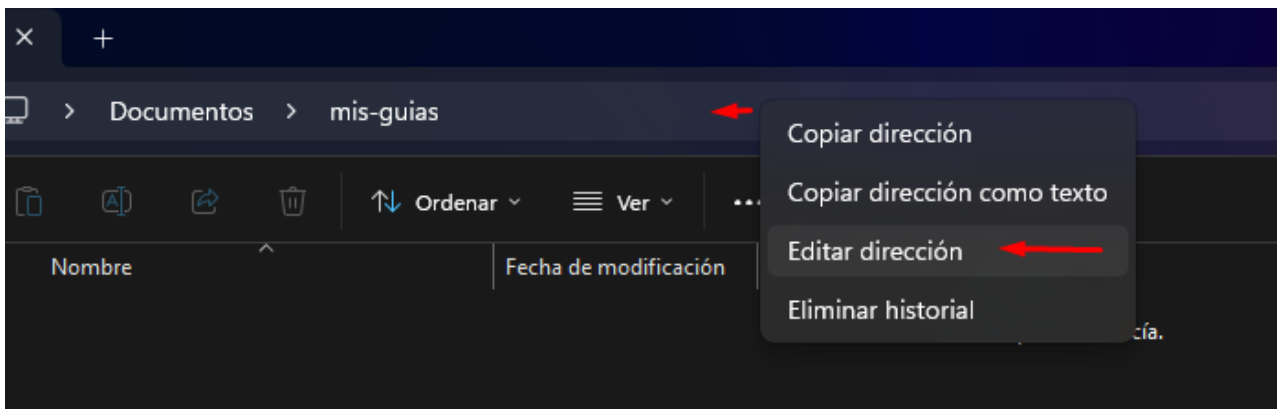


Imagen 1: Menú contextual del explorador de archivos para acceder a la barra de direcciones mediante la opción "Editar dirección".
*Sustituimos la ruta actual escribiendo **cmd** y presionamos la tecla **Enter**. Esto abrirá una ventana de la consola de comandos directamente enfocada en este directorio.*

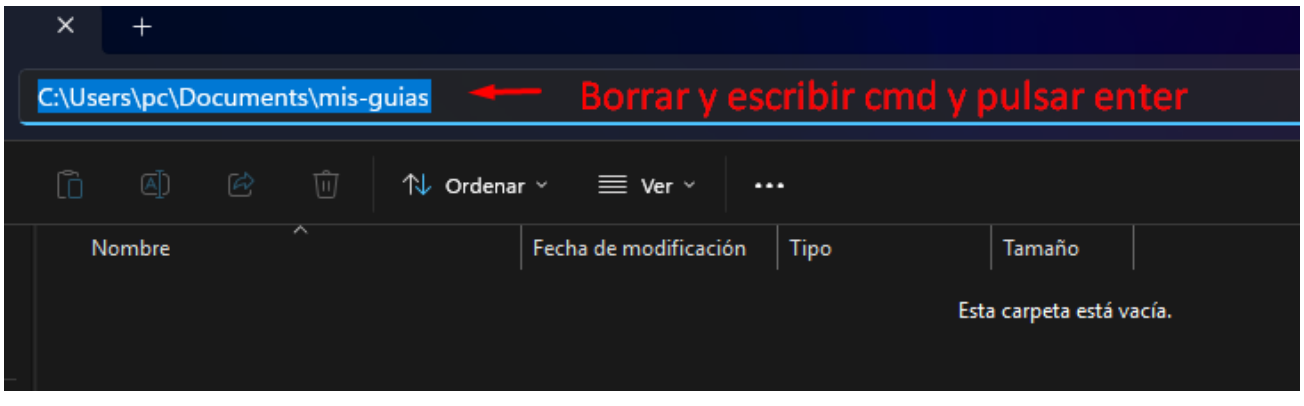


Imagen 2: Reemplazo de la ruta por el comando "cmd" en la barra de navegación para abrir la consola de comandos en la carpeta actual.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** cc + by + sa

---

Verificación de Versiones Instaladas

Ejecutamos los comandos **java --version** y **git --version** para confirmar que ambas herramientas están correctamente instaladas y configuradas en las variables de entorno del sistema.

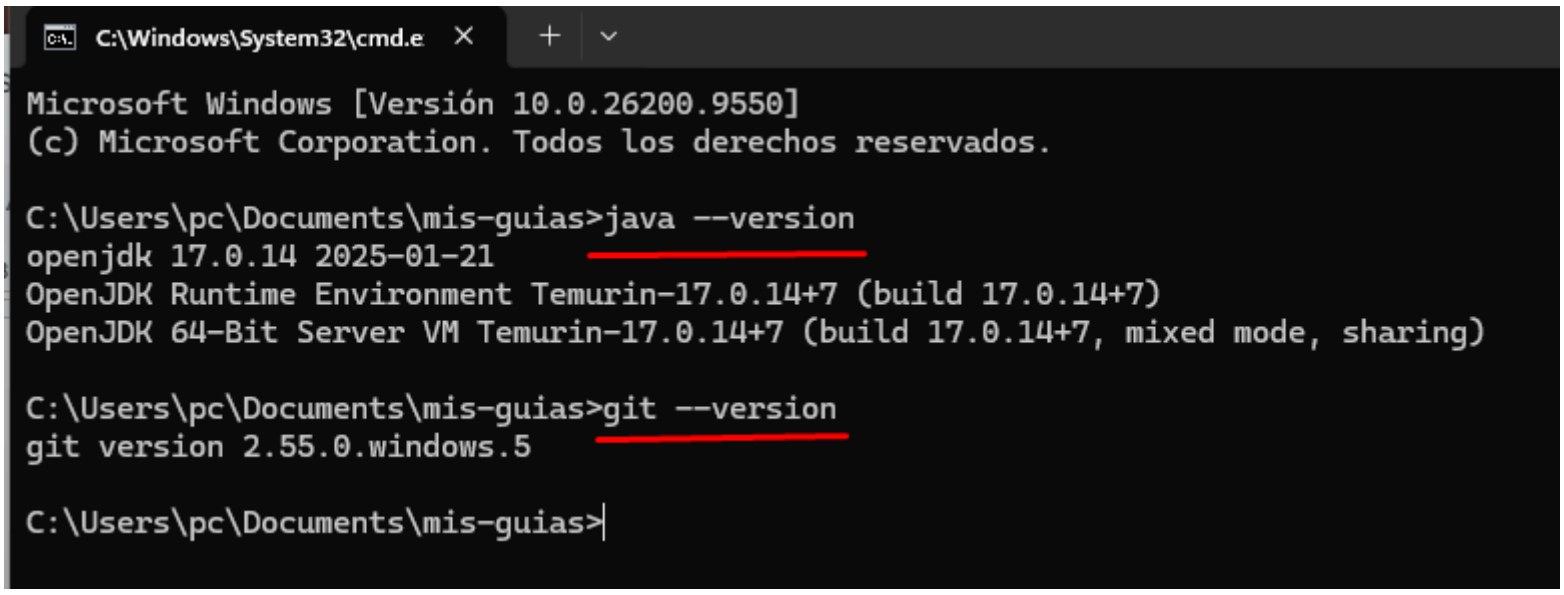


Imagen 3: Consola de comandos mostrando la verificación exitosa de las versiones de Java (JDK 17) y Git.

2. Clonación del Repositorio de Código Fuente

El siguiente paso consiste en obtener una copia local del repositorio remoto alojado en GitHub para comenzar a trabajar en nuestra máquina local.

NOTA IMPORTANTE:

El repo que aparece en las Imagenes de ejemplo, actualmente ya no existe, fue renombrado con este nombre: ***users-management-microservice*** y la URL es https://github.com/arrietajohn/users-management-microservice

Autenticación por PAT (Personal Access Token)

Creamos una cuenta en GitHub o iniciar sesión si ya tenemos una.

A continuación, detallamos el procedimiento completo para la creación de un **Personal Access Token (PAT) Clásico** en GitHub. Un PAT actúa como una credencial de seguridad sustitutiva de la contraseña personal al realizar operaciones de autenticación sobre HTTPS mediante Git CLI, IDEs o servicios de integración continua.

1. **Acceso a la configuración global del usuario**

En la esquina superior derecha de la interfaz de GitHub, haz clic sobre el icono de tu perfil de usuario y selecciona la opción **Settings** (configuración global). Esta sección administra la seguridad de tu identidad, claves SSH/GPG y tokens de acceso personal.

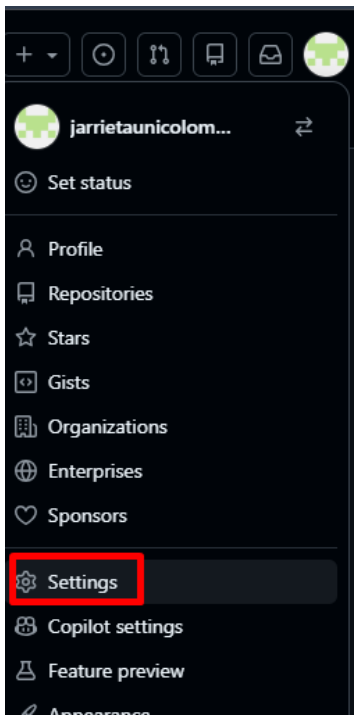


Imagen 4: Acceso al menú de configuración global de usuario en GitHub.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

---

2. Navegación hacia las herramientas de desarrollador

En la barra lateral izquierda de la pantalla de configuración, desplázate hasta el final de la lista de opciones y selecciona **Developer settings** (configuración de desarrollador). Desde esta sección se centraliza la creación y gestión de GitHub Apps, OAuth Apps y tokens de acceso personal.

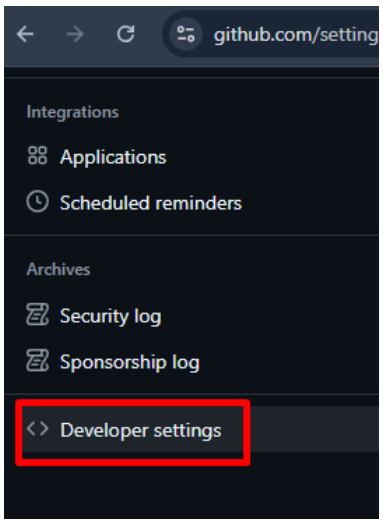


Imagen 5: Navegación hacia las herramientas de desarrollador en GitHub.

3. Selección del tipo de Token (Tokens Classic)

Dentro del menú de Developer Settings, despliega la opción **Personal access tokens** y haz clic en **Tokens (classic)**. Aunque existen tokens con grano fino (fine-grained), la modalidad clásica proporciona una compatibilidad amplia y directa con herramientas como Git CLI, IntelliJ IDEA y herramientas de automatización.

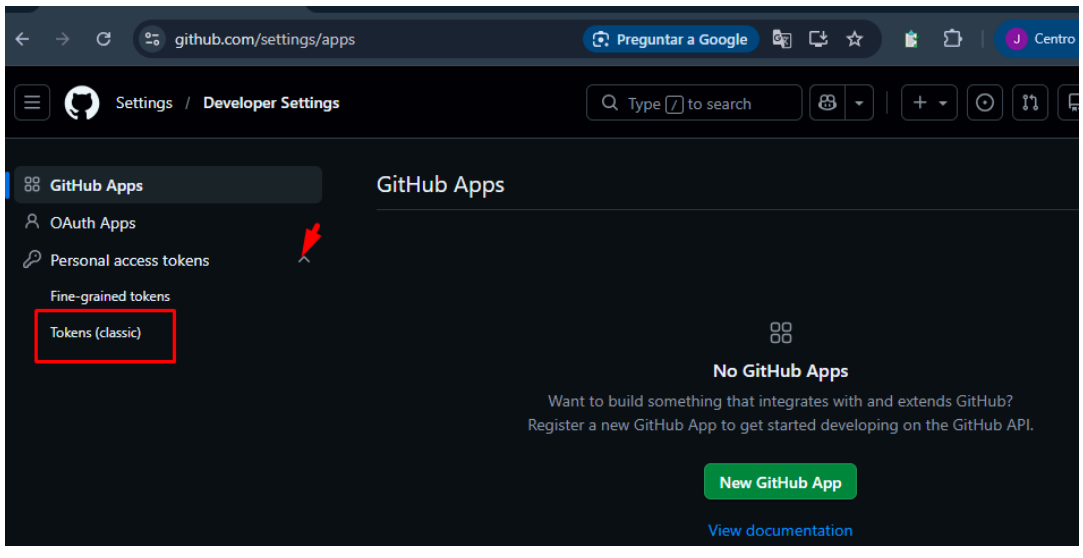
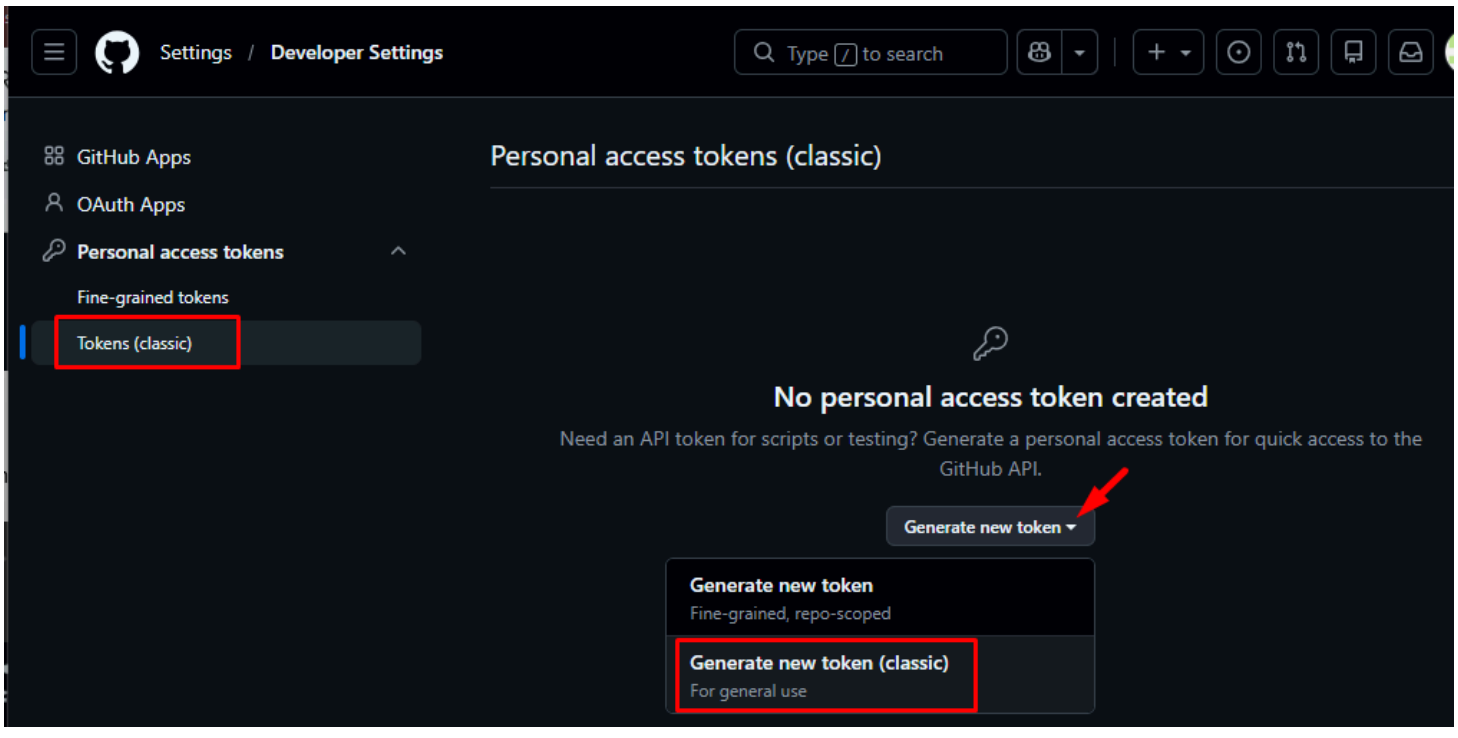


Imagen 6: Selección de la categoría Tokens clásicos de acceso personal.

4. Generación de la nueva credencial

Haz clic en el botón desplegable **Generate new token** ubicado en la parte superior derecha y selecciona **Generate new token (classic)**. Si el sistema lo requiere por políticas de seguridad, confirma tu contraseña o la autenticación de dos factores (2FA).



CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Imagen 7: Desplegable para generar un nuevo token clásico en GitHub.

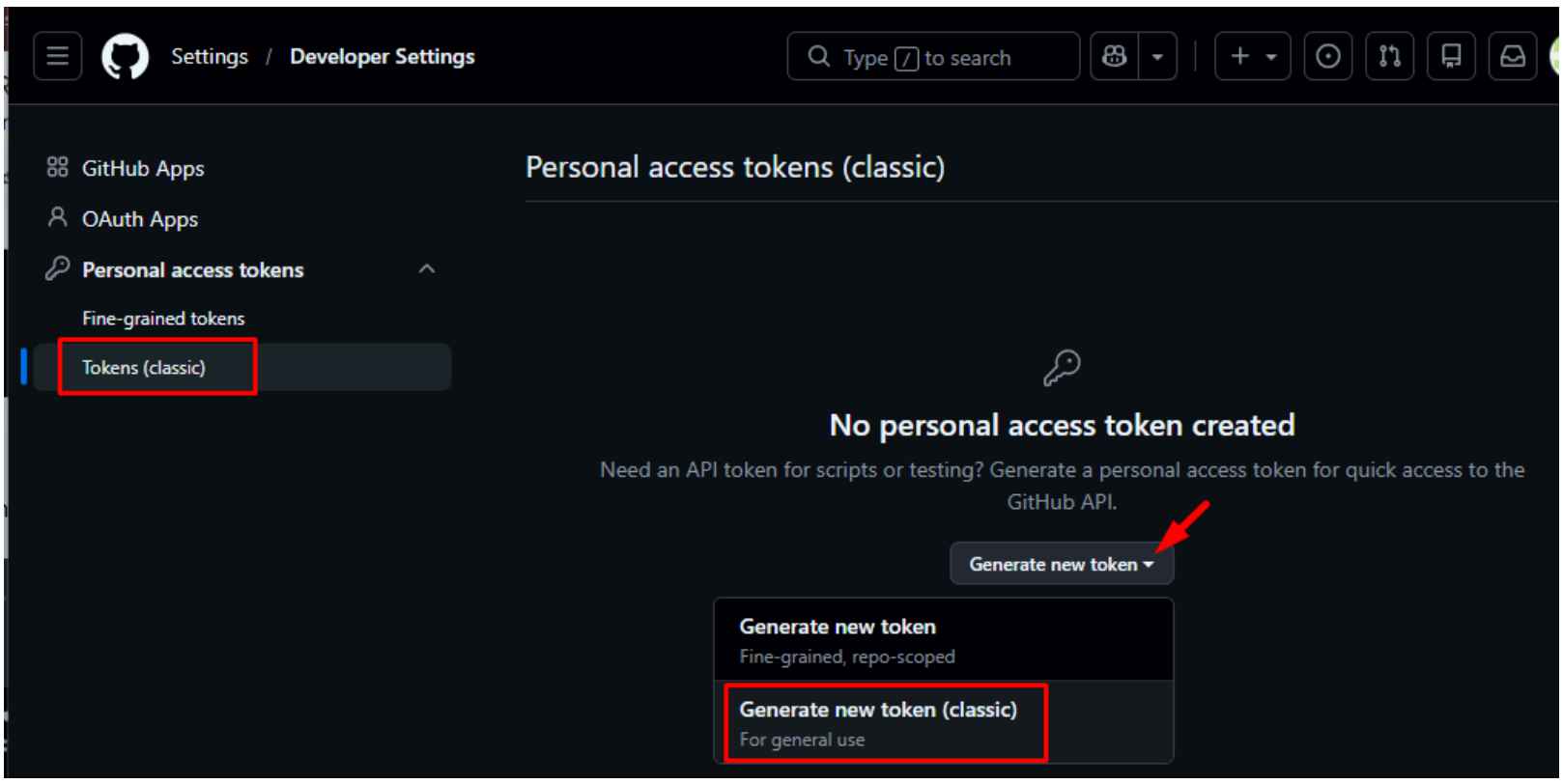


Imagen 8: Confirmación para la generación de un nuevo token clásico en GitHub.

5. **configuración del formulario de creación y asignación de permisos (Scopes)**

   En el formulario de configuración del PAT, especifica los siguientes parámetros clave:

- **1. Note (Nota descriptiva):** Asigna un nombre claro y contextual al token (por ejemplo, **`TOKEN-PARA-MI-USAR-GH-DESDE-EN-MI-PC`**). Esto permite auditar en el futuro desde qué equipo o servicio se está utilizando la credencial y revocarla individualmente en caso de extravío o exposición.
-
- **2. Expiration (Vigencia del Token):** Define el periodo de validez de la credencial (por ejemplo, 30 días, 60 días, 90 días o personalizado). Por buenas prácticas de ciberseguridad, se aconseja renovar periódicamente los tokens para mitigar el riesgo derivado de una eventual filtración.
-
- **3. Justificación detallada de los Permisos (Scopes) seleccionados:**
  - **repo (Full control of private repositories):** Otorga acceso total para lectura, escritura, creación, clonación y envío de commits (git push) en repositorios públicos y privados. Es esencial para la interacción cotidiana mediante Git CLI e IDEs.
  - **workflow (Update GitHub Action workflows):** Permite actualizar y administrar los archivos de definición de flujos de integración y despliegue continuos (CI/CD) alojados en el directorio **`.github/workflows/`**.
  - **write:packages / delete:packages:** Permite publicar, actualizar y eliminar paquetes de software o Imagenes de contenedor en el registro oficial GitHub Packages.
  - **admin:org (Full control of orgs and teams):** Facilita la administración de miembros, equipos y configuraciones dentro de organizaciones de GitHub a las que pertenece el usuario.
  - **admin:public_key (Full control of user public keys):** Otorga permisos para listar, agregar o eliminar claves SSH públicas asociadas a la cuenta del usuario.
  - **admin:repo_hook (Full control of repository hooks):** Otorga el control de los Webhooks de repositorio para notificar automáticamente eventos a servicios externos (como Jenkins, Discord o Render).
  - **audit_log (Full control of audit log):** Permite la lectura y análisis de los registros de auditoría de seguridad para consultar eventos de acceso e historial de acciones administrativas.
  - **codespace (Full control of codespaces):** Habilita la creación, administración y conexión remota a entornos de desarrollo en la nube administrados por GitHub Codespaces.
  - **copilot (Full control of GitHub Copilot settings):** Permite administrar licencias, configuraciones y accesos a las herramientas de asistencia de desarrollo basadas en IA (GitHub Copilot).
  - **write:network_configurations:** Permite administrar las configuraciones de red e infraestructura privada de ejecución para los agentes de GitHub Actions.
  - **project (Full control of projects):** Proporciona control total sobre los tableros de gestión de proyectos, mapas de ruta e incidencias (Projects v2).
  - **admin:gpg_key / admin:ssh_signing_key:** Permite gestionar las claves de firma criptográfica (GPG y SSH) para validar la autenticidad y autoría de los commits y etiquetas firmadas.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:**

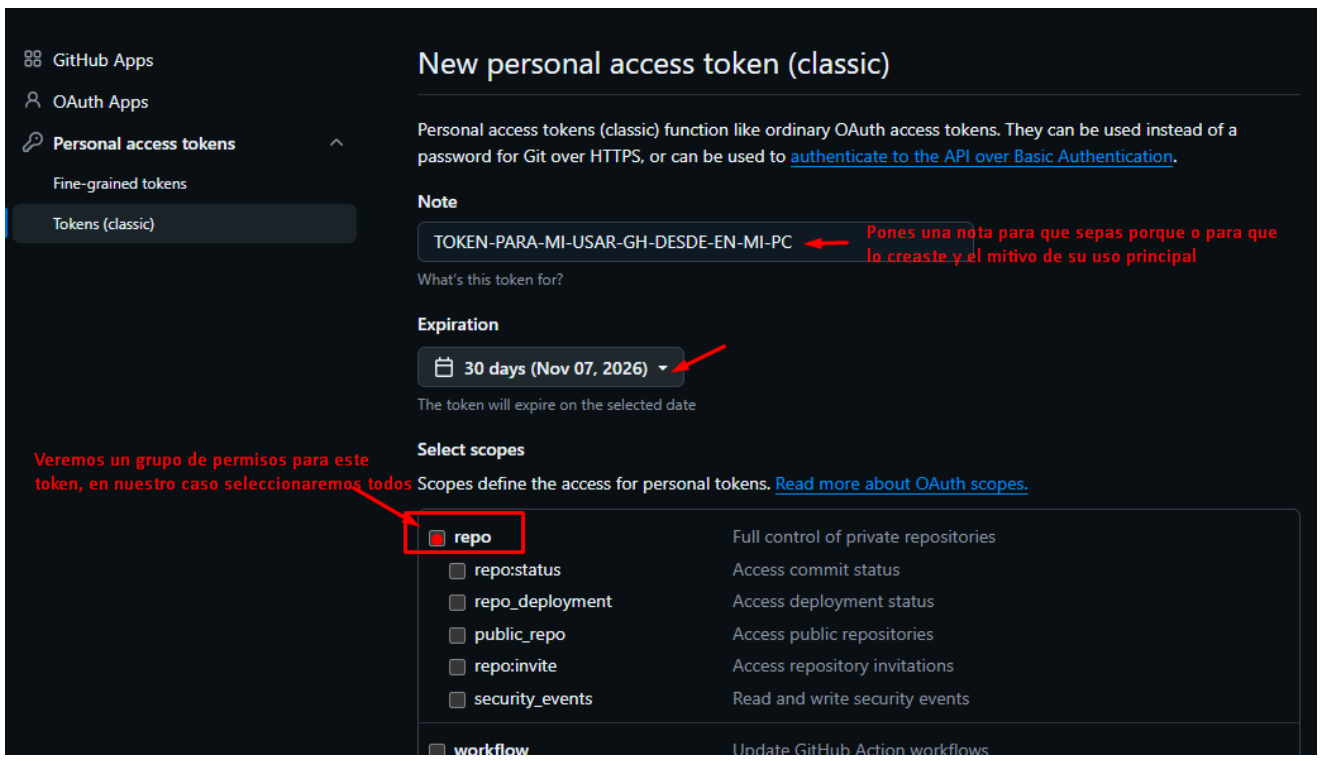


Imagen 9: Formulario de asignación de nombre, expiración y permisos del token.

Asegúrate de marcar las casillas correspondientes a los grupos de permisos requeridos para la práctica.

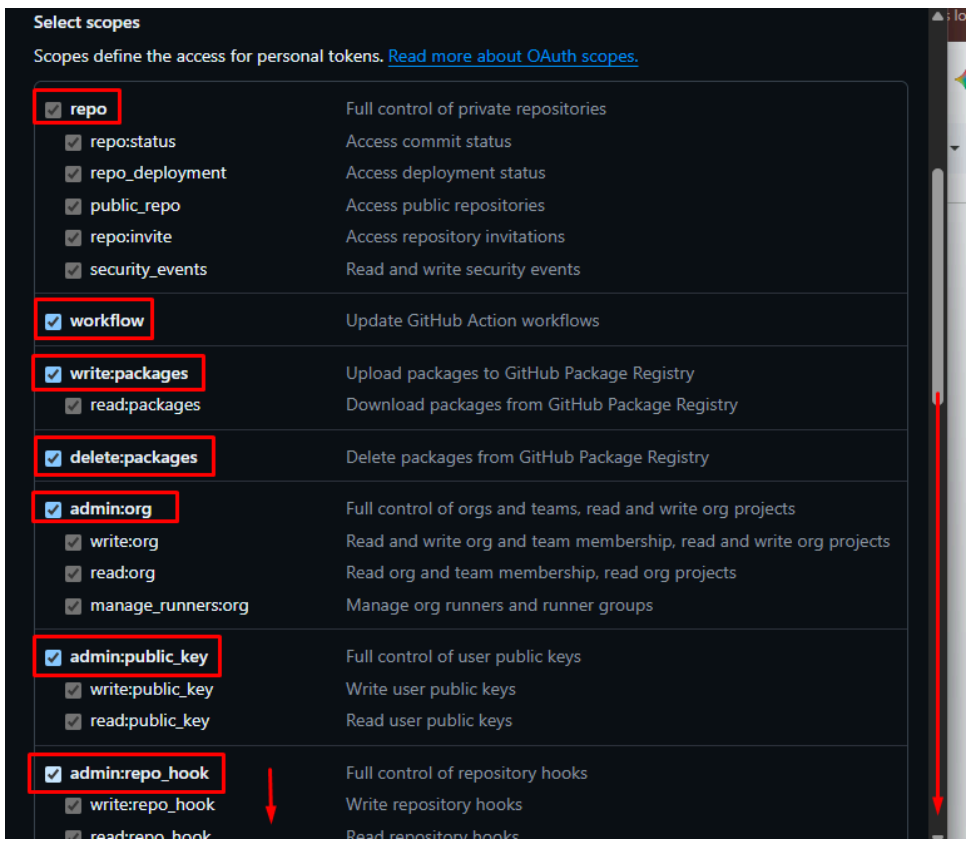


Imagen 10: Selección detallada de permisos e itinerario de acceso requeridos.

Tras revisar la selección de permisos, desplázate al final de la página y haz clic en el botón verde **Generate token**.

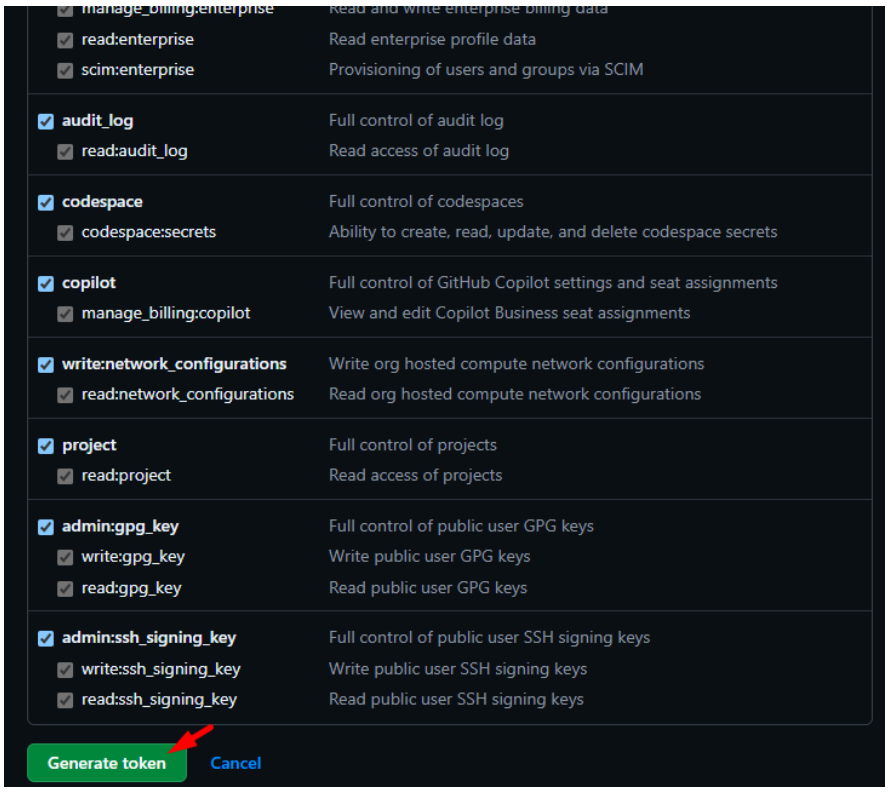


Imagen 11: Confirmación final y envío del formulario para generar el token.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

6. Resguardo seguro de la clave generada

Una vez generado el token, la interfaz mostrará una cadena alfanumérica con el prefijo **ghp_**. Copia de inmediato este valor y guárdalo en un gestor de contraseñas o ubicación segura. Por razones de seguridad, GitHub jamás volverá a mostrar esta clave en pantalla.

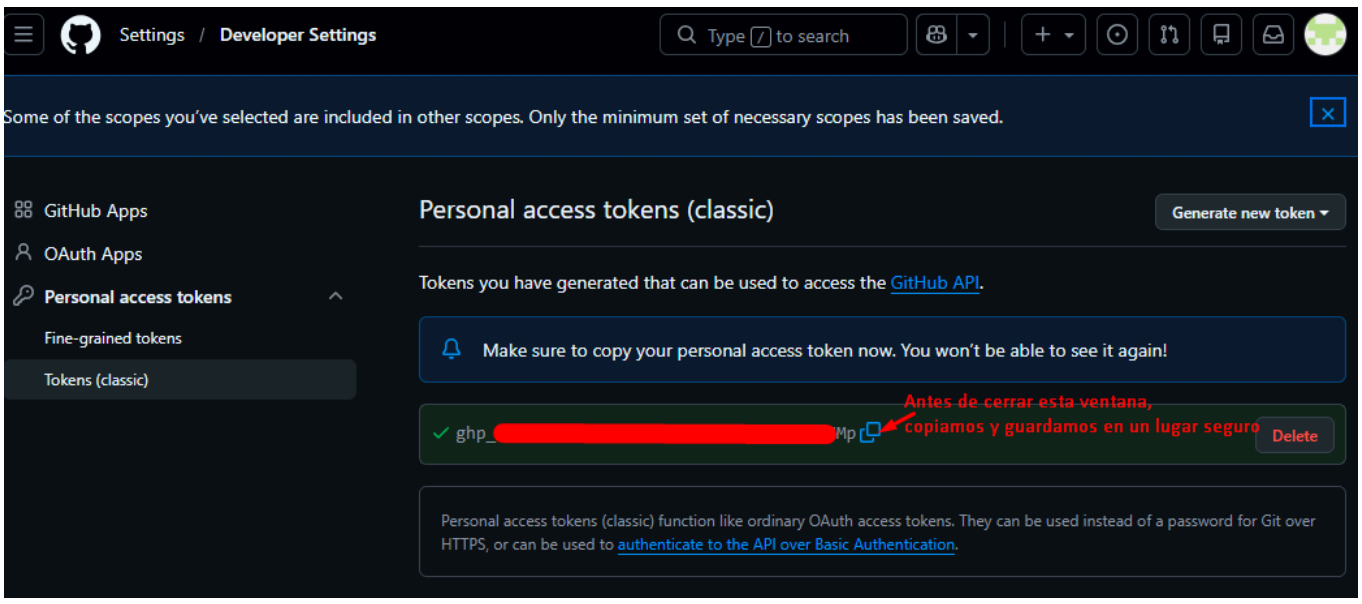


Imagen 12: Pantalla de presentación y copia del Personal Access Token generado.

7. Confirmación de credenciales activas

En el listado general de Personal Access Tokens podrás verificar que la credencial recién creada aparece con el estado activo, mostrando su nota identificativa, fecha de expiración y alcance de permisos asignados.

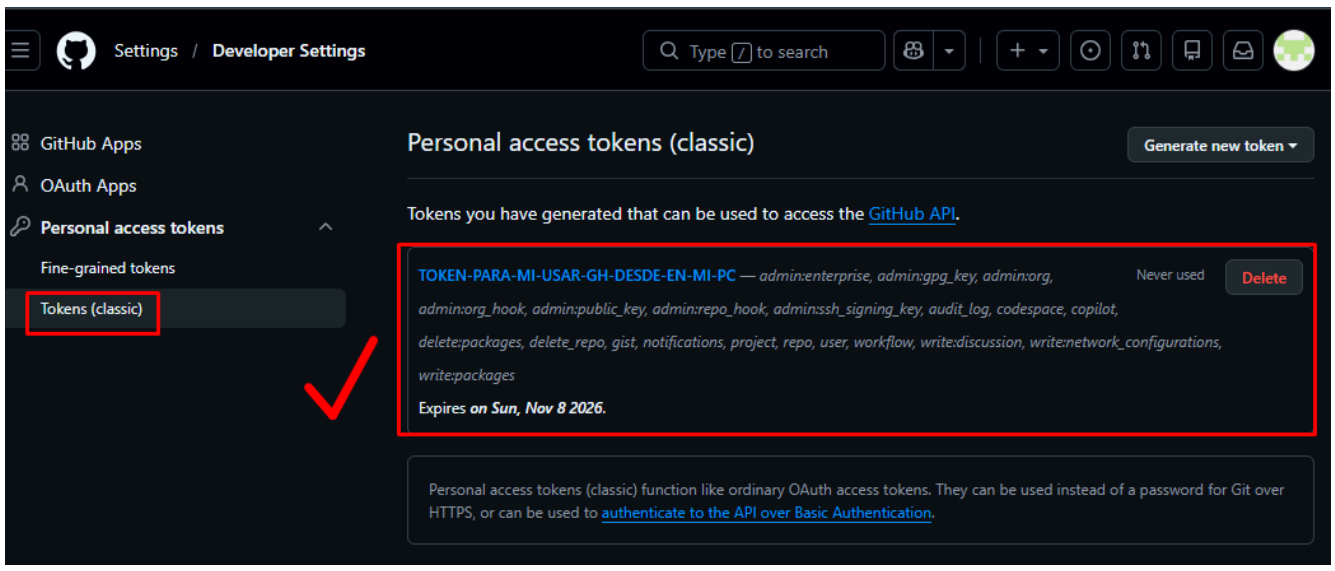


Imagen 13: Lista de tokens de acceso personal confirmando la credencial activa.

NOTA IMPORTANTE:

El repo que aparece en las Imagenes de ejemplo, actualmente ya no existe, fue renombrado con este nombre: ***users-management-microservice*** y la URL es https://github.com/arrietajohn/users-management-microservice

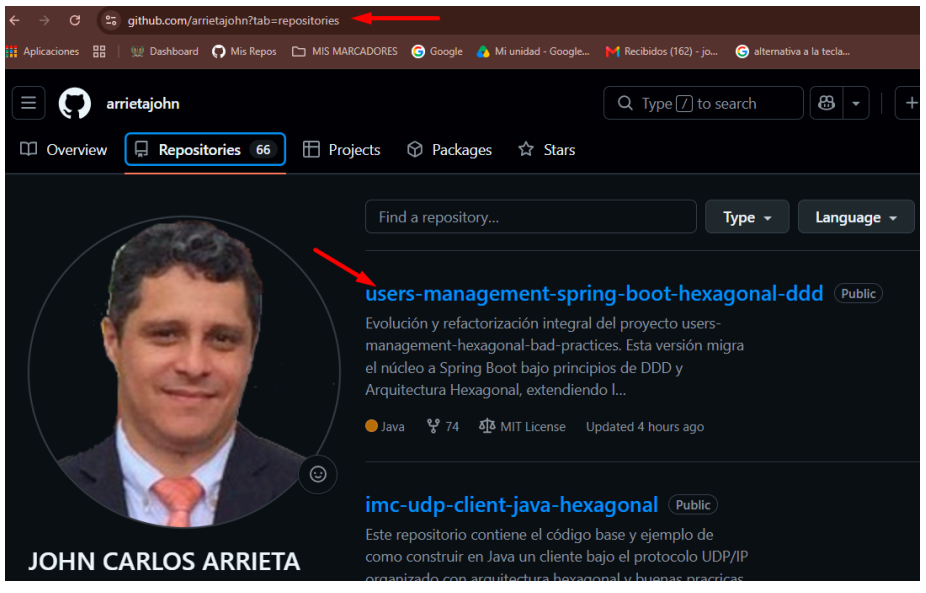


Imagen 14: Vista del perfil de GitHub con la lista de repositorios disponibles, destacando el proyecto base.
https://github.com/arrietajohn/users-management-microservice

Guia 1 - Clonar el repositorio y levantar el microservicio en local

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:**

Selección del Repositorio en GitHub

Navegamos al perfil del proyecto en GitHub y localizamos el repositorio deseado (en este caso, users-management-microservice).

Creación de un Fork (Bifurcación)

Hacemos clic en el botón **Fork** ubicado en la esquina superior derecha para crear una copia personal del proyecto en nuestra cuenta de GitHub, lo que nos permitirá realizar cambios libremente.

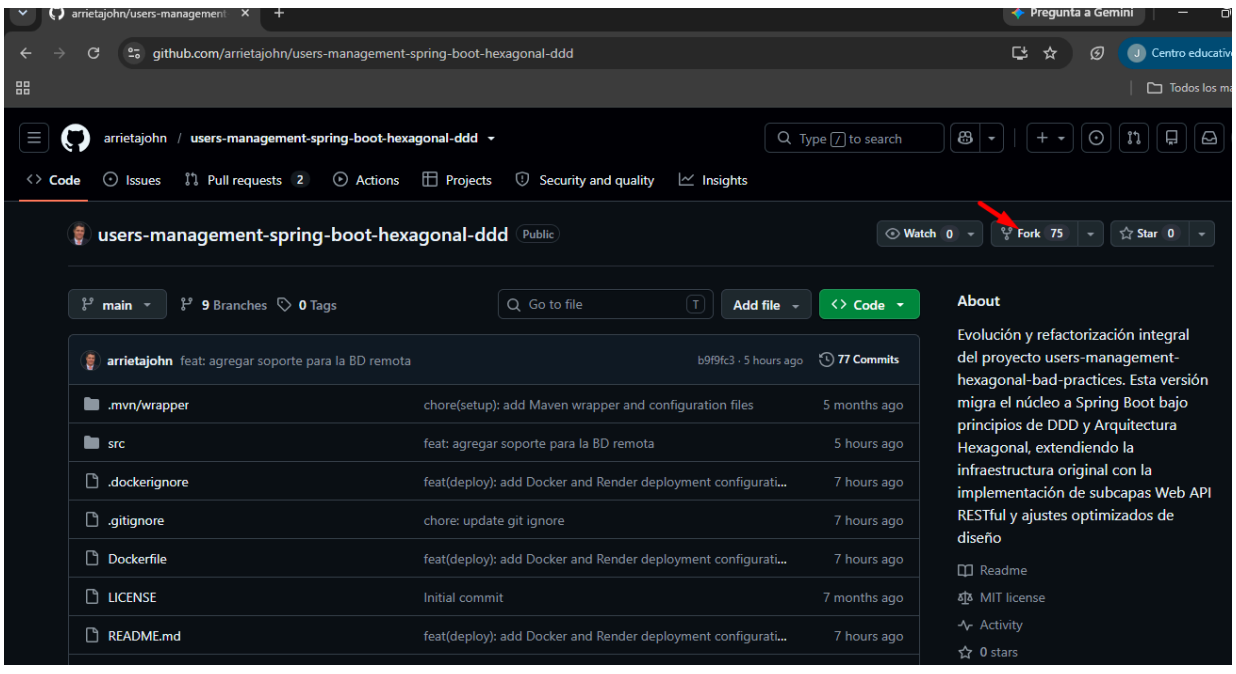


Imagen 15: Vista principal del repositorio de GitHub resaltando la opción "Fork" en la esquina superior derecha.

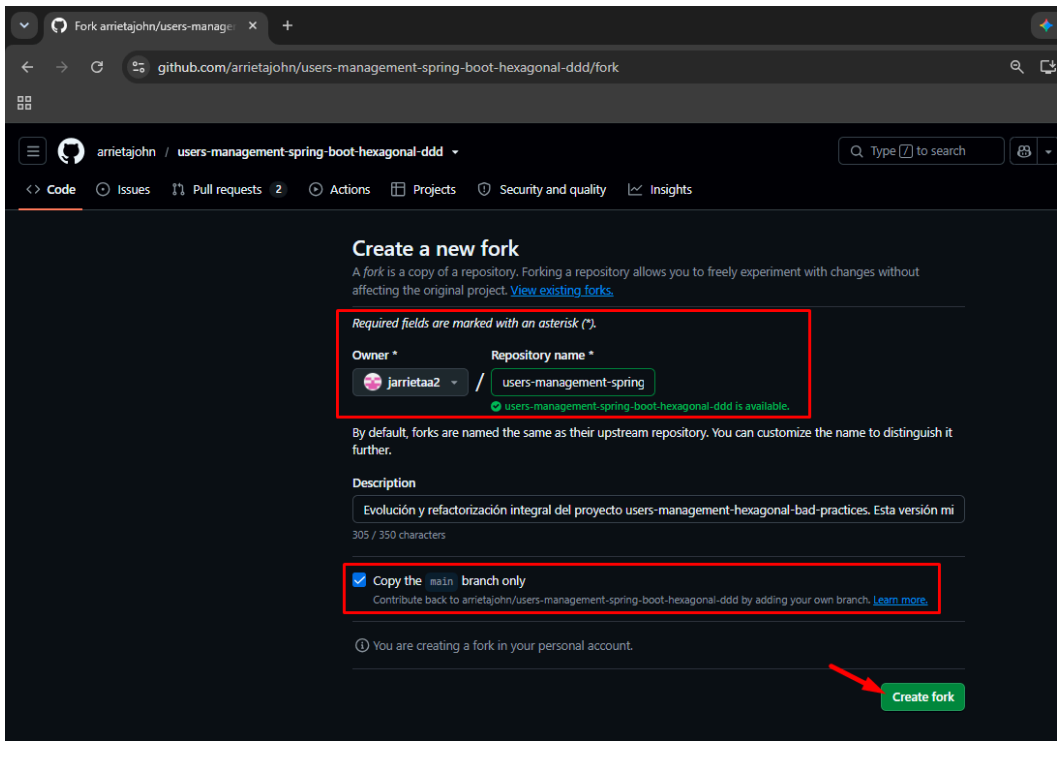


Imagen 16: Formulario de creación de Fork en GitHub para duplicar el repositorio en la cuenta personal del usuario.

Copia de la URL de Clonación

Dentro de nuestro fork del repositorio, hacemos clic en el botón verde **Code** y copiamos la dirección HTTPS correspondiente.

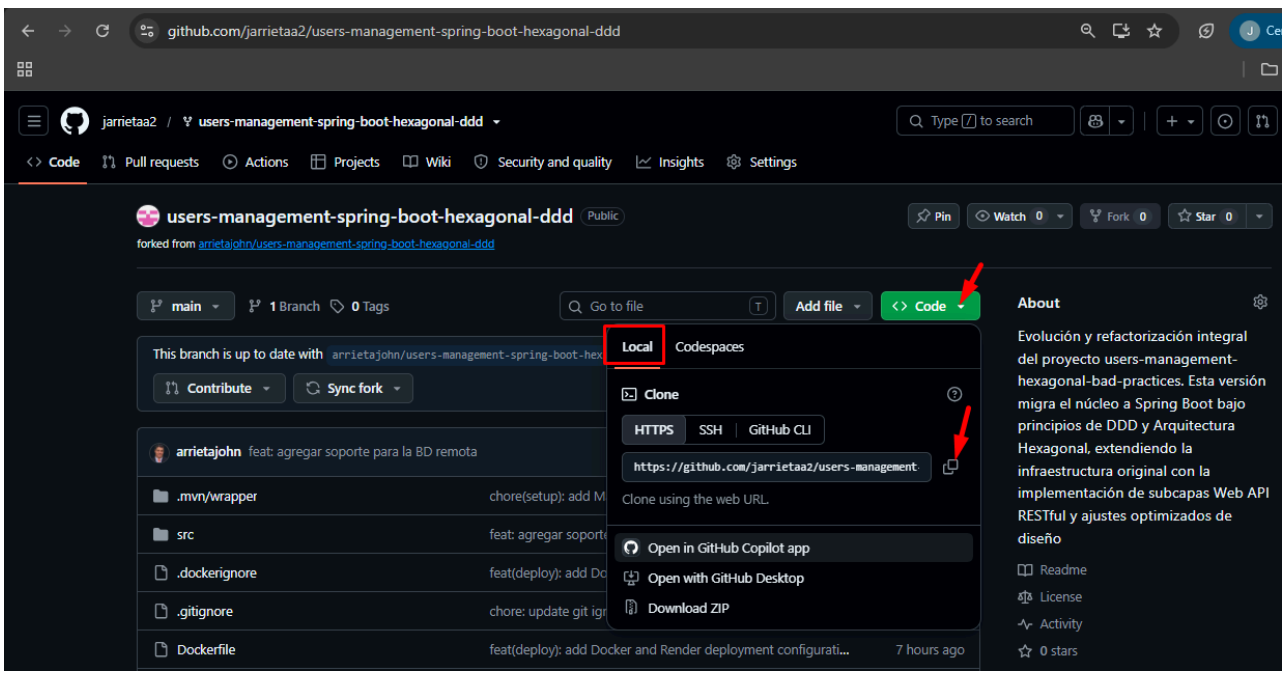


Imagen 17: Menú desplegable "Code" en GitHub con la URL HTTPS seleccionada para copiar al portapapeles.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Ejecución del Comando Git Clone

En la consola de comandos anteriormente abierta, ejecutamos **git clone <URL-del-repositorio>**. Esto descargará la estructura completa del proyecto a nuestra carpeta local.

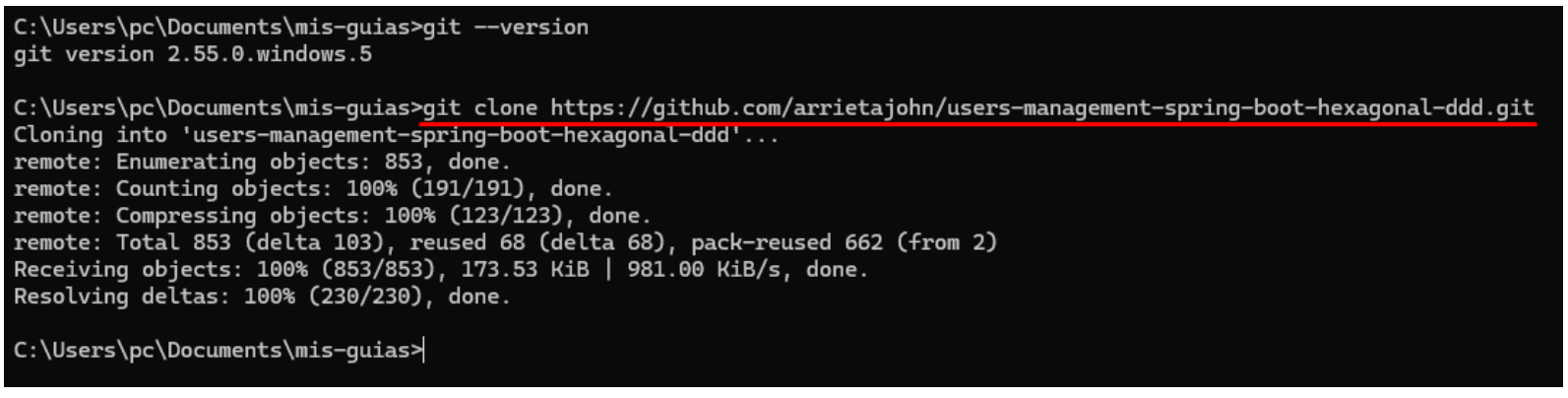


Imagen 18: Ejecución del comando `git clone` en la terminal local descargando con éxito el proyecto.

3. Apertura del Proyecto en el IDE (IntelliJ IDEA)

Una vez descargado el código fuente, procedemos a abrir el proyecto en nuestro entorno de desarrollo para realizar la compilación y configuración de variables.

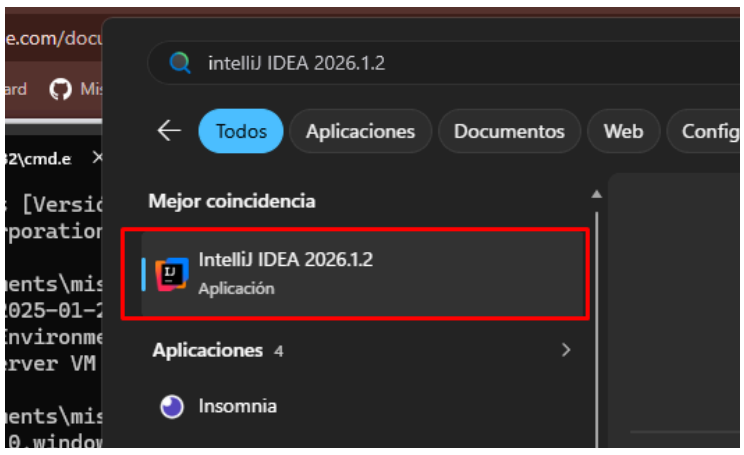


Imagen 19: Acceso al ejecutable de IntelliJ IDEA a través del menú de búsqueda de Windows.

Inicio de IntelliJ IDEA y Selección de Opción Abrir

Iniciamos el IDE e identificamos la opción **Open** en el menú de bienvenida para cargar un proyecto existente desde el disco duro.

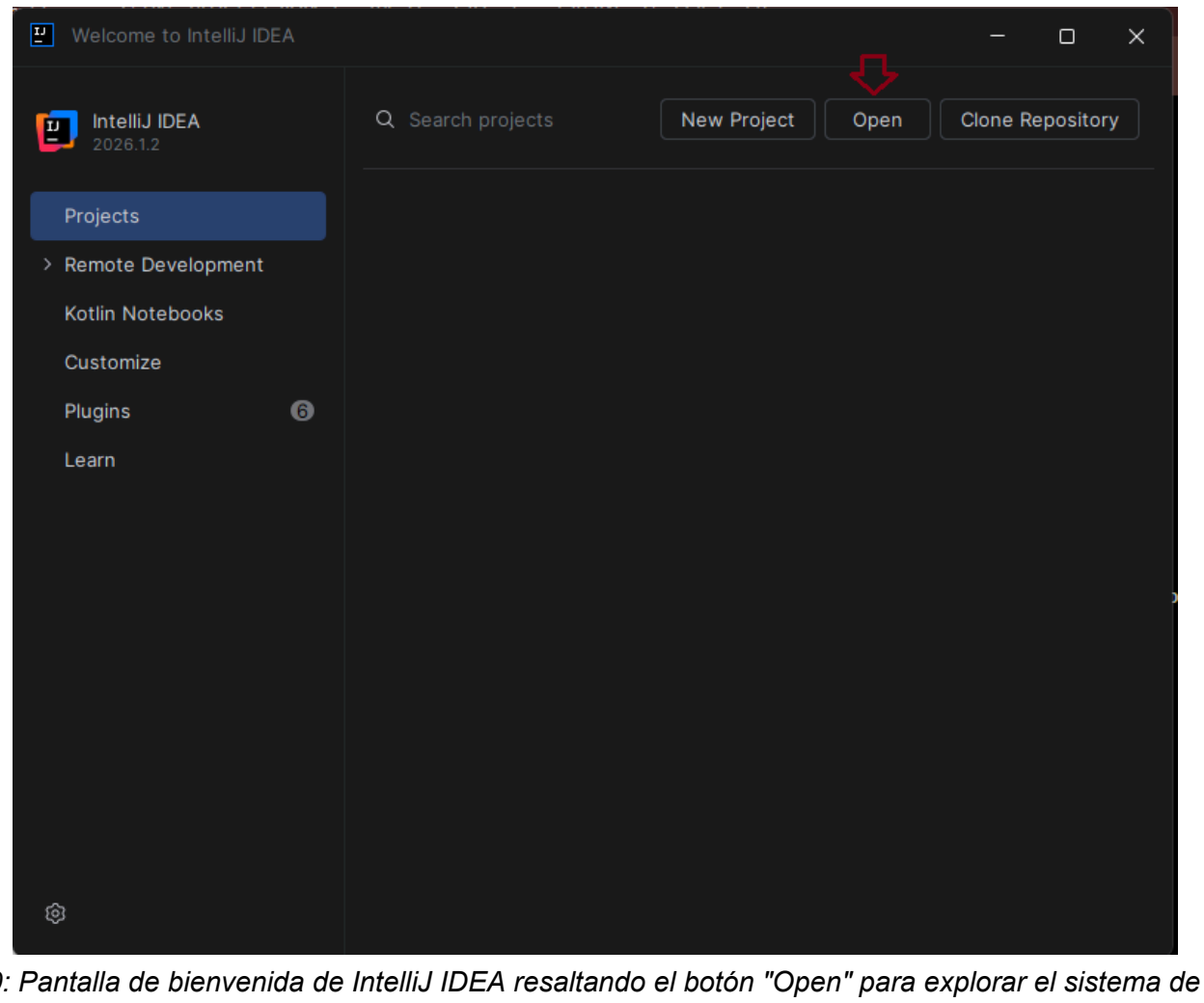


Imagen 20: Pantalla de bienvenida de IntelliJ IDEA resaltando el botón "Open" para explorar el sistema de archivos.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Selección del Directorio del Proyecto

Buscamos la carpeta clonada dentro del explorador de archivos y presionamos **Select Folder** u **OK**.

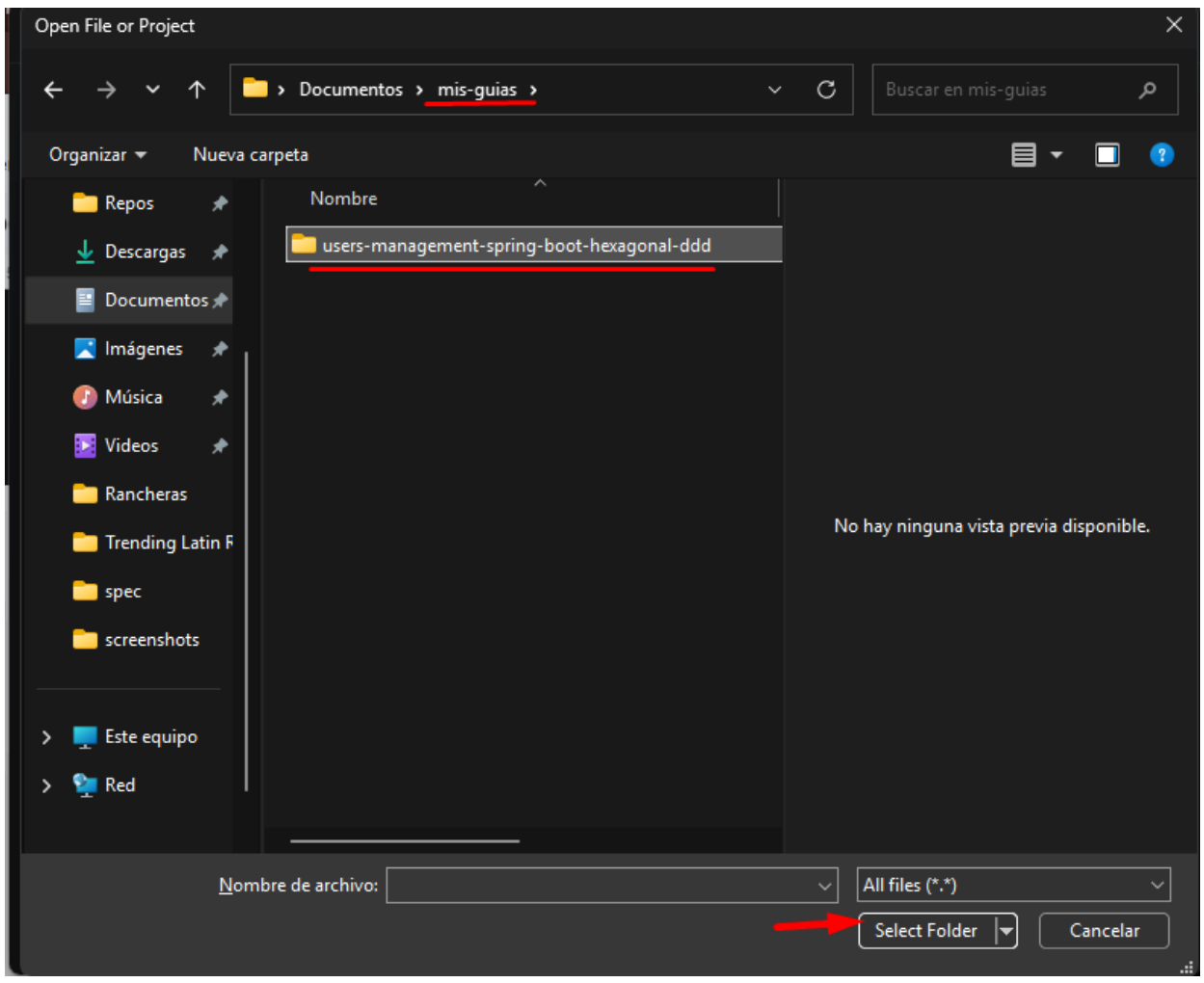


Imagen 21: Explorador de archivos modal seleccionando el directorio raíz del proyecto Java clonado.

Confirmación de Confianza en el Proyecto

Al abrir el proyecto por primera vez, el IDE solicitará confirmación de seguridad. Marcamos las casillas correspondientes y seleccionamos **Trust Project**.

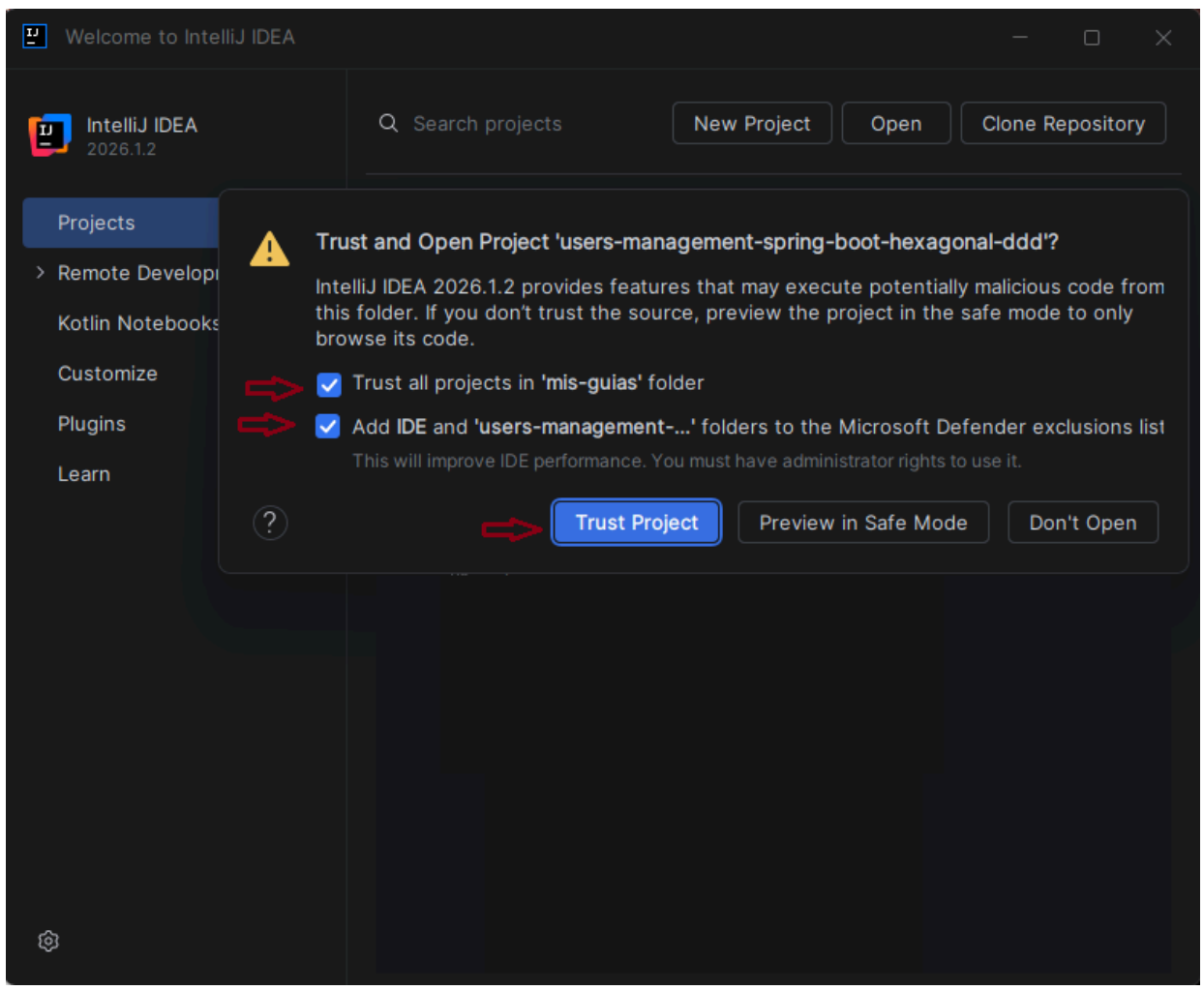


Imagen 22: Diálogo de seguridad en IntelliJ IDEA confirmando la confianza en la procedencia del proyecto ("Trust Project").

Exploración de la Estructura de Archivos

Guia 1 - Clonar el repositorio y levantar el microservicio en local

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Verificamos que la estructura del proyecto en arquitectura hexagonal y DDD se haya cargado correctamente en la vista lateral izquierda, incluyendo el archivo **README.md** y la carpeta **src**.

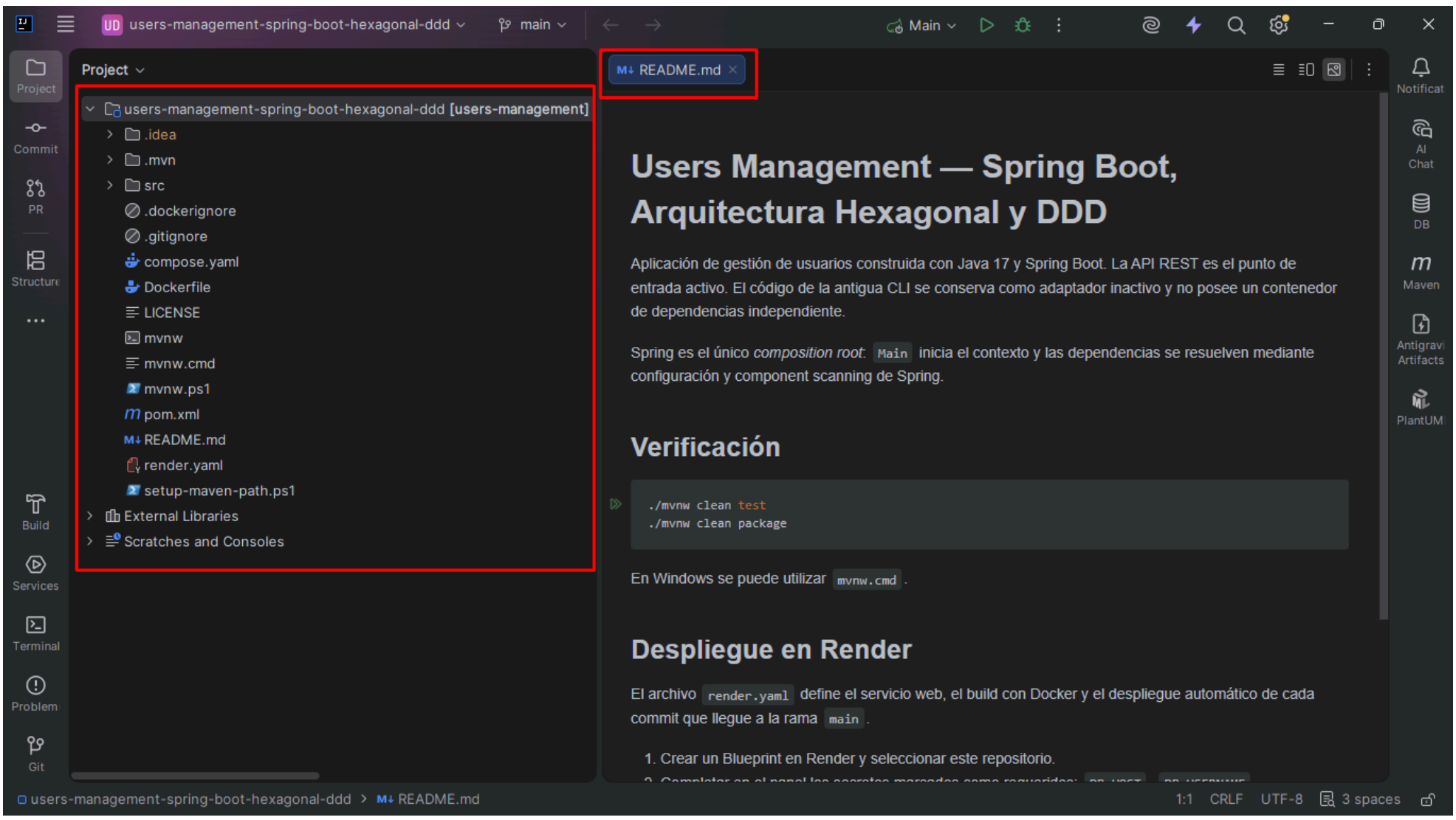


Imagen 23: Entorno de IntelliJ IDEA abierto cargando la arquitectura del proyecto y mostrando la documentación principal.

Apertura de la Terminal Integrada

Abrimos la ventana de terminal integrada dentro de IntelliJ IDEA y seleccionamos el perfil de consola deseado (por ejemplo, **Command Prompt**).

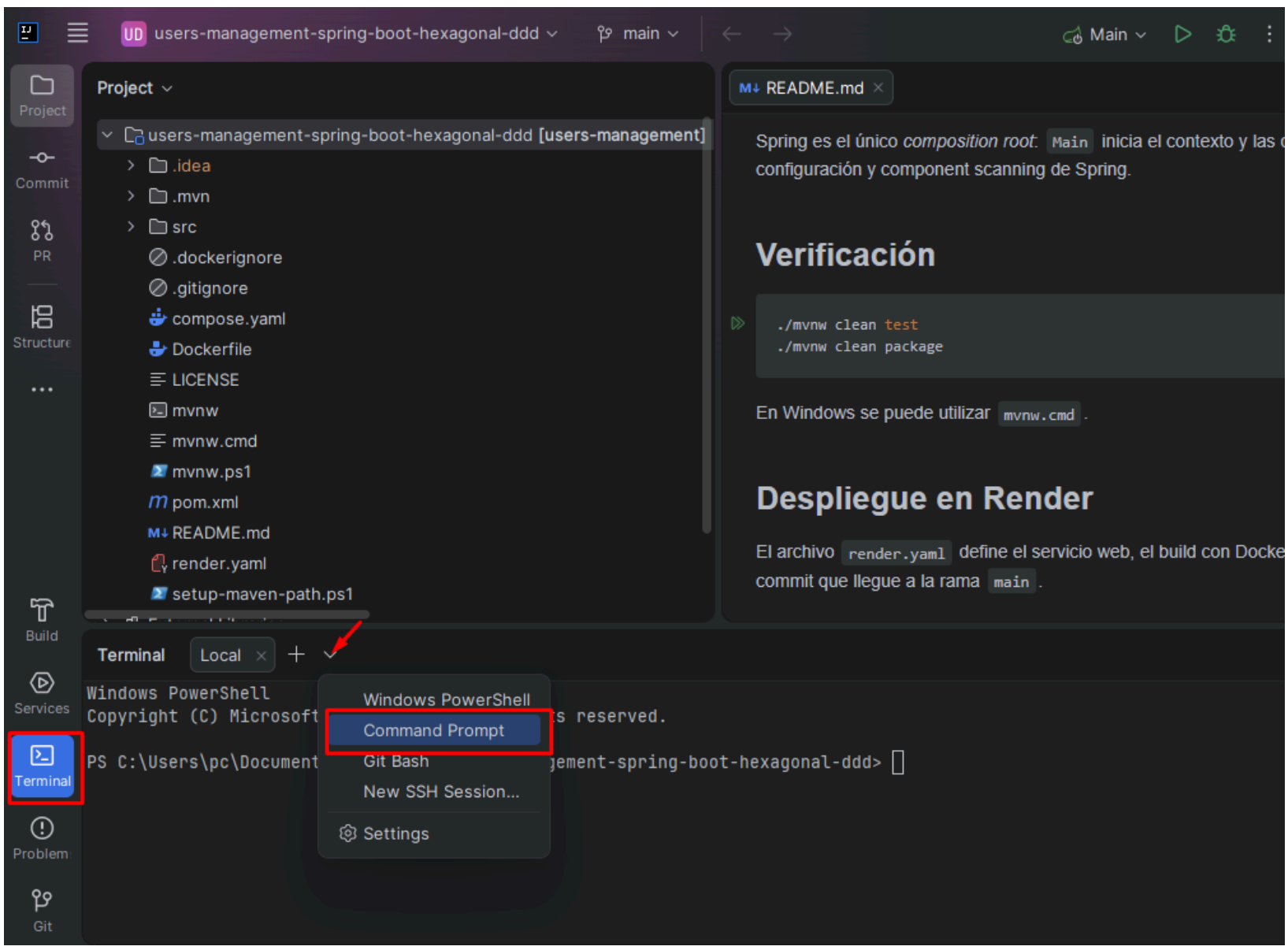


Imagen 24: Panel inferior del IDE activando la terminal integrada e iniciando una sesión de Command Prompt.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Compilación Inicial con Maven Wrapper

Ejecutamos el comando **mvnw.cmd clean install -U** para descargar todas las dependencias declaradas en el archivo **pom.xml** y verificar la construcción inicial.

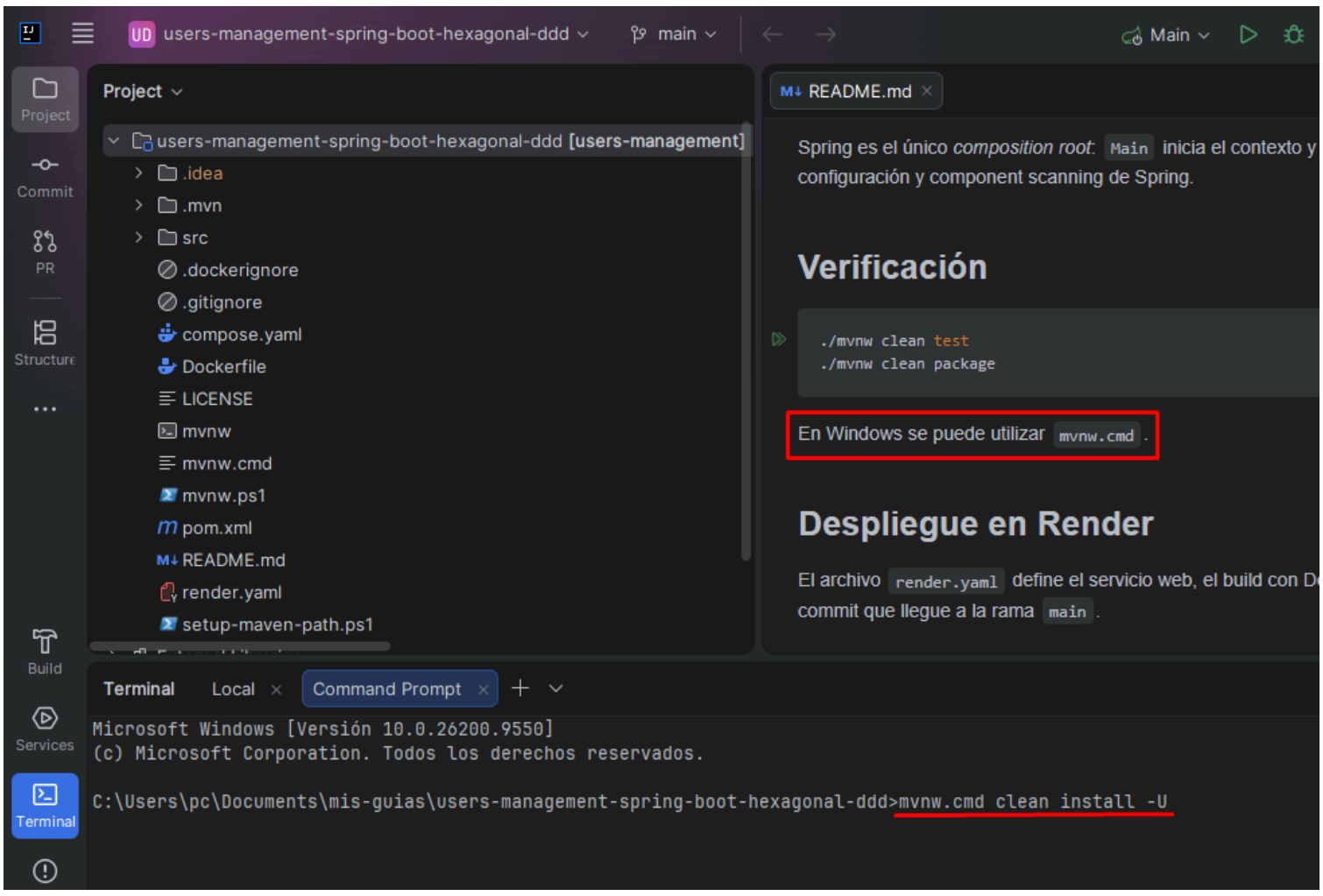


Imagen 25: Ingreso del comando `mvnw.cmd clean install -U` en la terminal embebida del IDE.

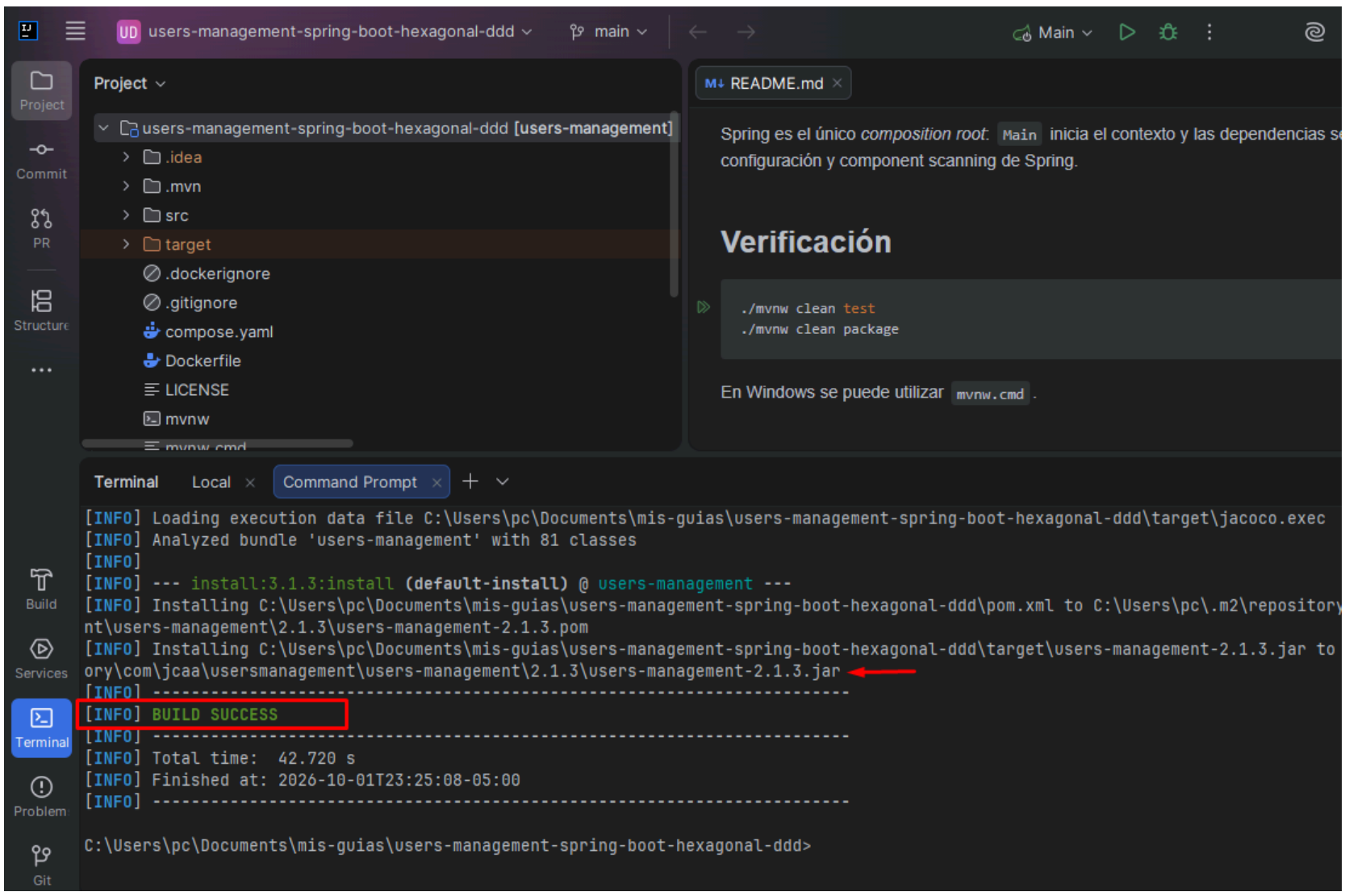


Imagen 26: Resultado exitoso de la compilación inicial mostrando la confirmación "BUILD SUCCESS".

Verificación del Resultado de Compilación

Confirmamos que la salida en consola finalice con el mensaje **BUILD SUCCESS**, indicando que la estructura del proyecto no contiene errores sintácticos ni dependencias faltantes.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

4. Instalación y configuración de MySQL Server Local

Para que nuestra aplicación Spring Boot se conecte a un motor de persistencia, procedemos a descargar y configurar una instancia local de MySQL Server.
URL de descarga: https://dev.mysql.com/downloads/mysql/9.7.html (versión utilizada para esta guía).

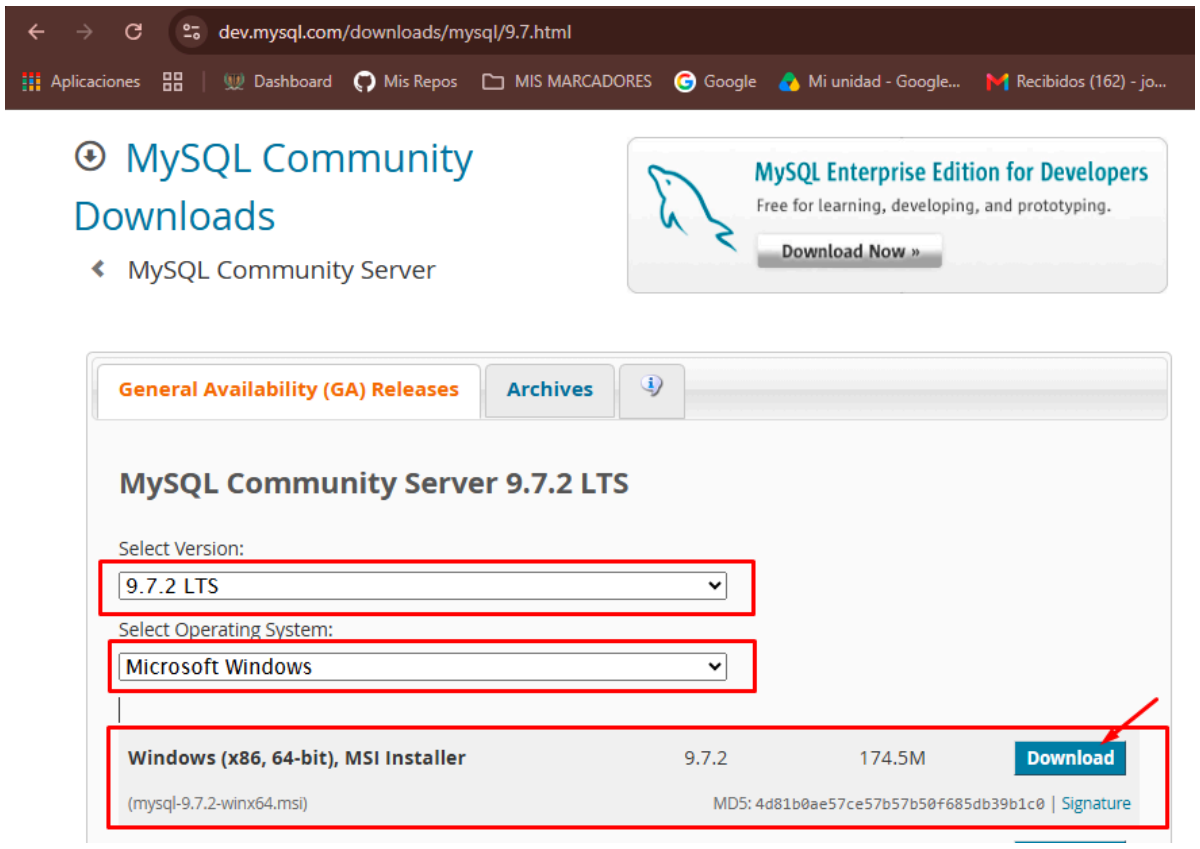


Imagen 27: Página oficial de MySQL seleccionando la descarga del instalador ejecutable (MSI) para Windows.

Descarga del Instalador Oficial

Accedemos al portal oficial de descargas de MySQL y seleccionamos el instalador ejecutable (MSI) para el sistema operativo Windows.
Descargar, instalar y configurar, seguir el asistente de instalación

Ajuste de Inicio del Servicio en Windows

Para optimizar el rendimiento del sistema local, buscamos la ventana de **Servicios** de Windows a través del menú de inicio.
Desactivar el inicio o ejecución automática al iniciar el PC

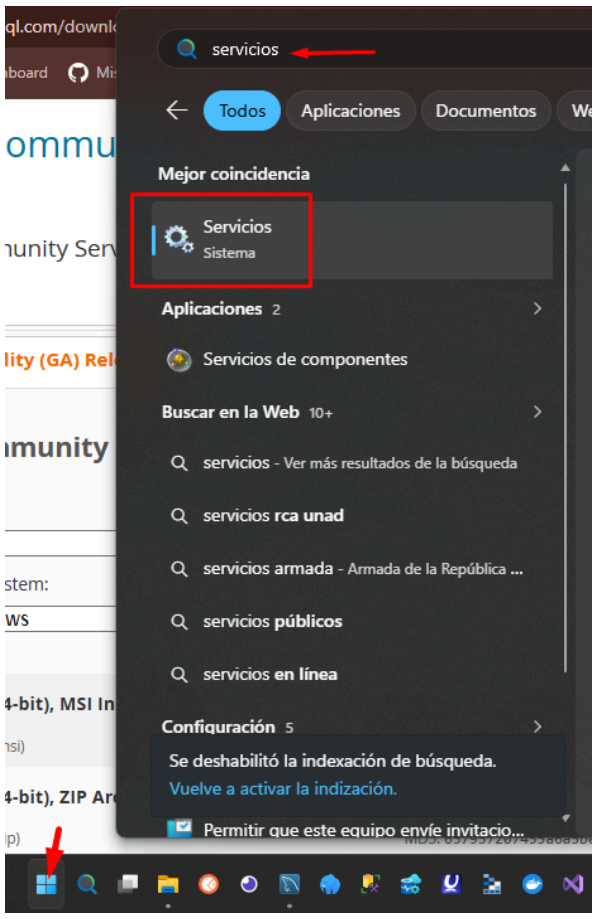


Imagen 28: Búsqueda de la herramienta de administración "Servicios" en el menú de inicio de Windows.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

configuración del Tipo de Inicio Manual

Localizamos el servicio **MySQL97** (o equivalente según la versión), hacemos clic derecho, seleccionamos **Propiedades** y cambiamos el tipo de inicio a **Manual**. Esto evitará que el servicio consuma recursos al iniciar la computadora de forma automática.

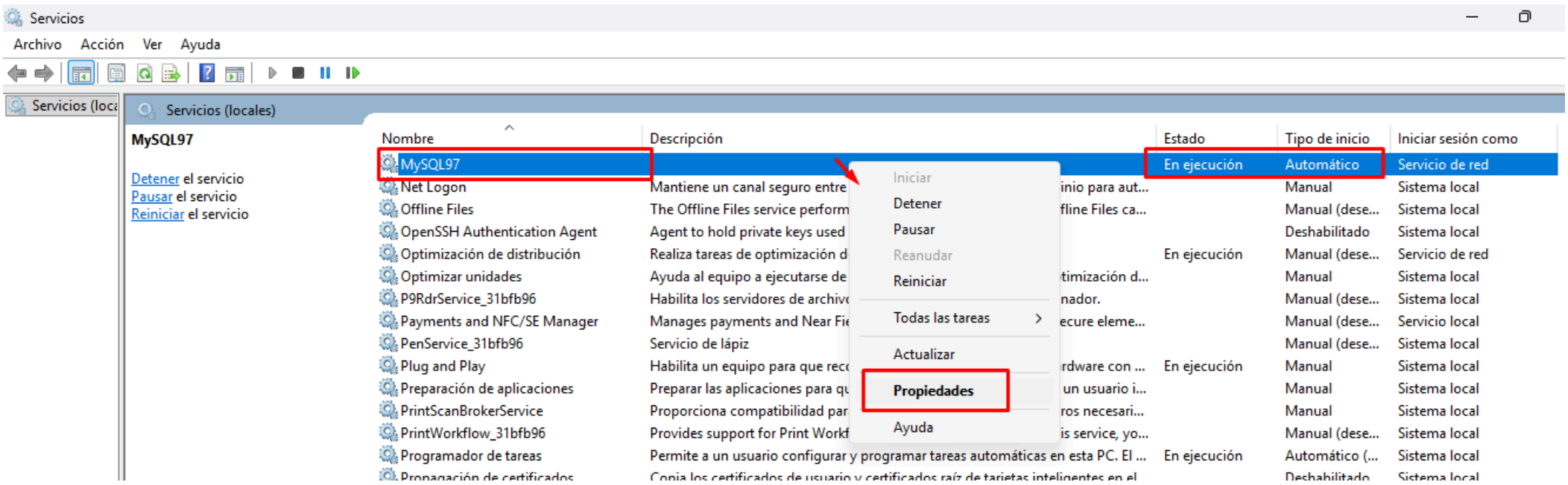


Imagen 29: Menú contextual sobre el servicio MySQL97 para acceder a sus propiedades del sistema.

Reinicio del Servicio de Base de Datos

Guardamos los cambios aplicados y seleccionamos la opción **Reiniciar** para asegurarnos de que el motor de base de datos se encuentre ejecutándose activamente.

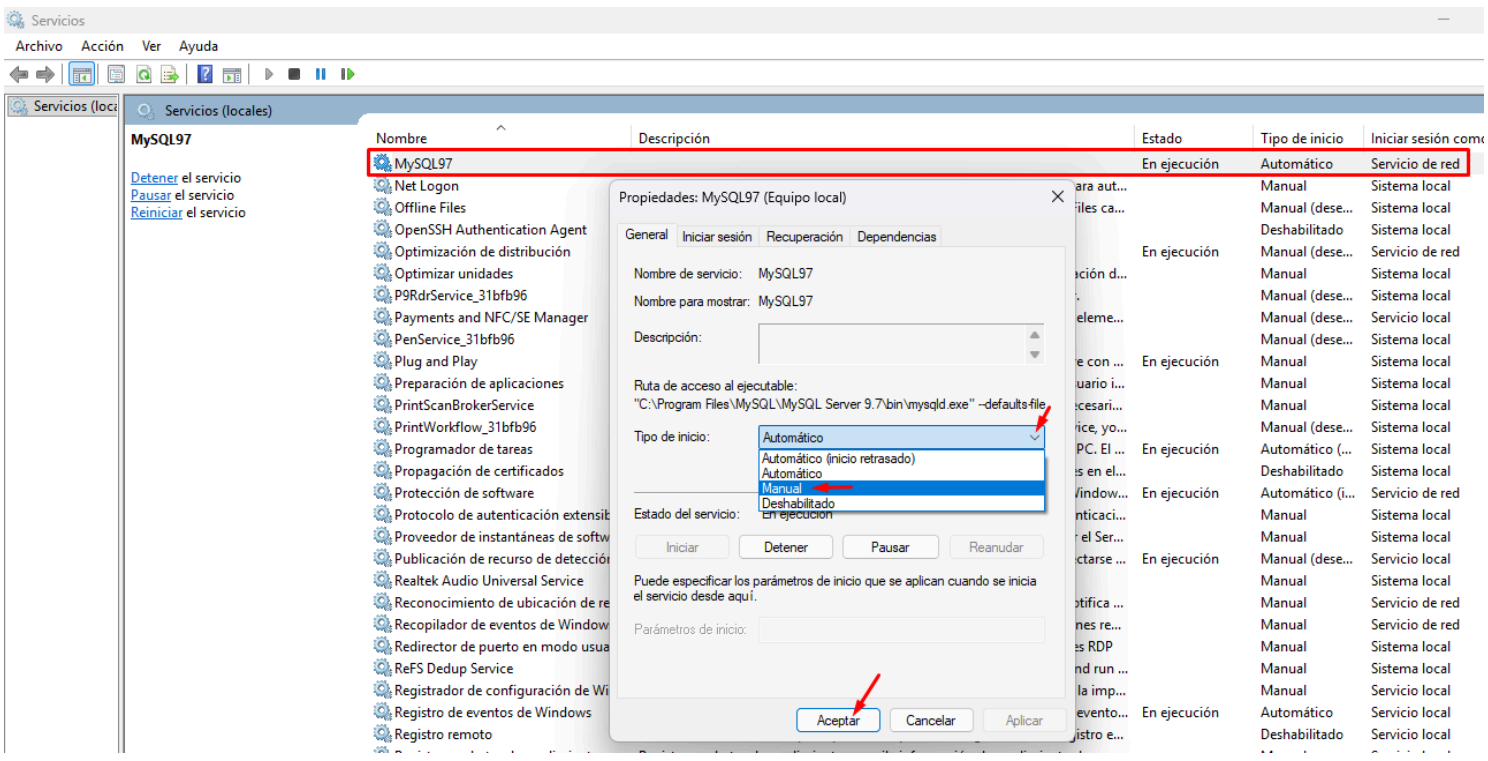


Imagen 30: Modificación del tipo de inicio a "Manual" en las propiedades del servicio MySQL.

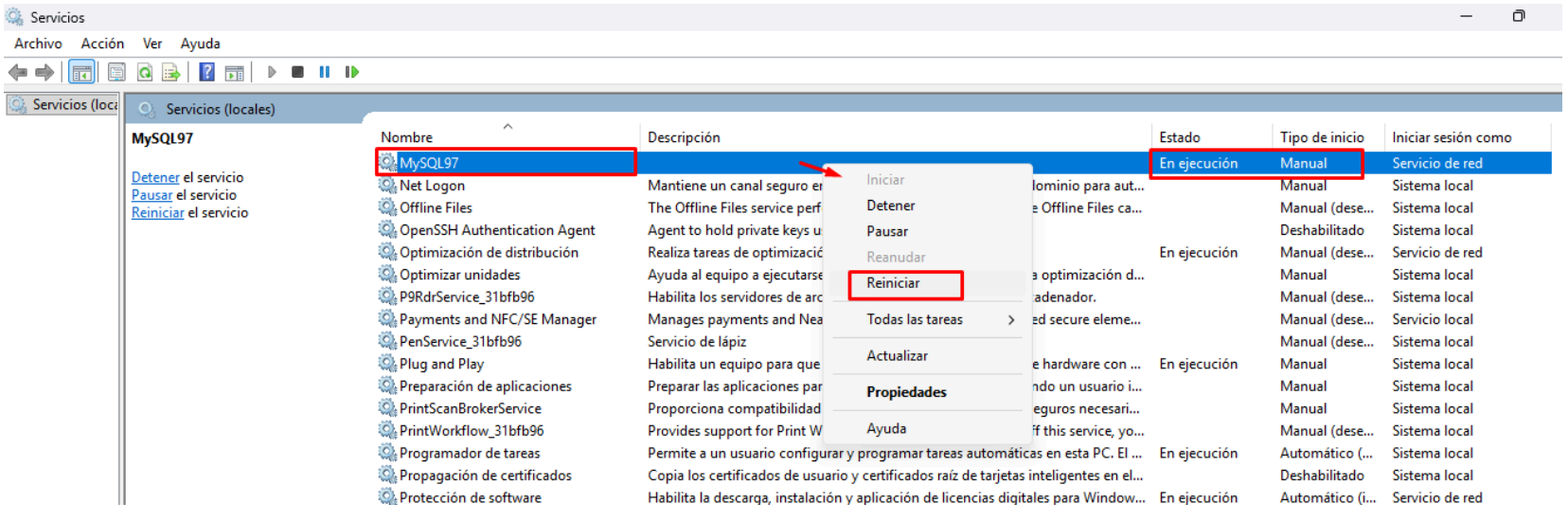


Imagen 31: Opción de reinicio del servicio de base de datos para validar su arranque correcto.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

---

5. Conexión y Ejecución de Scripts SQL en el IDE

En esta fase configuraremos el driver conector JDBC y estableceremos comunicación entre el entorno de desarrollo y la base de datos MySQL.

Conectar el IDE con MySQL Server

Búsqueda del Conector MySQL JDBC

Navegamos al repositorio central de Maven (MVN Repository) para buscar la dependencia oficial del conector de MySQL (**mysql-connector-j**).

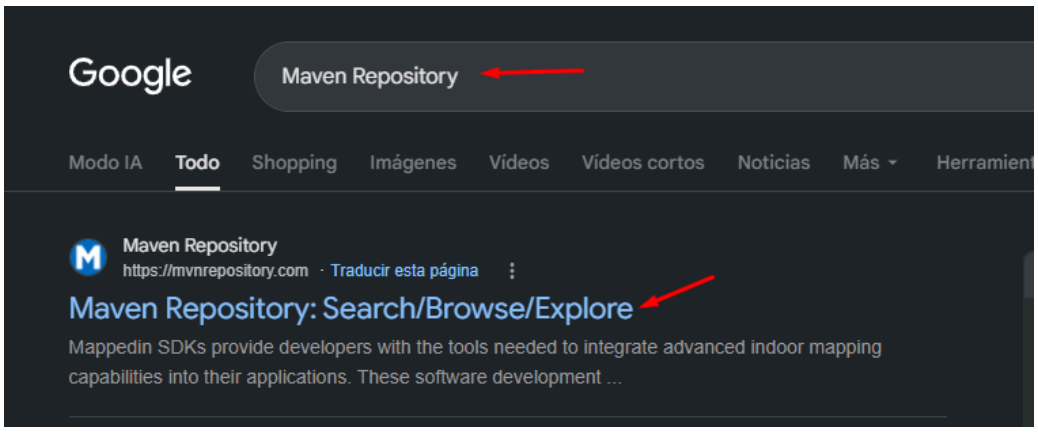


Imagen 32: Búsqueda del portal Maven Repository en el navegador web.

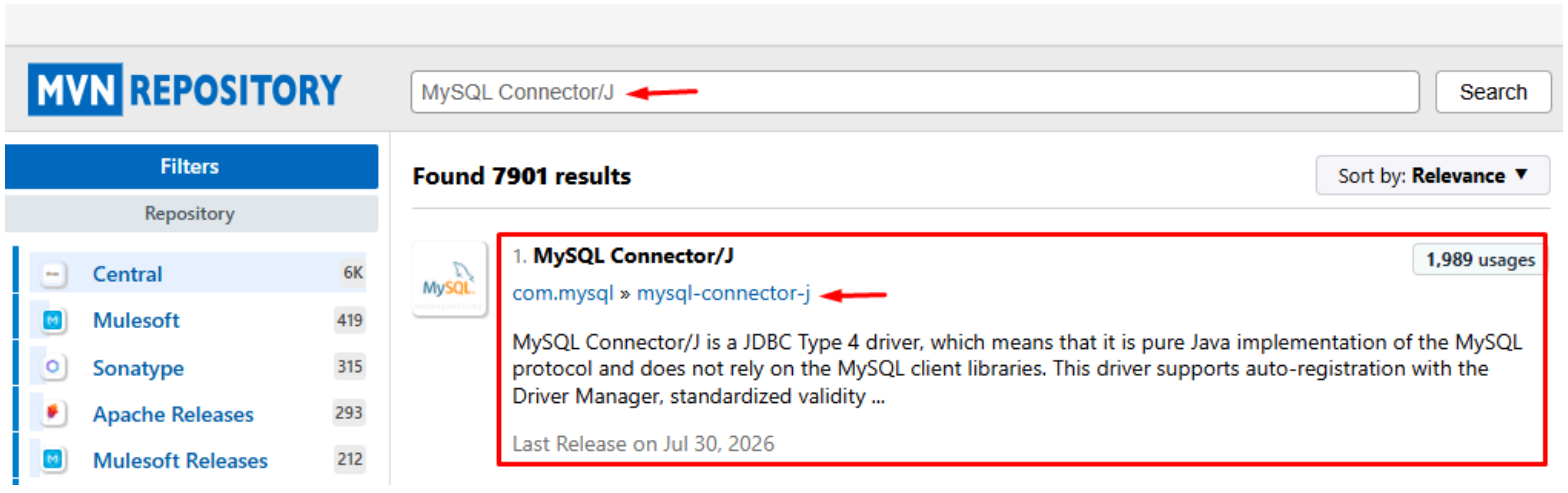


Imagen 33: Selección del conector MySQL (`com.mysql » mysql-connector-j`) en los resultados de Maven Repository.

Descarga del Archivo JAR

Seleccionamos la versión compatible más reciente del conector y descargamos directamente el archivo comprimido **.jar** a nuestro equipo.

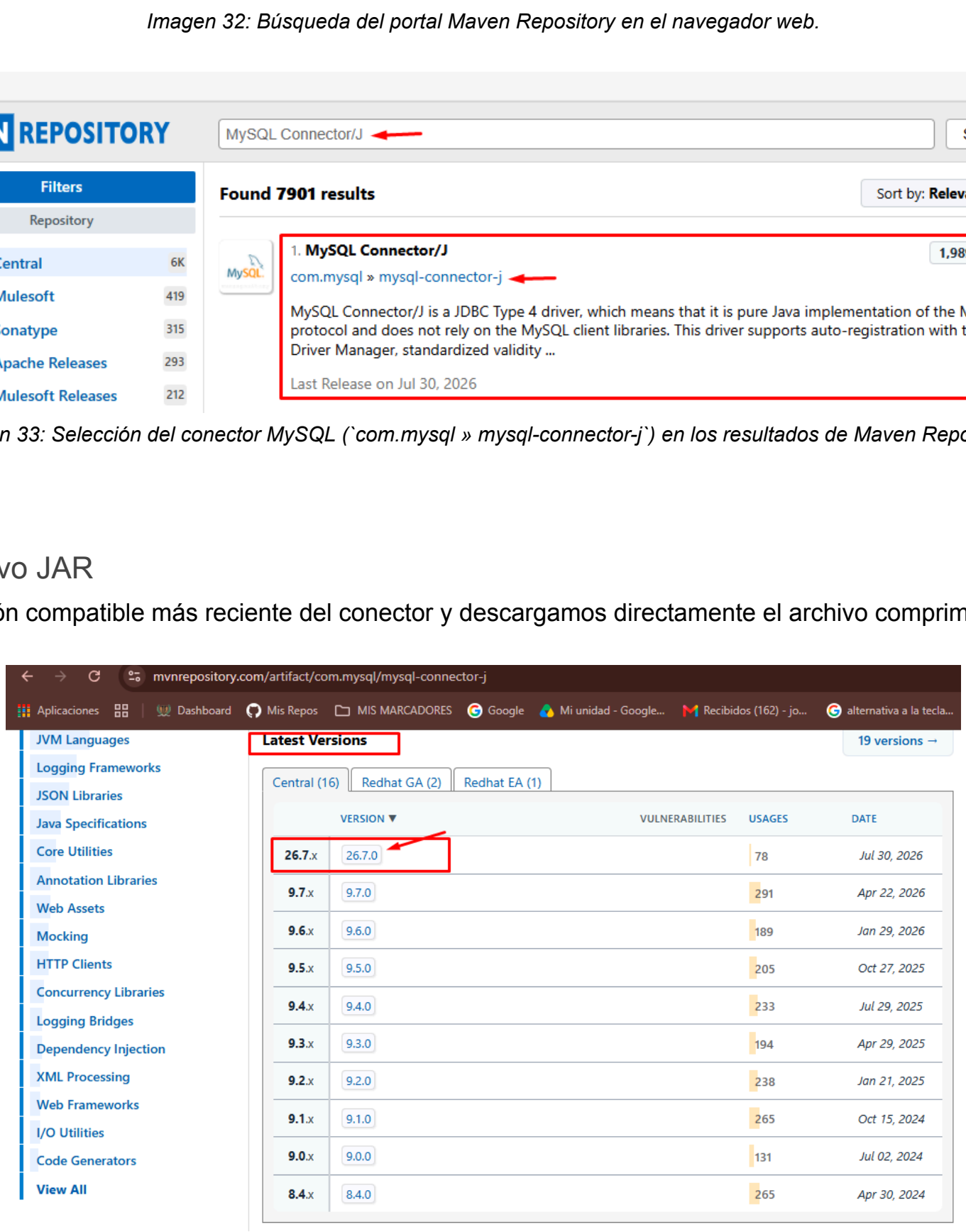


Imagen 34: Listado de versiones disponibles del conector JDBC en el repositorio.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

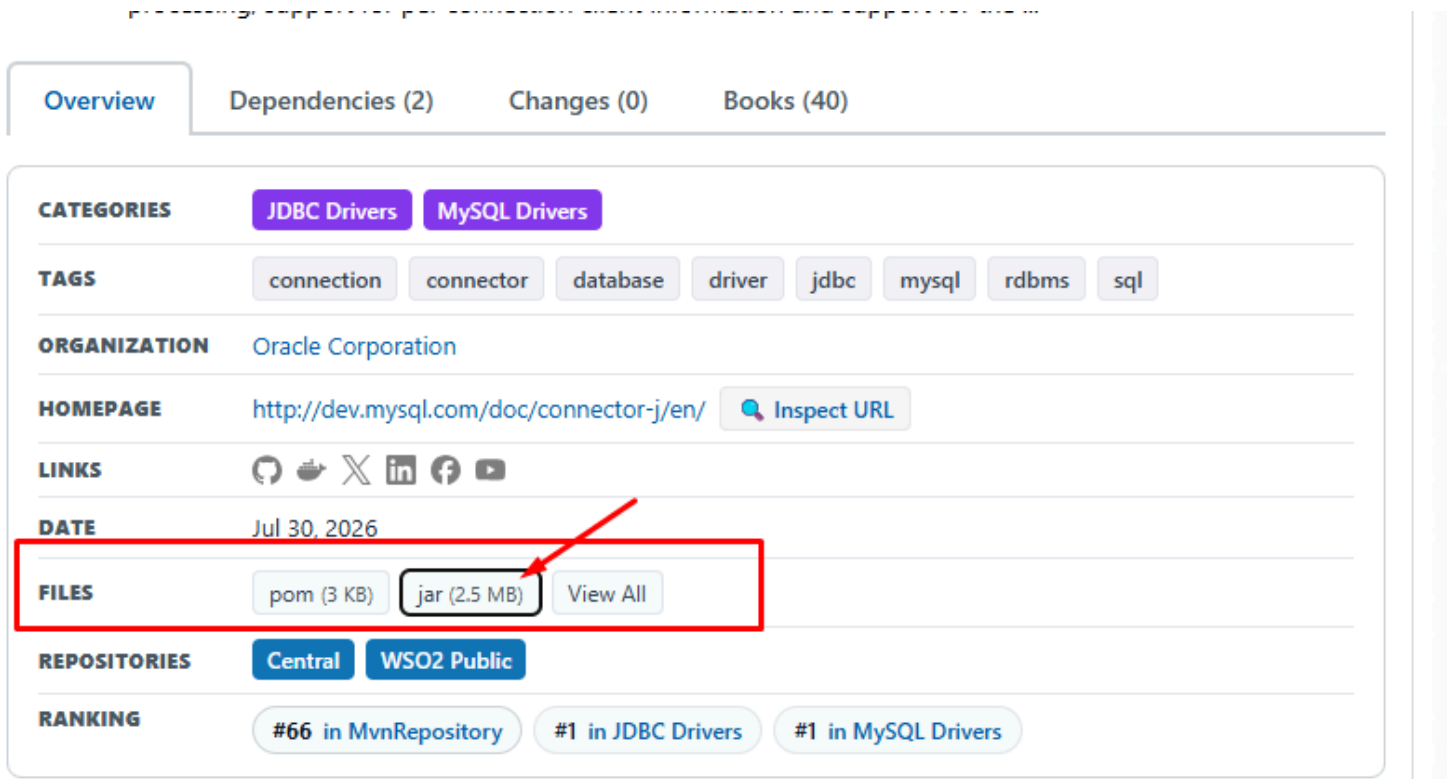


Imagen 35: Enlace directo de descarga del archivo .jar de la versión del driver seleccionada.

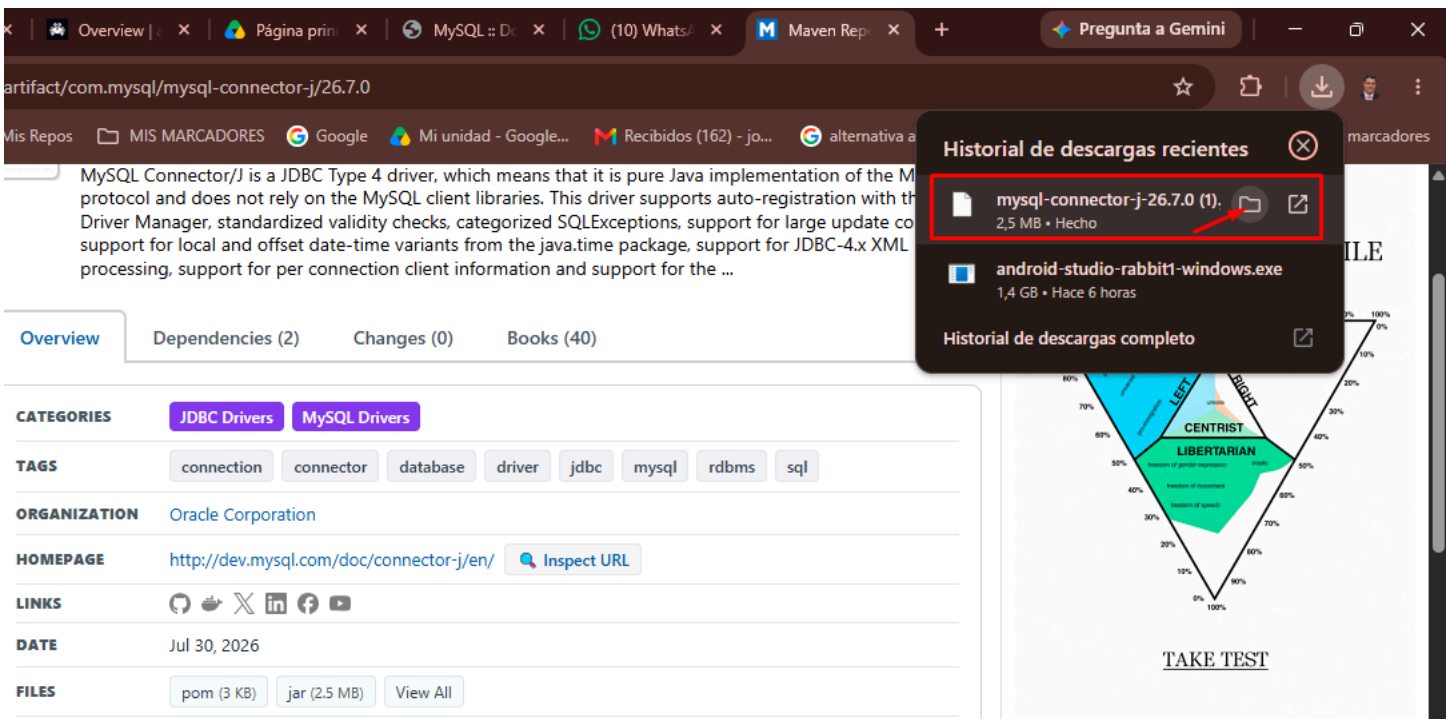


Imagen 36: Descarga finalizada del controlador de MySQL en la carpeta local de descargas.

configuración del Driver de Base de Datos en IntelliJ

En el panel lateral **Database** del IDE, añadimos un nuevo driver personalizado seleccionando la opción **Driver** y vinculando el archivo JAR descargado previamente.

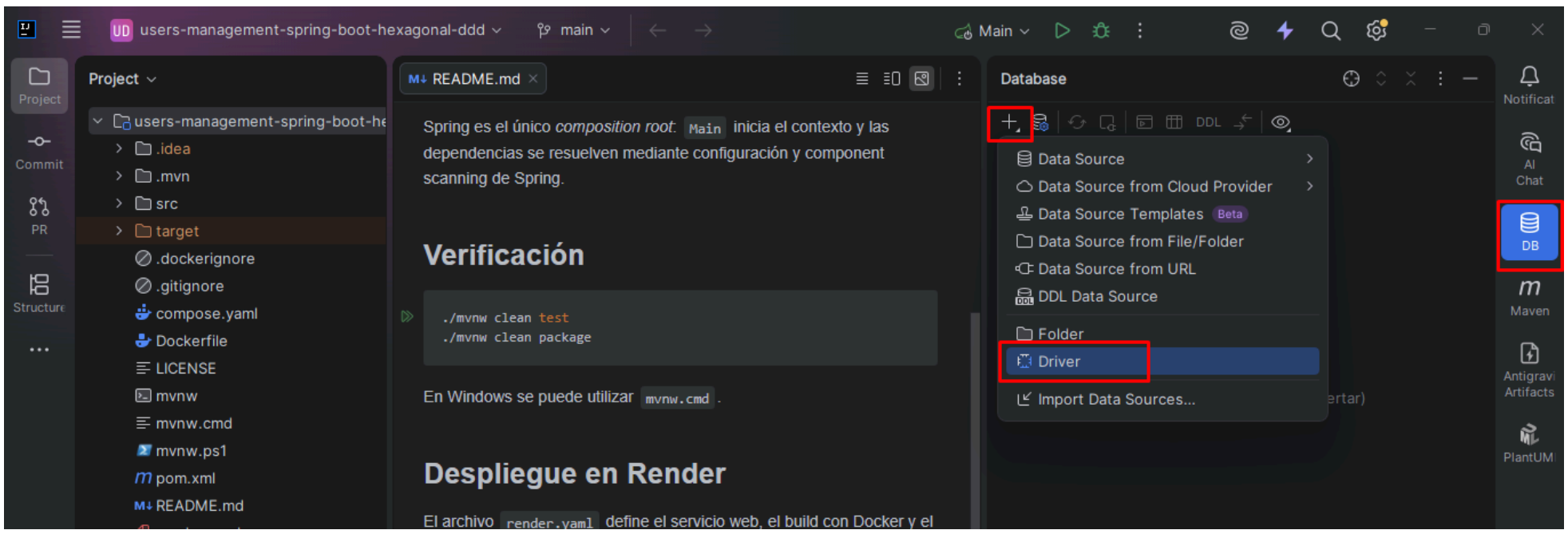


Imagen 37: Panel "Database" en IntelliJ IDEA seleccionando la opción para agregar un nuevo Driver de base de datos.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

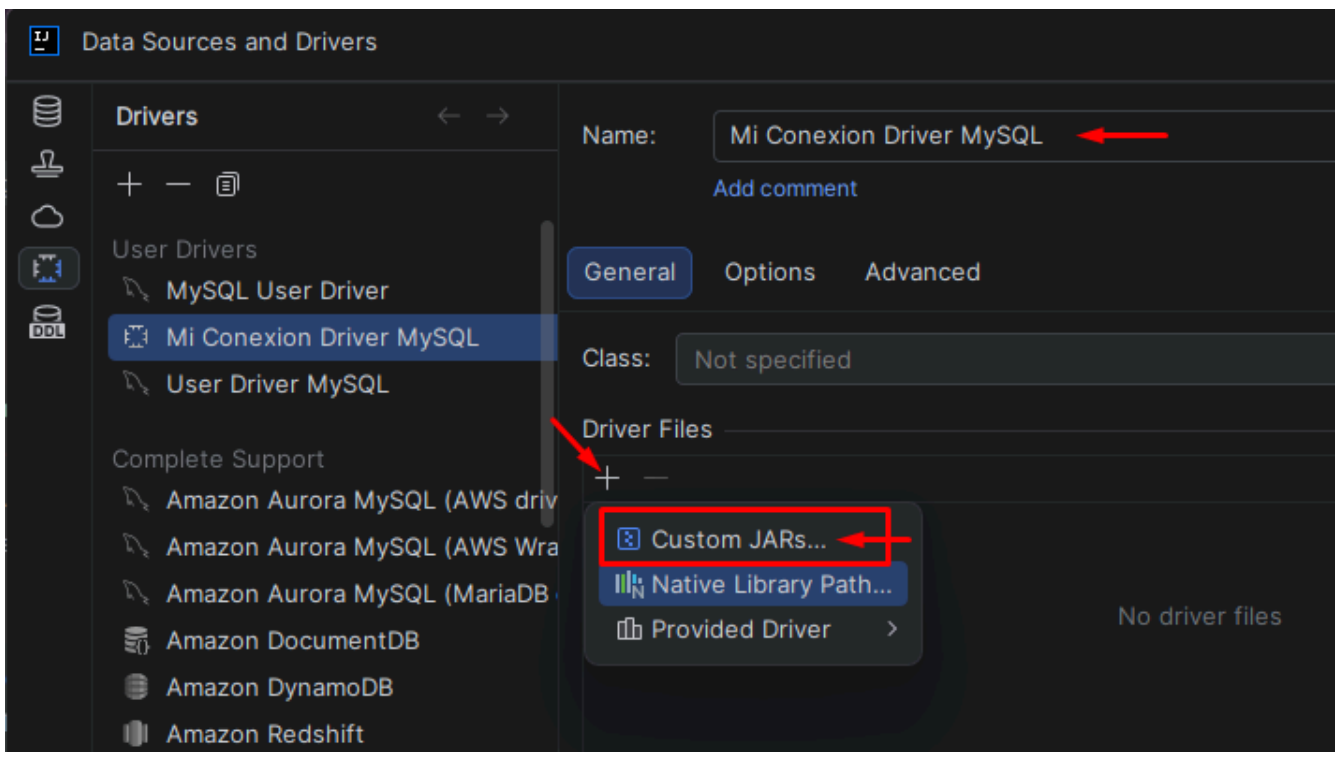


Imagen 38: Menú de configuración para importar el archivo JAR descargado mediante "Custom JARs...".

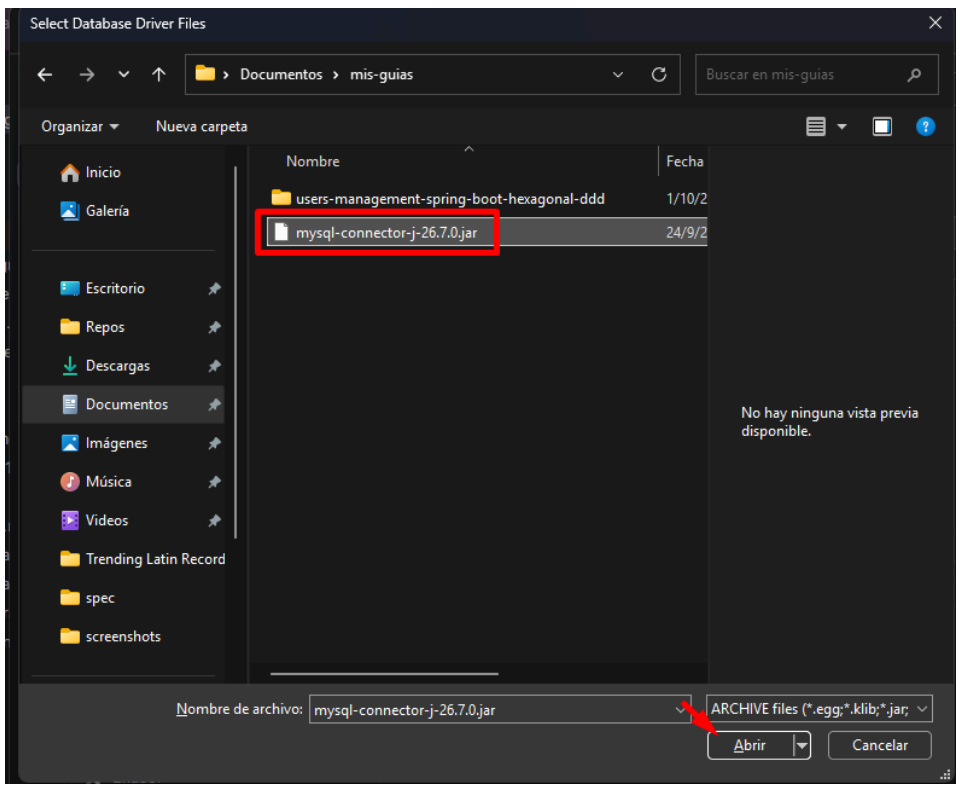


Imagen 39: Selección del archivo `mysql-connector-j-26.7.0.jar` en la ventana del explorador.

Creación de la Fuente de Datos (Data Source)

Creamos una nueva conexión seleccionando **Data Source > MySQL** e ingresamos los parámetros de autenticación local (Host: **localhost**, Puerto: **3306**, Usuario: **root** y la contraseña configurada durante la instalación).

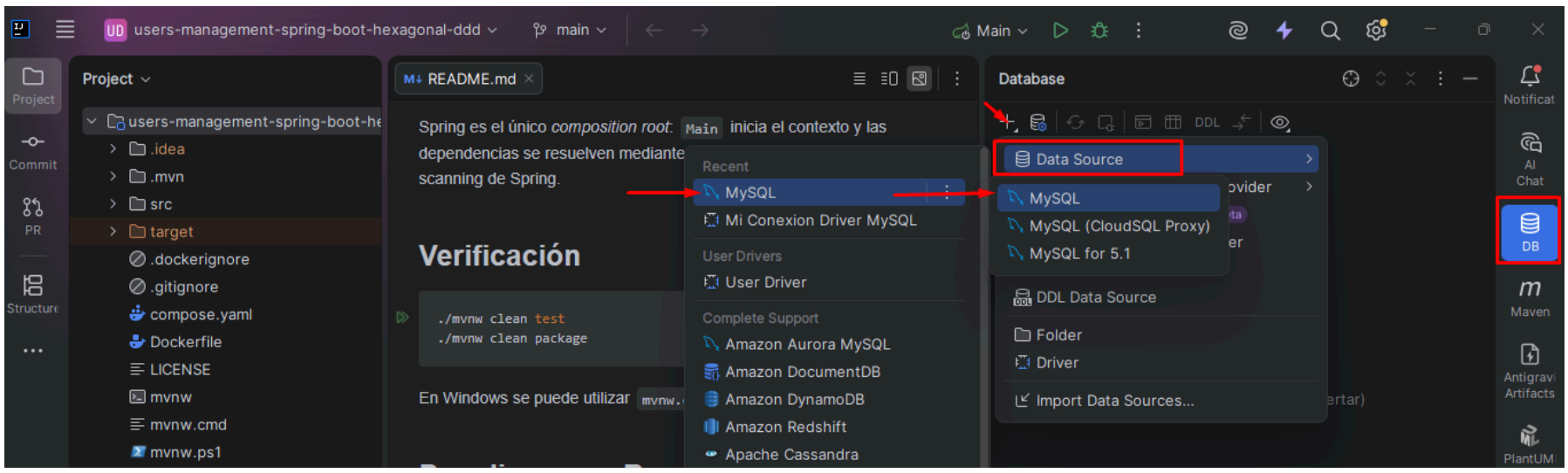


Imagen 40: Selección del motor MySQL dentro de las opciones de Data Source en el IDE.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

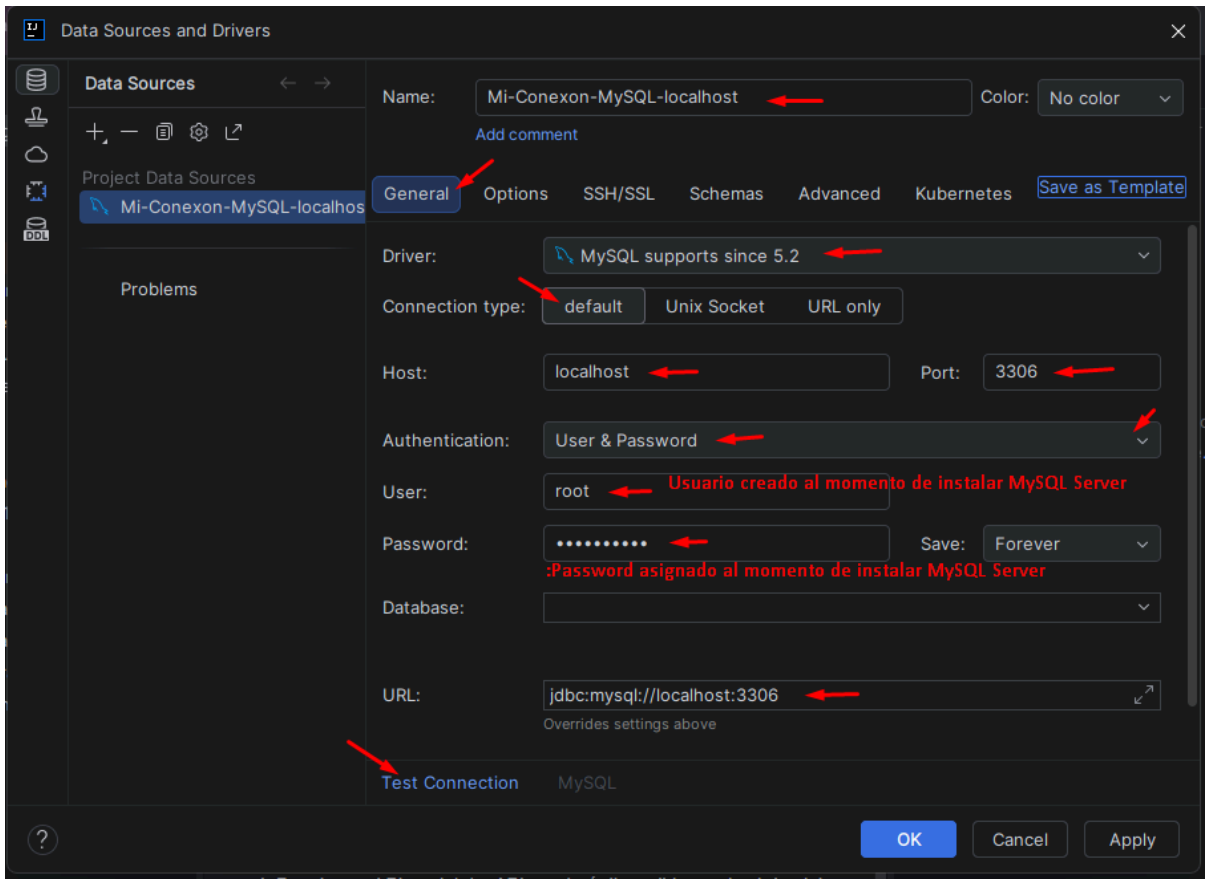


Imagen 41: Formulario de configuración de parámetros de conexión a MySQL local (Host, Port, User, Password).

Prueba de Conexión Exitosa

Presionamos sobre el botón **Test Connection** y confirmamos que la prueba devuelva un estado exitoso (Succeeded), verificando la comunicación fluida con la base de datos.

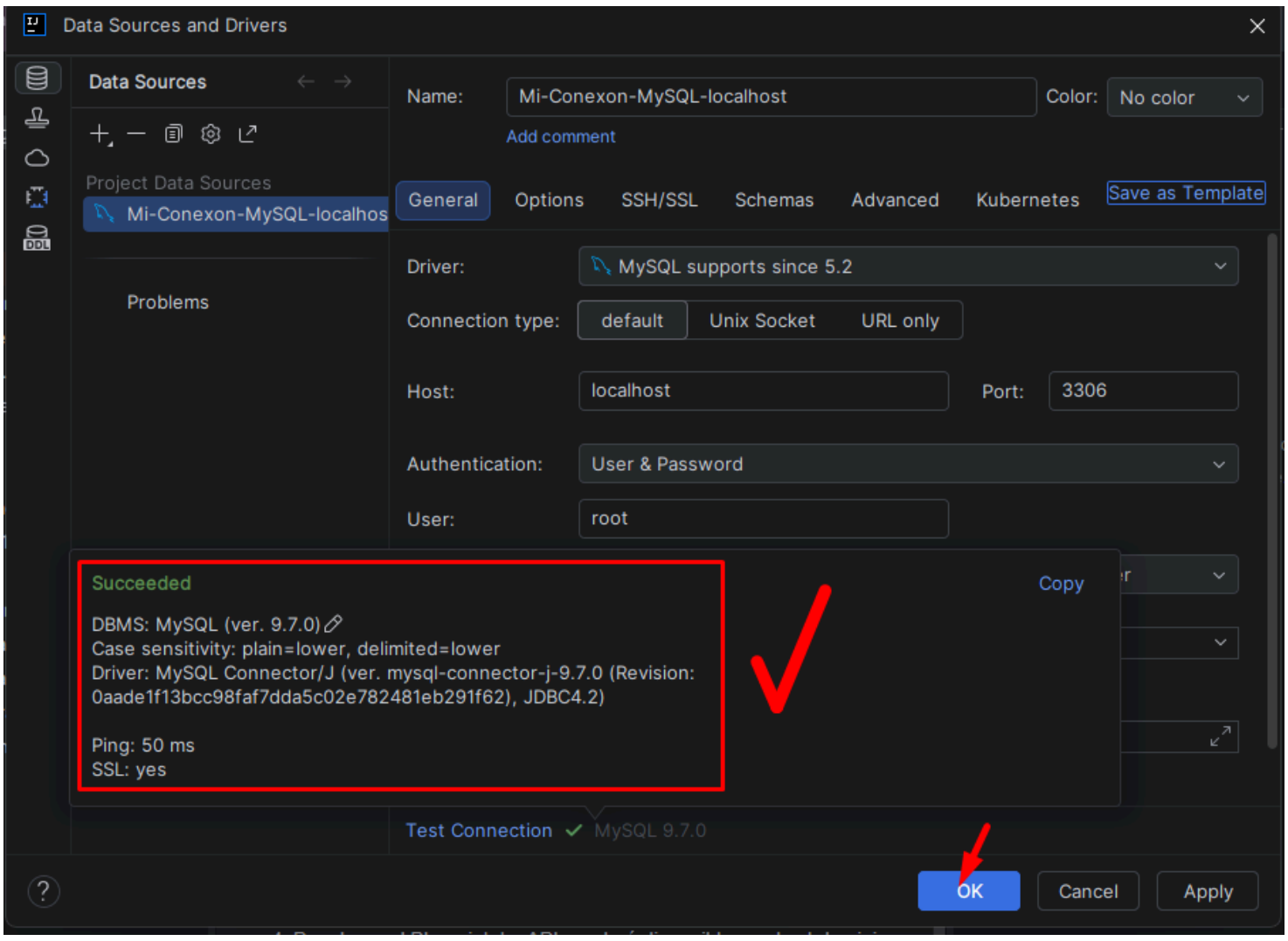


Imagen 42: Confirmación de prueba de conexión satisfactoria ("Succeeded") conectando a MySQL.

Apertura del Script SQL de Estructura (schema.sql)

Abrimos el archivo **schema.sql** ubicado en la carpeta **src/main/resources**, el cual contiene las instrucciones DDL para la creación de la base de datos **crud_usuarios**, sus tablas asociadas y la inserción de registros iniciales.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

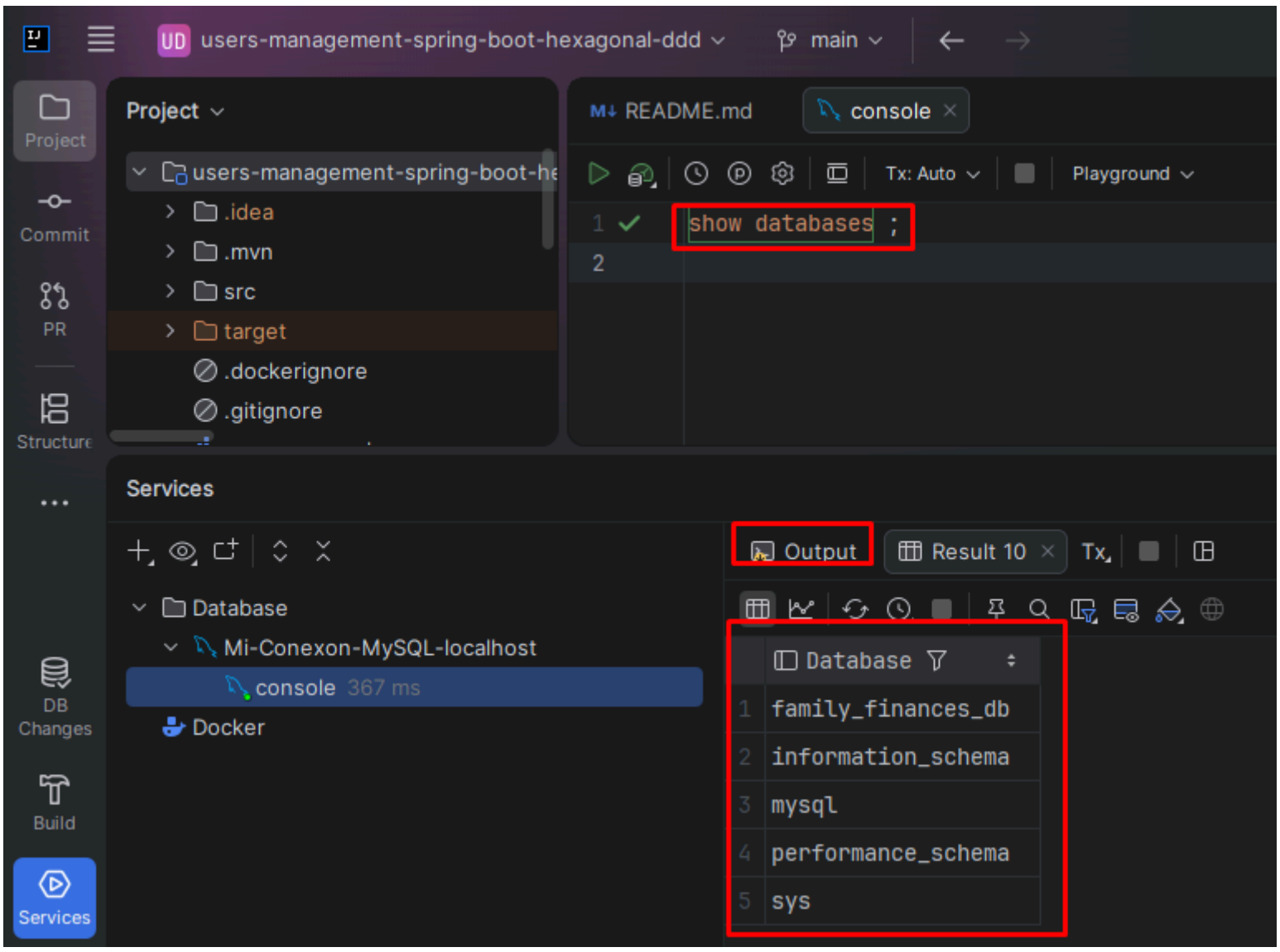


Imagen 43: Consola de consultas SQL ejecutando la instrucción `show databases;` para listar las esquemas existentes.

Ejecución del Script en la Conexión Activa

Vinculamos la consola de ejecución con la conexión activa, seleccionamos todas las sentencias SQL y presionamos el botón de ejecución (Play) para estructurar la base de datos en local.

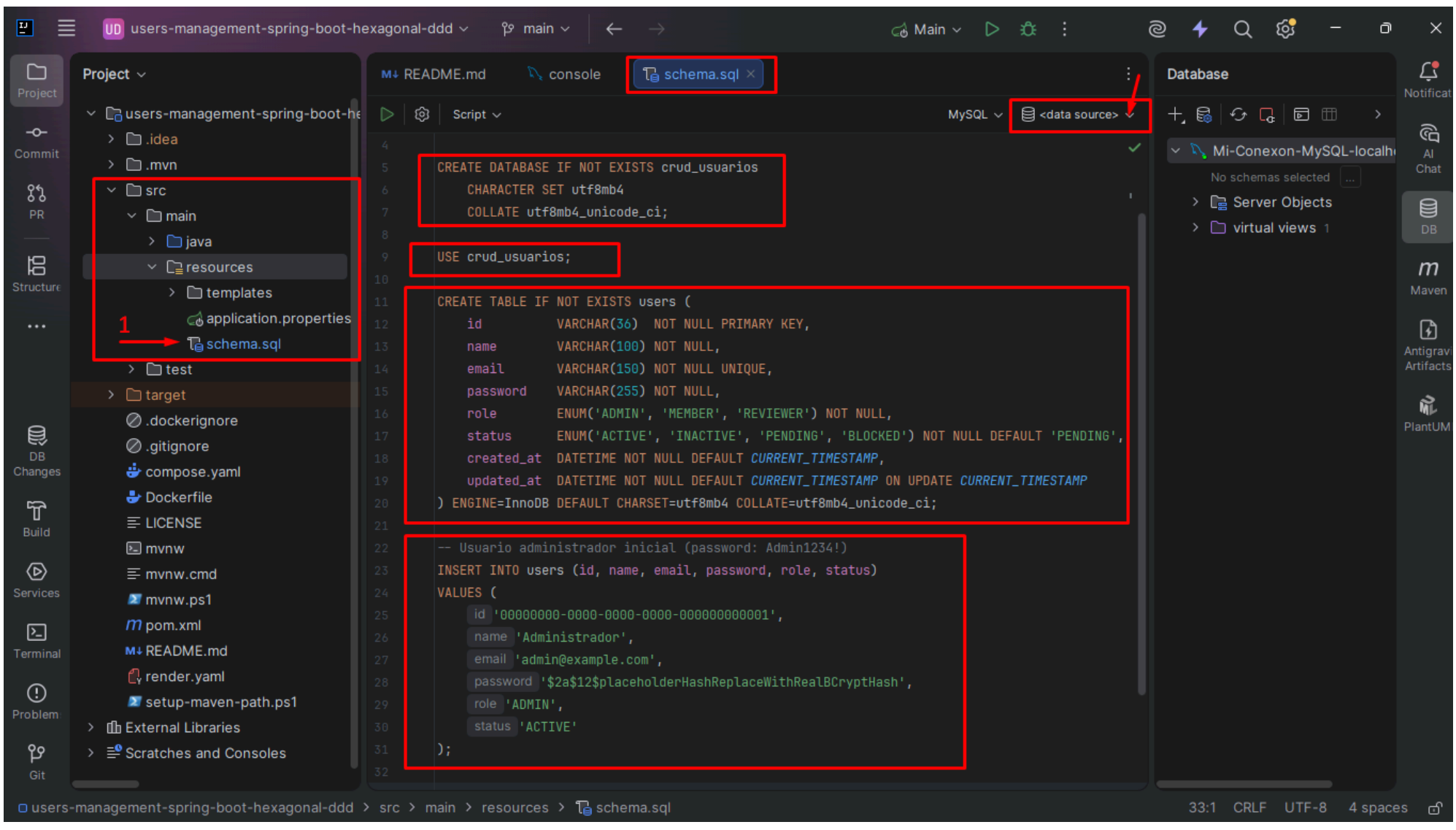


Imagen 44: Vista del archivo `schema.sql` en el editor listo para ser vinculado con la base de datos.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

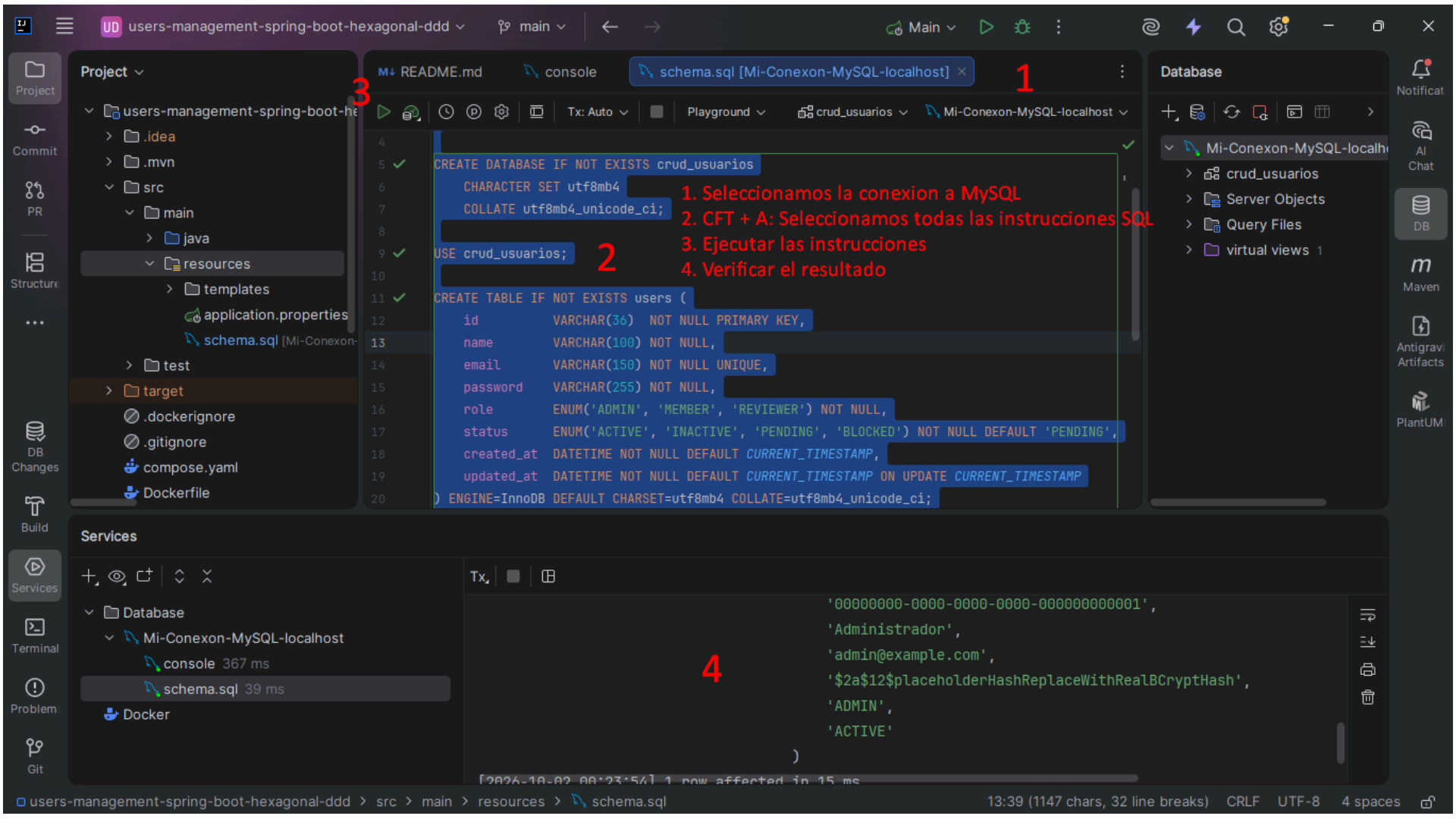


Imagen 45: Selección y ejecución de todas las instrucciones SQL para la creación del esquema y registros iniciales.

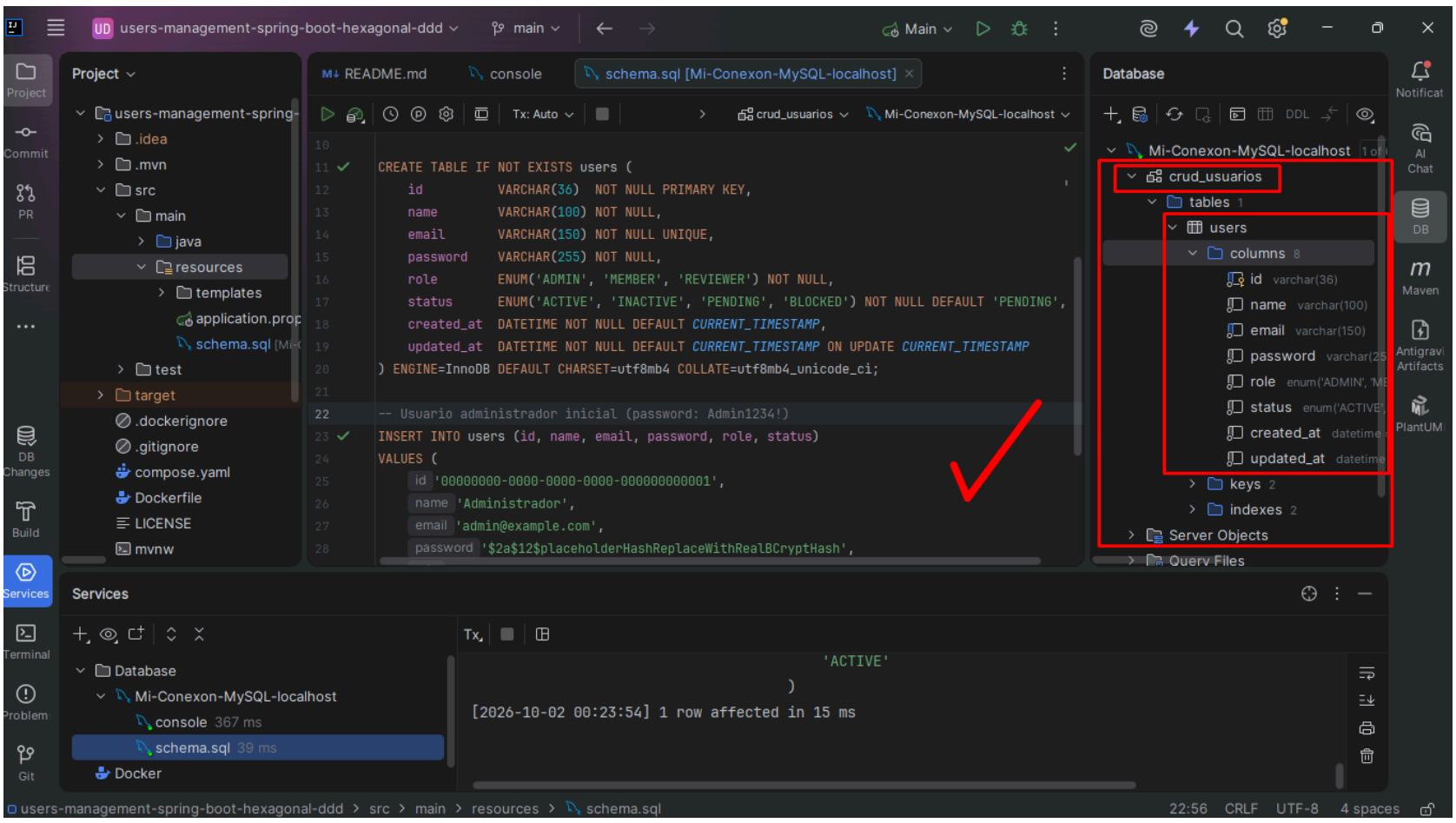


Imagen 46: Árbol de la base de datos en el IDE mostrando la tabla `users` creada con su estructura de columnas.

Verificación de Tablas Creadas

Exploramos el árbol del panel **Database** para constatar que la base de datos **crud_usuarios** y la tabla **users** con sus respectivas columnas hayan sido generadas correctamente.

6. configuración de Archivos del Proyecto (application.properties)

En esta sección configuraremos las variables de entorno necesarias para establecer la conexión a la base de datos, el servidor de correo electrónico SMTP y la seguridad mediante tokens JWT.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Edición de Parámetros de Base de Datos

Accedemos al archivo **application.properties** y actualizamos los campos referentes a la URL, usuario y contraseña de MySQL local, asegurando la concordancia con las credenciales de nuestro motor de base de datos.

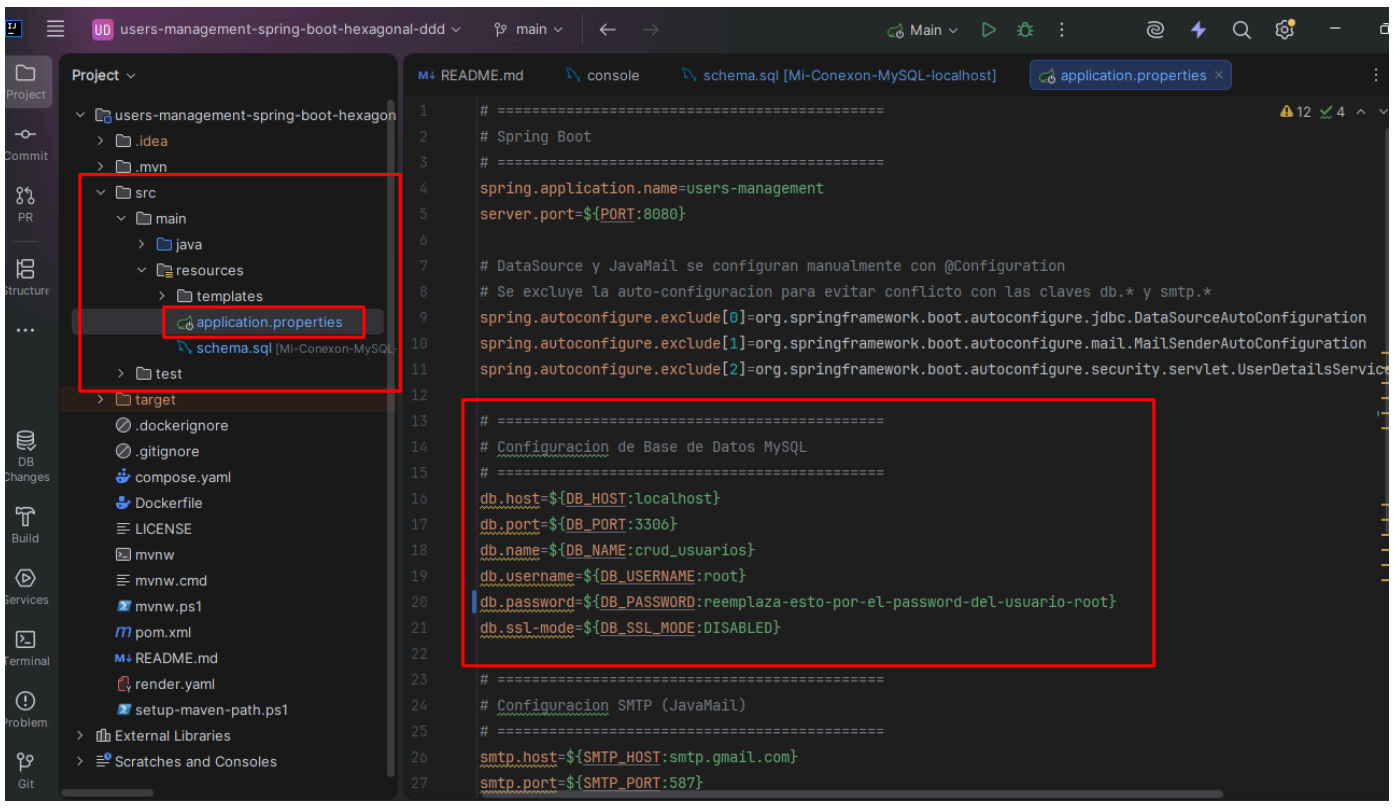


Imagen 47: Vista del archivo `application.properties` editando los parámetros de conexión a MySQL local.

Generación de Contraseña de Aplicación de Google (SMTP)

Para habilitar el envío automático de correos (notificaciones de bienvenida o restablecimiento), nos dirigimos a la configuración de seguridad de nuestra cuenta de Google y activamos la **Verificación en 2 pasos**.

configurar los parámetros que la aplicación usará para comunicarse con el servidor SMTP que enviará los correos de notificaciones

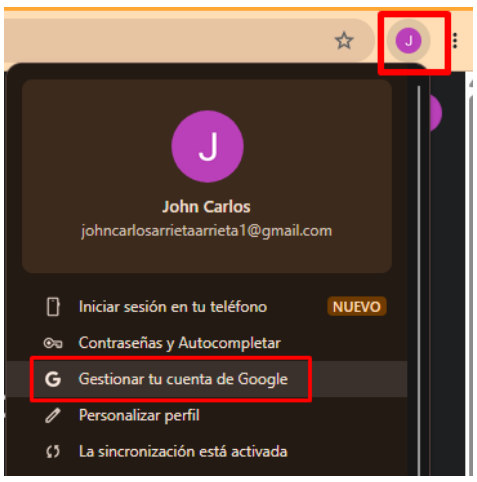


Imagen 48: Menú de cuenta de Google para acceder a la gestión de seguridad del usuario.

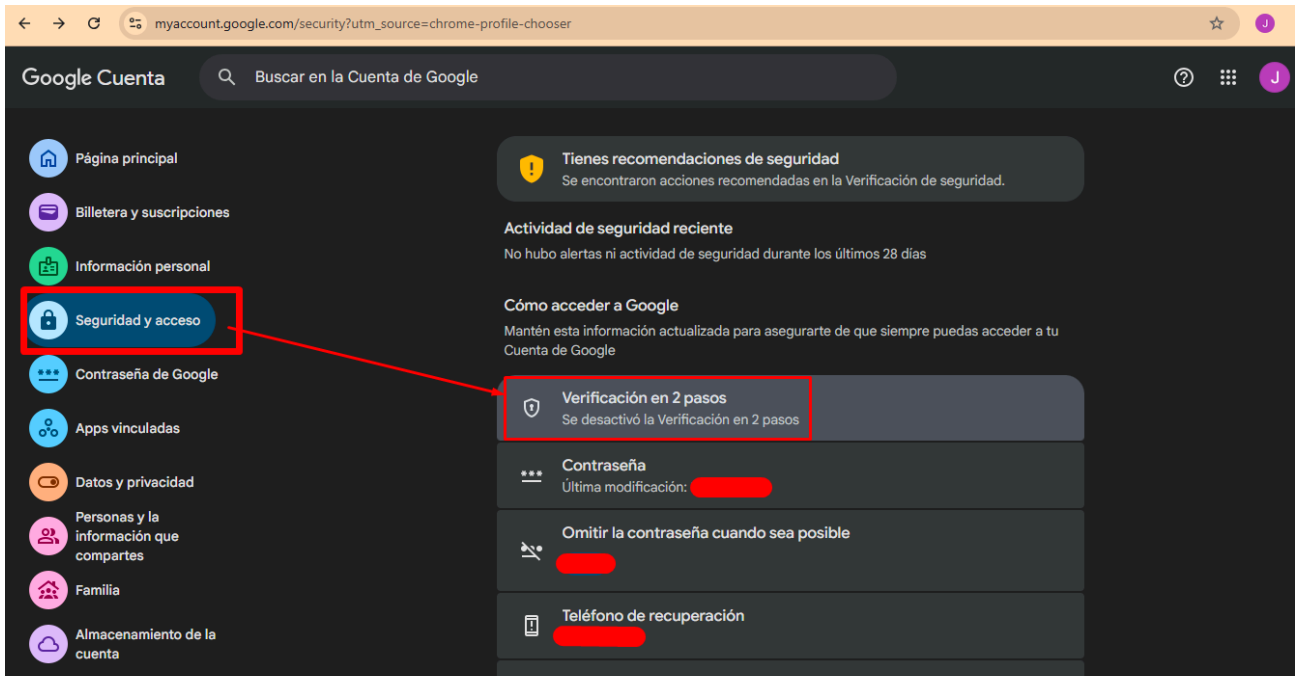


Imagen 49: Sección "Seguridad y acceso" en la cuenta de Google verificando la opción de Verificación en 2 pasos.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:**

Creación de una Clave de Aplicación Específica

En el apartado de **Contraseñas de aplicaciones**, generamos un nuevo token asignándole un nombre descriptivo (por ejemplo, **Clave-para-enviar-correos**).

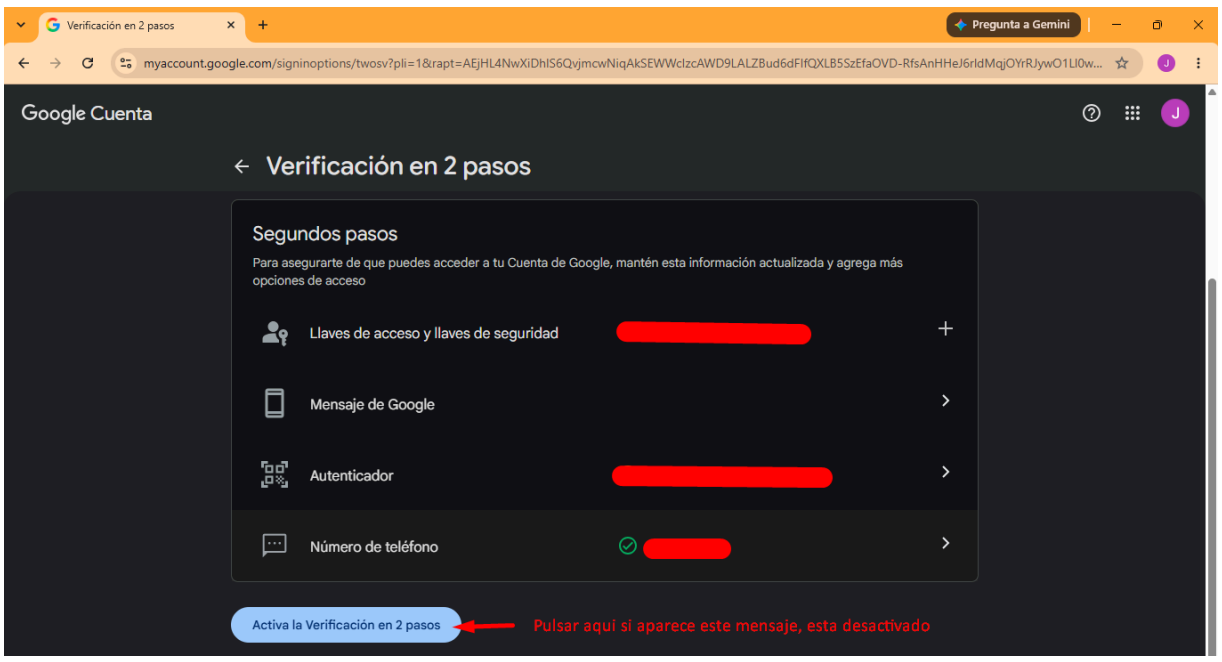


Imagen 50: Pantalla de activación de Verificación en 2 pasos dentro de la cuenta de Google.

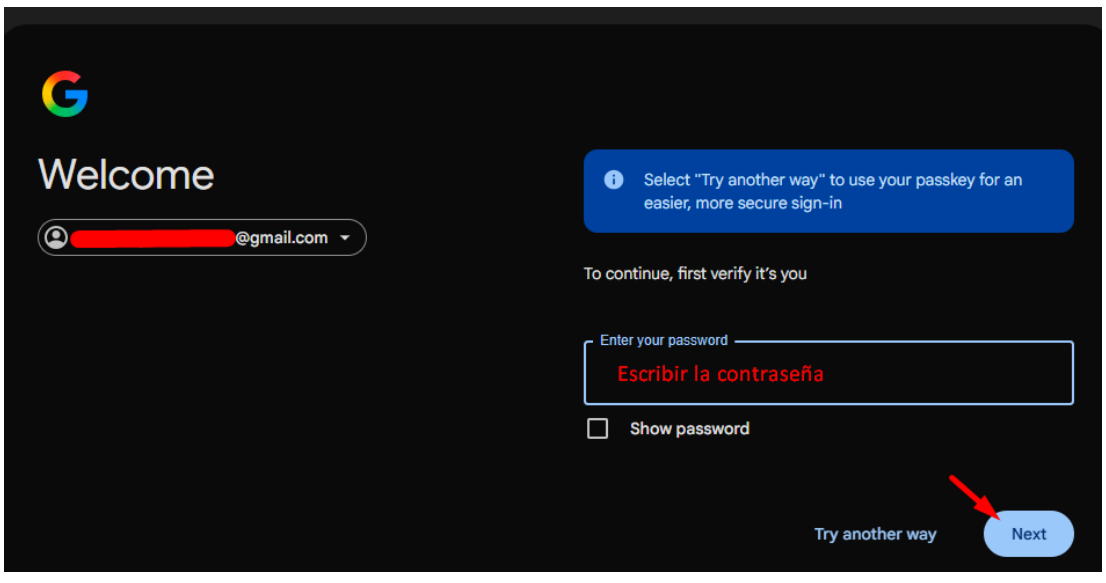


Imagen 51: Confirmación de autenticación de seguridad en la cuenta de Google ingresando la contraseña personal.

Copia Segura de la Clave Generada

Copiamos el token de 16 caracteres generado por Google. Esta contraseña especial permitirá que la aplicación Spring Boot se autentique de forma segura frente al servidor SMTP de Gmail.

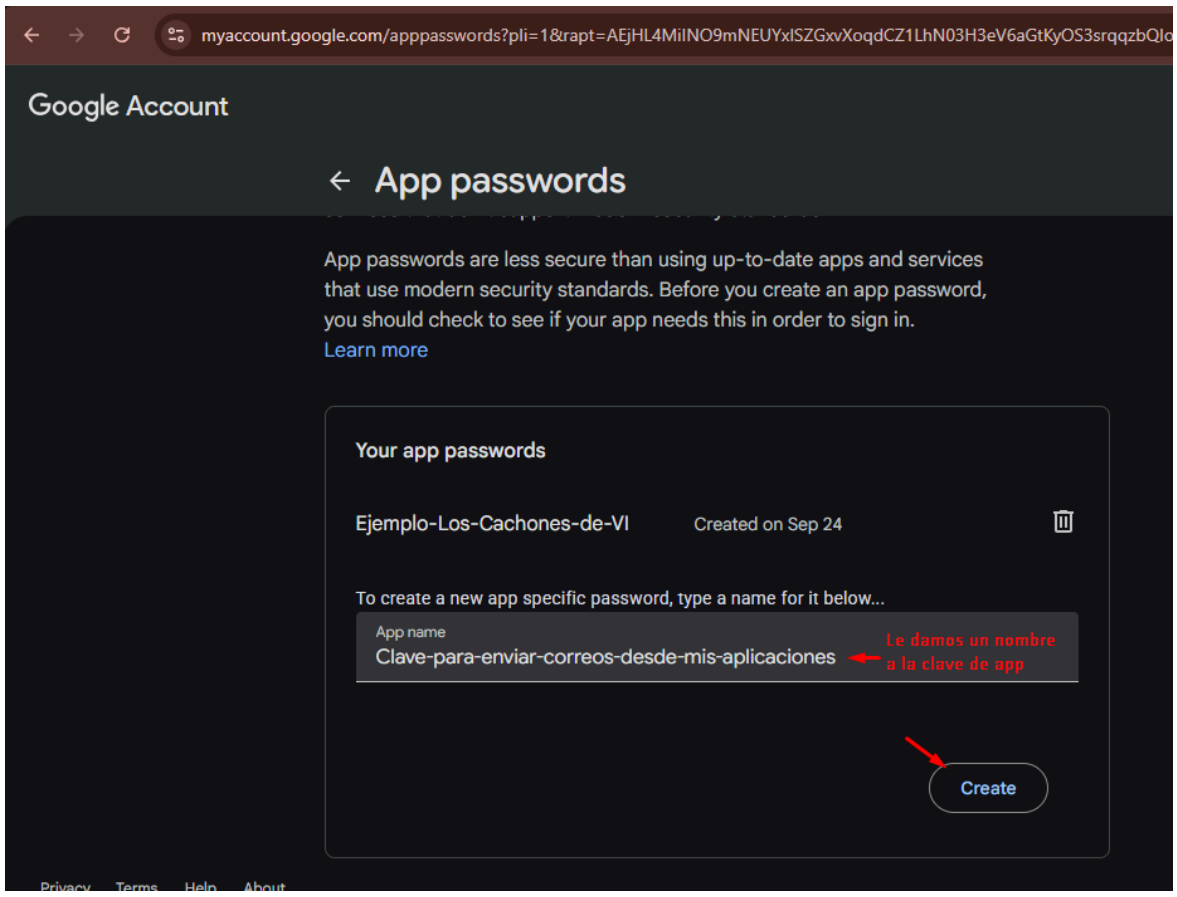


Imagen 52: Asignación de un nombre descriptivo para crear la contraseña de aplicación especial.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ©+ⓘ+↻

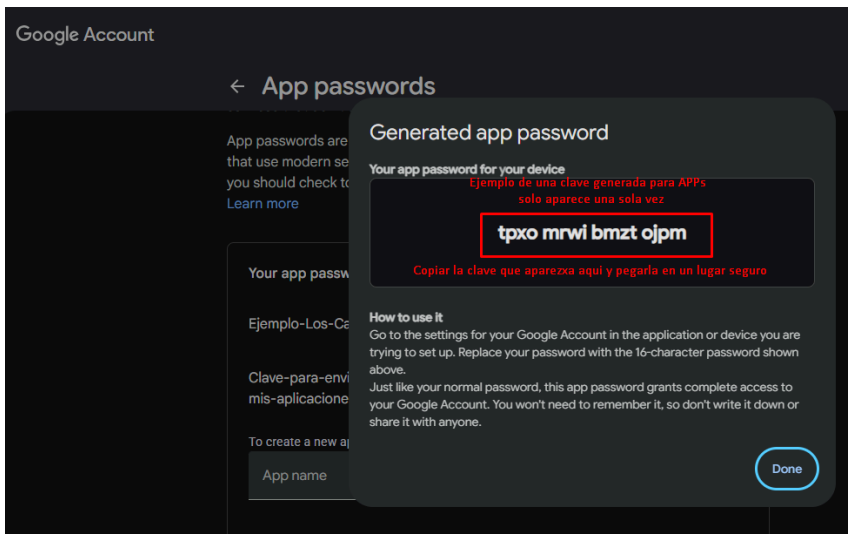


Imagen 53: Clave de aplicación generada por Google (16 caracteres) lista para ser copiada.

Asignación de Parámetros SMTP en el Proyecto

Pegamos la clave obtenida en la propiedad **smtp.password** del archivo **application.properties** y completamos los campos correspondientes a la cuenta de correo de origen.

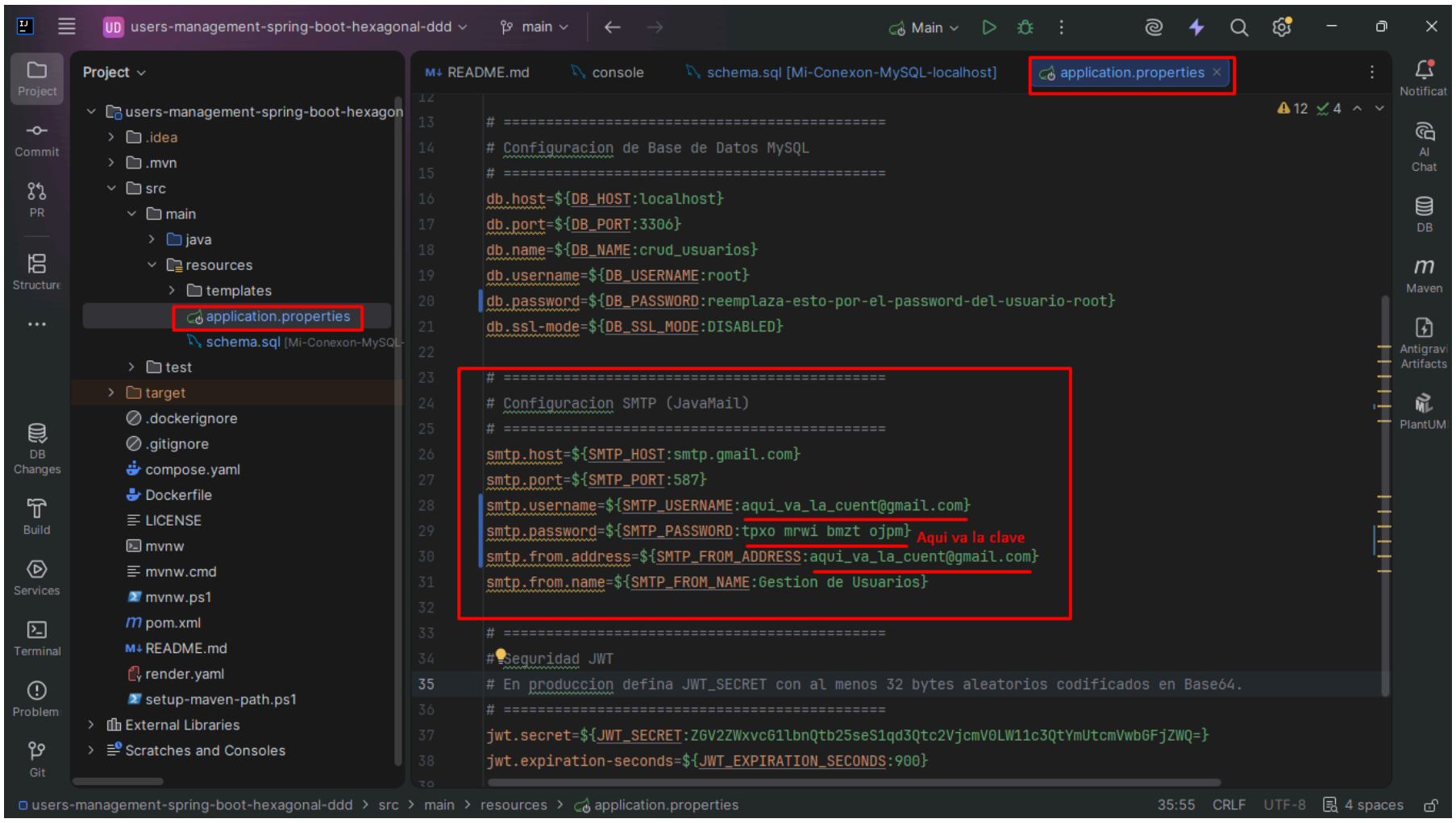


Imagen 54: configuración completa de los campos SMTP en `application.properties` incluyendo la clave generada.

7. Ejecución y Pruebas Locales del Servicio

Con todas las configuraciones establecidas, procedemos a iniciar el servidor embebido Tomcat para verificar la operatividad de la aplicación y sus endpoints.

Compilar, ejecutar y probar.

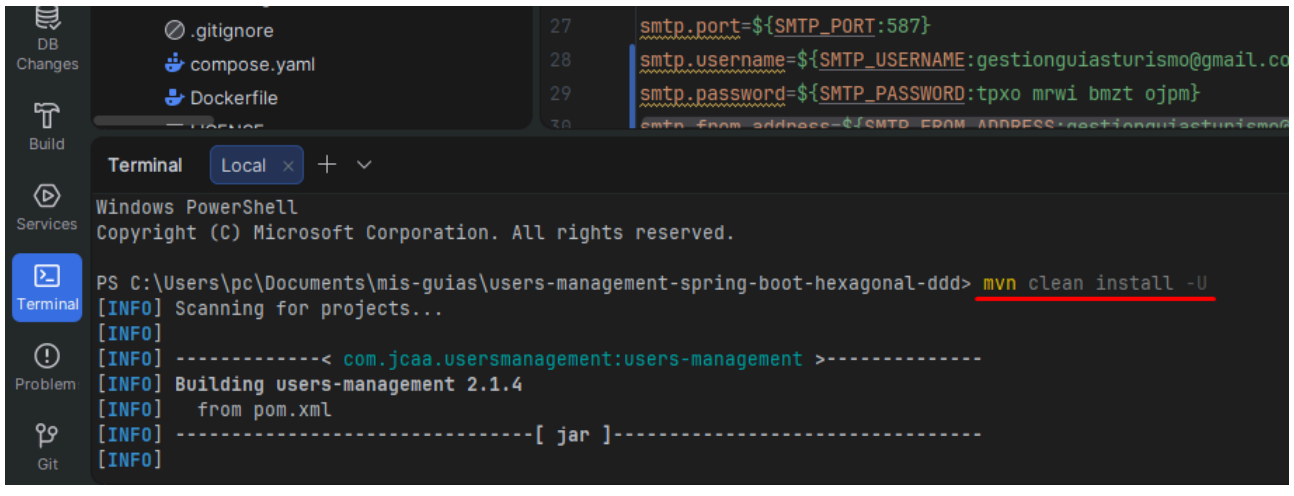


Imagen 55: Comando de compilación con Maven Wrapper ejecutándose en la terminal local.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

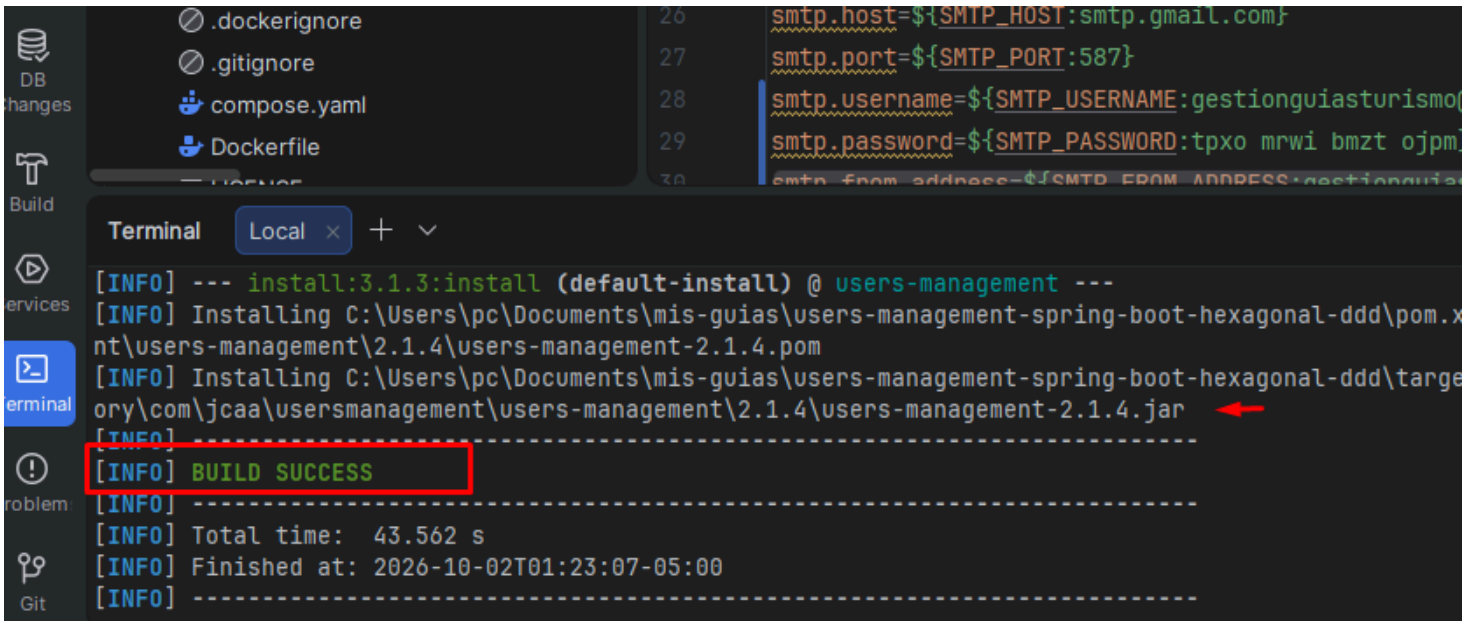


Imagen 56: Resultado exitoso del empaquetado JAR en la carpeta target.

Despliegue del Servicio con Spring Boot Run

En la terminal integrada del IDE, ejecutamos el comando **mvn spring-boot:run** para compilar e iniciar el servidor web embebido en el puerto por defecto (8080).

Imagen 57: Ejecución del comando `mvn spring-boot:run` para iniciar la aplicación servidor.

Verificación del Arranque del Servidor

Confirmamos la correcta inicialización revisando que los logs de la consola muestren que el servidor Tomcat se ha iniciado satisfactoriamente en el puerto 8080.

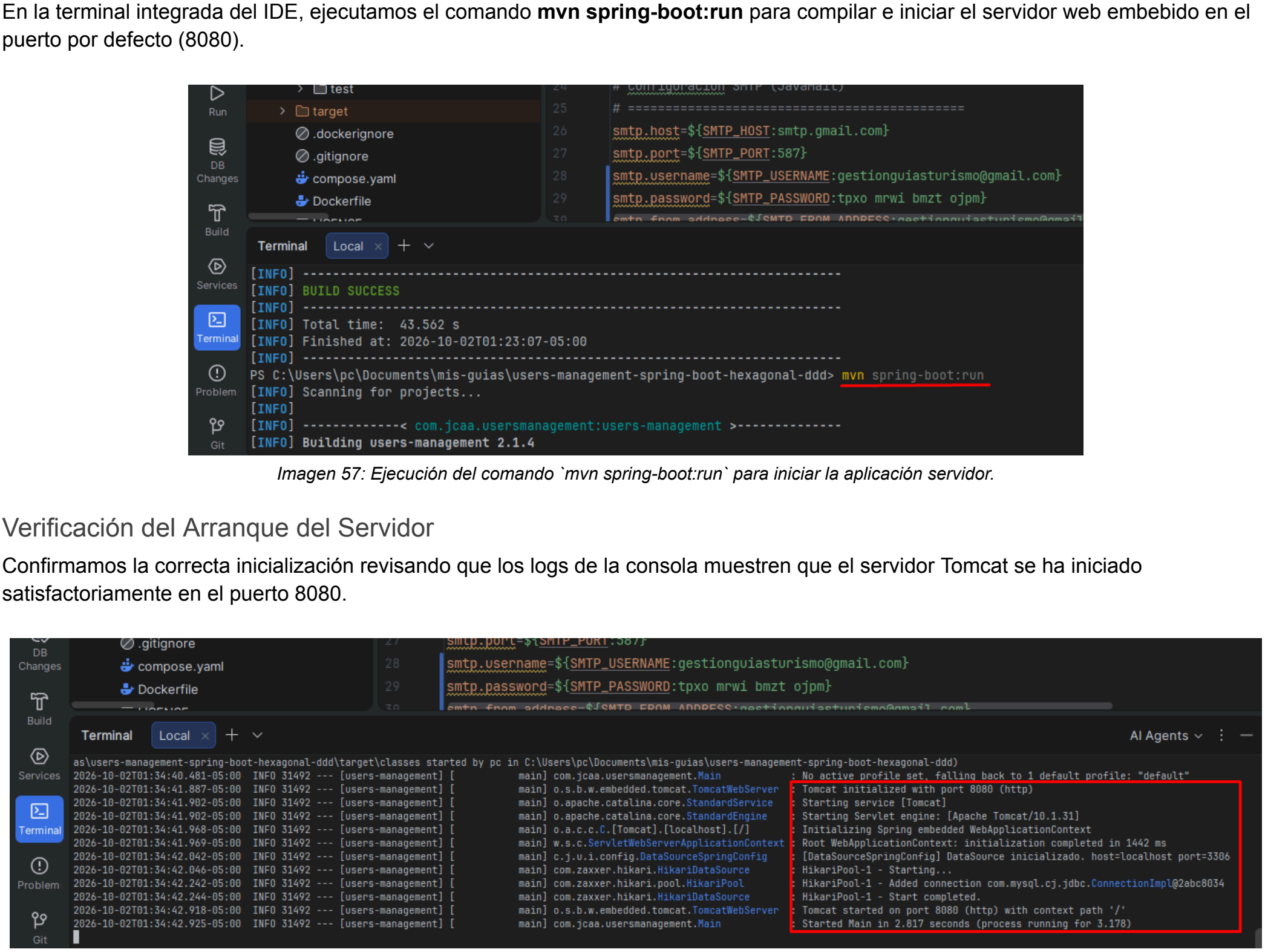


Imagen 58: Logs de consola indicando la inicialización del servidor embebido Tomcat en el puerto 8080.

Acceso a la Interfaz de Swagger UI

Ingresamos desde el navegador web a la ruta **http://localhost:8080/swagger-ui/index.html** para interactuar de forma gráfica con la documentación interactiva OpenAPI y probar los controladores expuestos.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

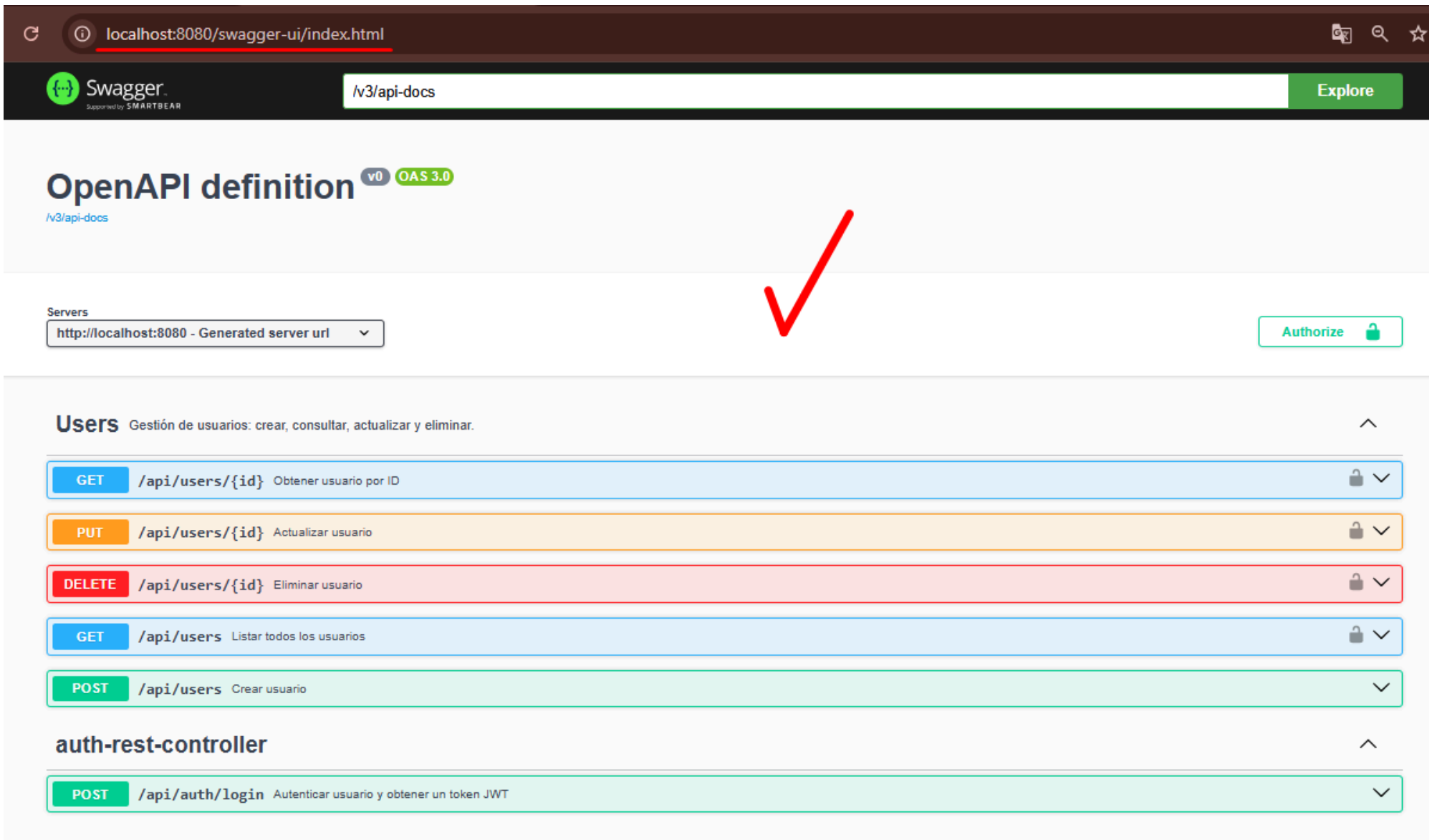


Imagen 59: Interfaz interactiva de Swagger UI listando los endpoints REST disponibles en el controlador.

Prueba de Registro de Usuario (POST /api/users)

Seleccionamos la petición **POST /api/users**, presionamos **Try it out** e ingresamos una estructura JSON con los datos requeridos para crear un nuevo usuario en el sistema.

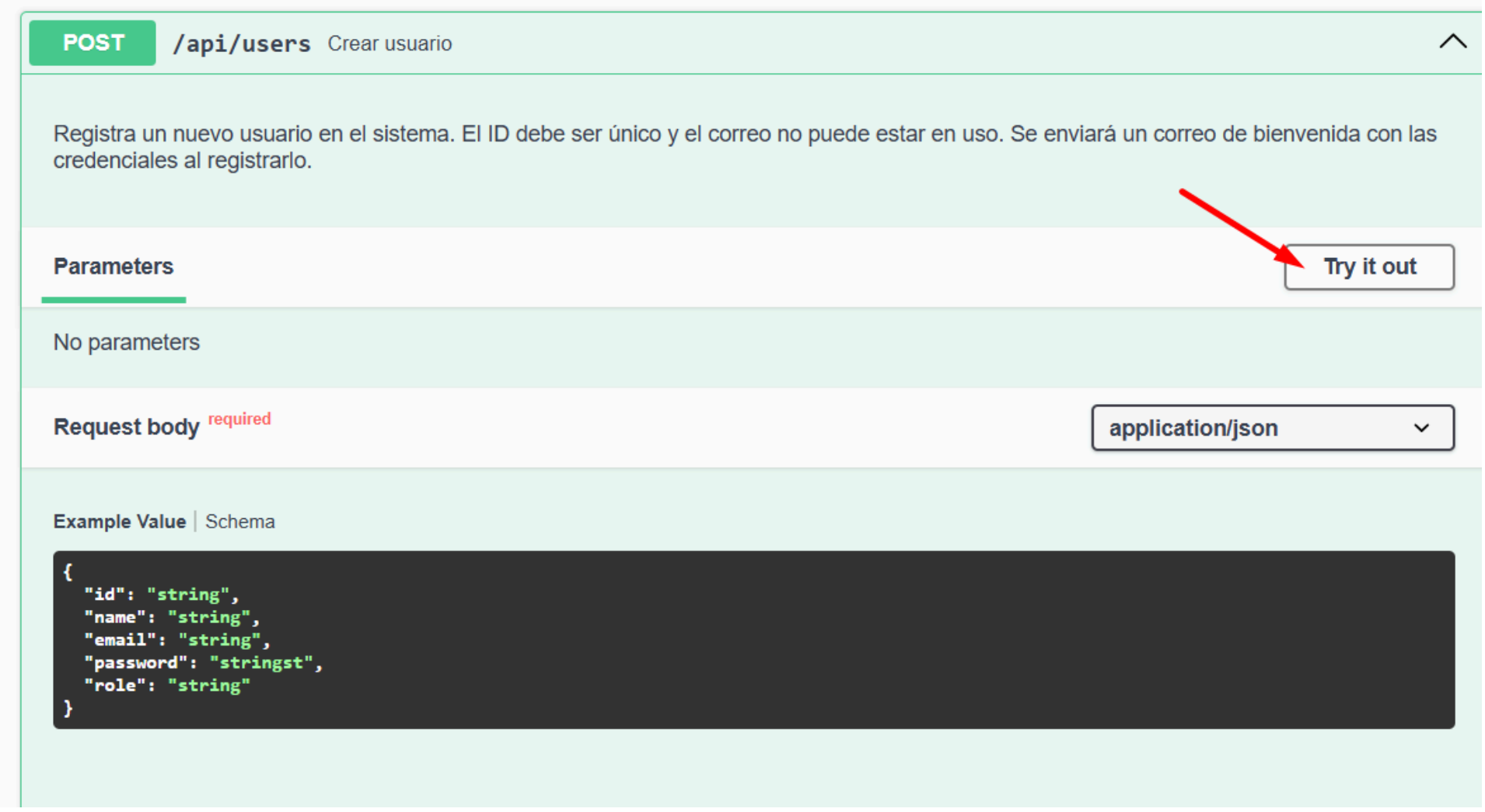


Imagen 60: Vista del formulario de prueba para la petición POST /api/users haciendo clic en "Try it out".

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

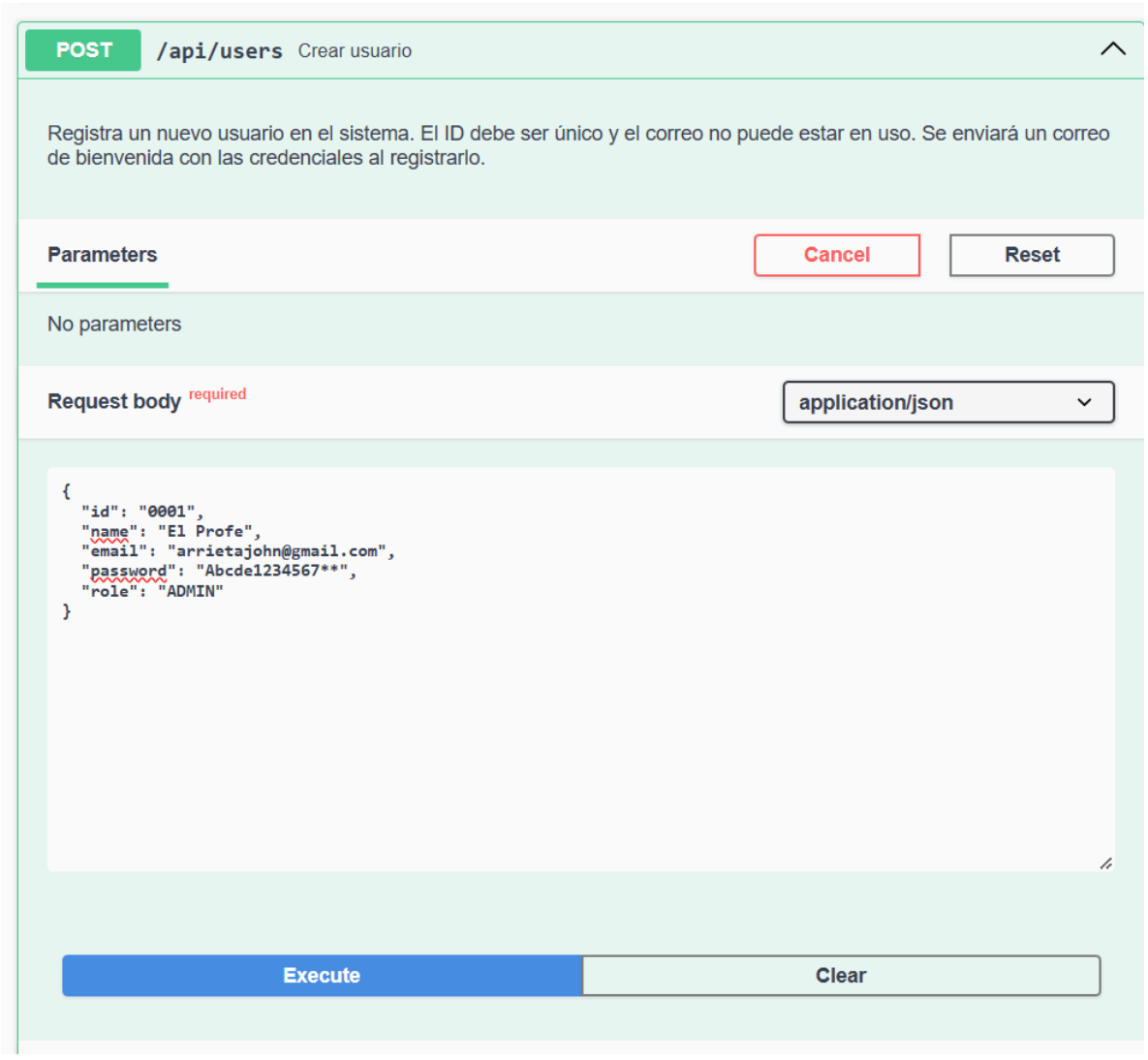


Imagen 61: Cuerpo JSON de la petición ingresado con la información del nuevo usuario a registrar.

Verificación de Respuesta HTTP 201 Created

Verificamos que el servidor responda con un código de estado **201 Created** y devuelva el cuerpo del objeto creado con su identificador y estado inicial en estado PENDING.

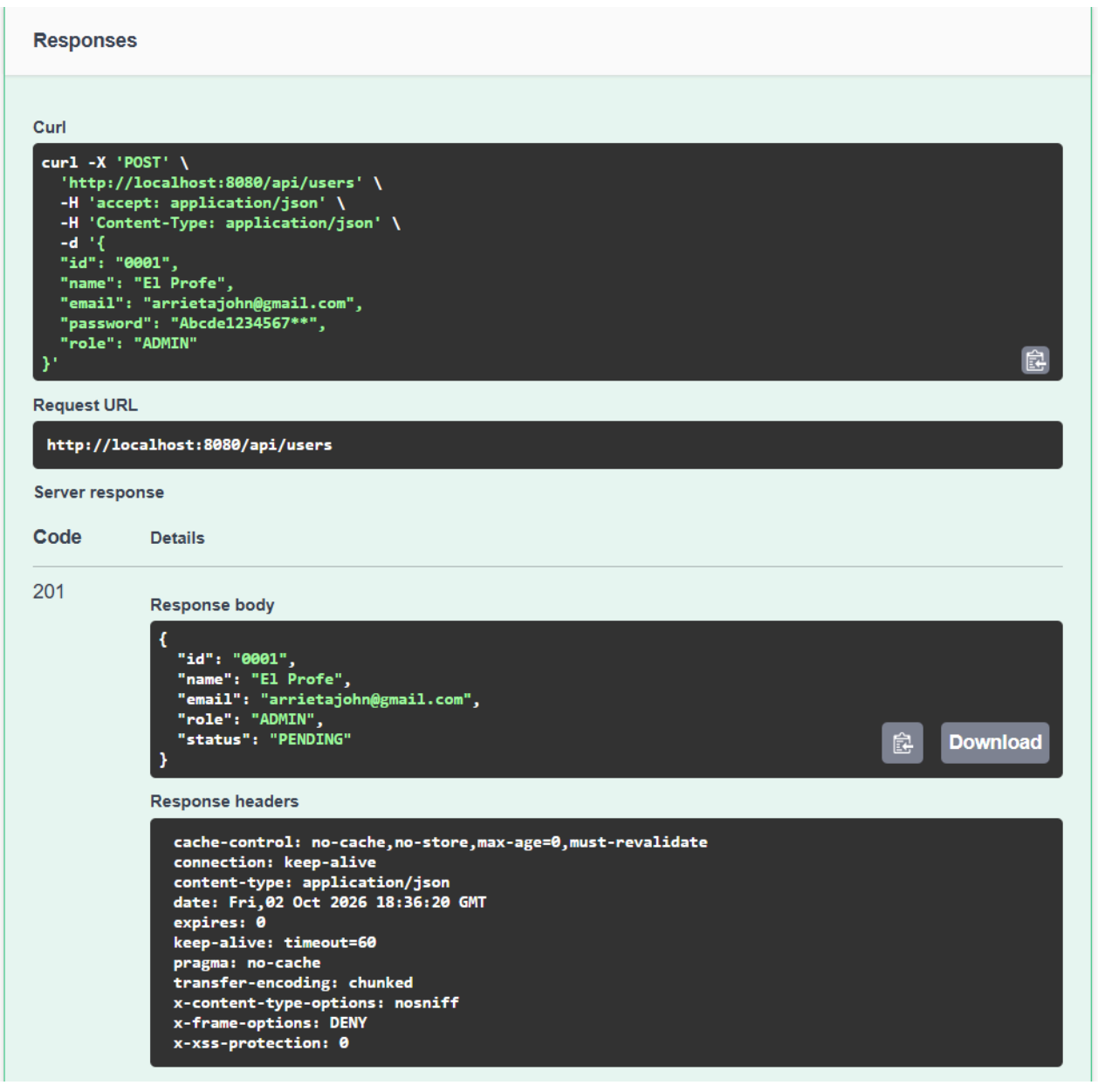


Imagen 62: Respuesta HTTP 201 Created recibida desde el servidor confirmando la creación del usuario.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Comprobación del Envío de Correo Electrónico

Revisamos la bandeja de entrada de la cuenta registrada para corroborar la recepción automática del mensaje de alta con la información de credenciales temporales.
Confirmación del correo de alta de usuario

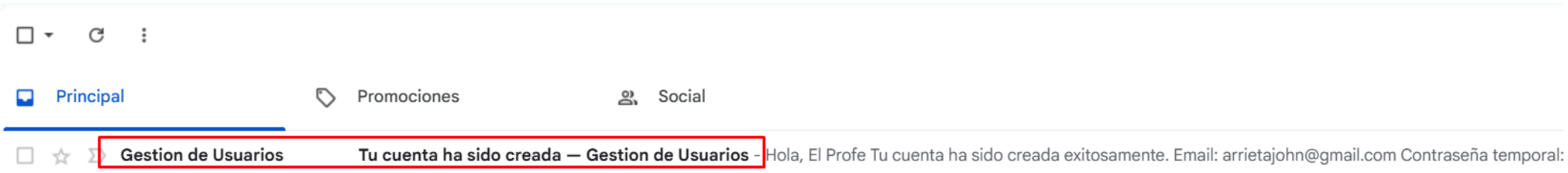


Imagen 63: Notificación de correo entrante recibido en la bandeja de entrada tras el registro.

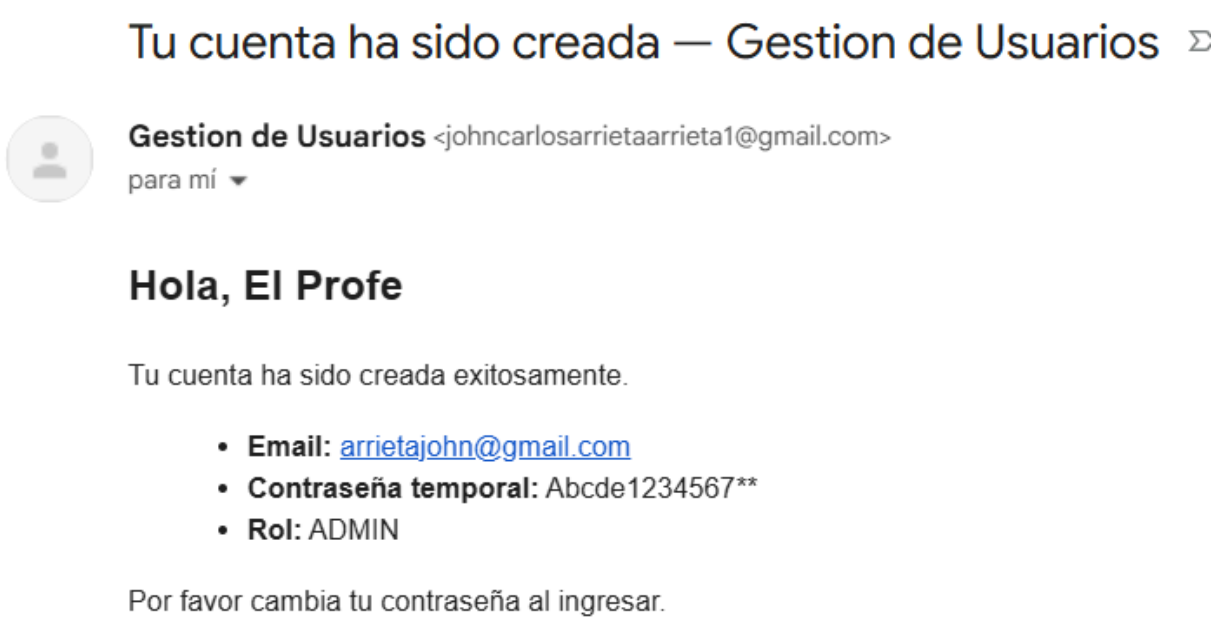


Imagen 64: Contenido del correo electrónico de bienvenida con las credenciales asignadas al usuario.

Autenticación y Obtención del Token JWT (POST /api/auth/login)

Para consumir endpoints protegidos, enviamos una petición de inicio de sesión con el usuario administrador por defecto para obtener un token de autenticación Bearer JWT.
Iniciar sesión para obtener el JWT válido para realizar las otras operaciones (invocar los otros endpoints del api/users)

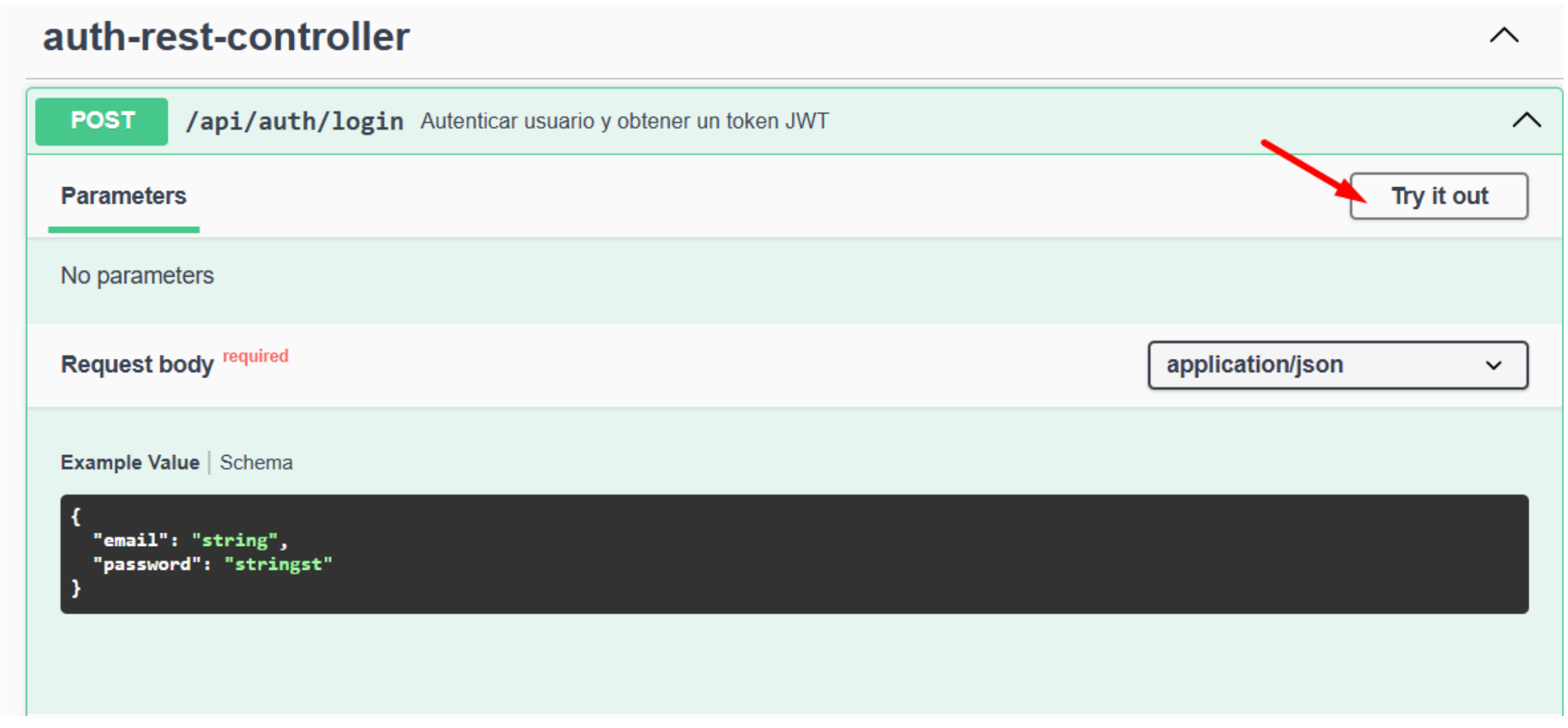


Imagen 65: Petición de autenticación POST /api/auth/login desplegada en Swagger UI.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

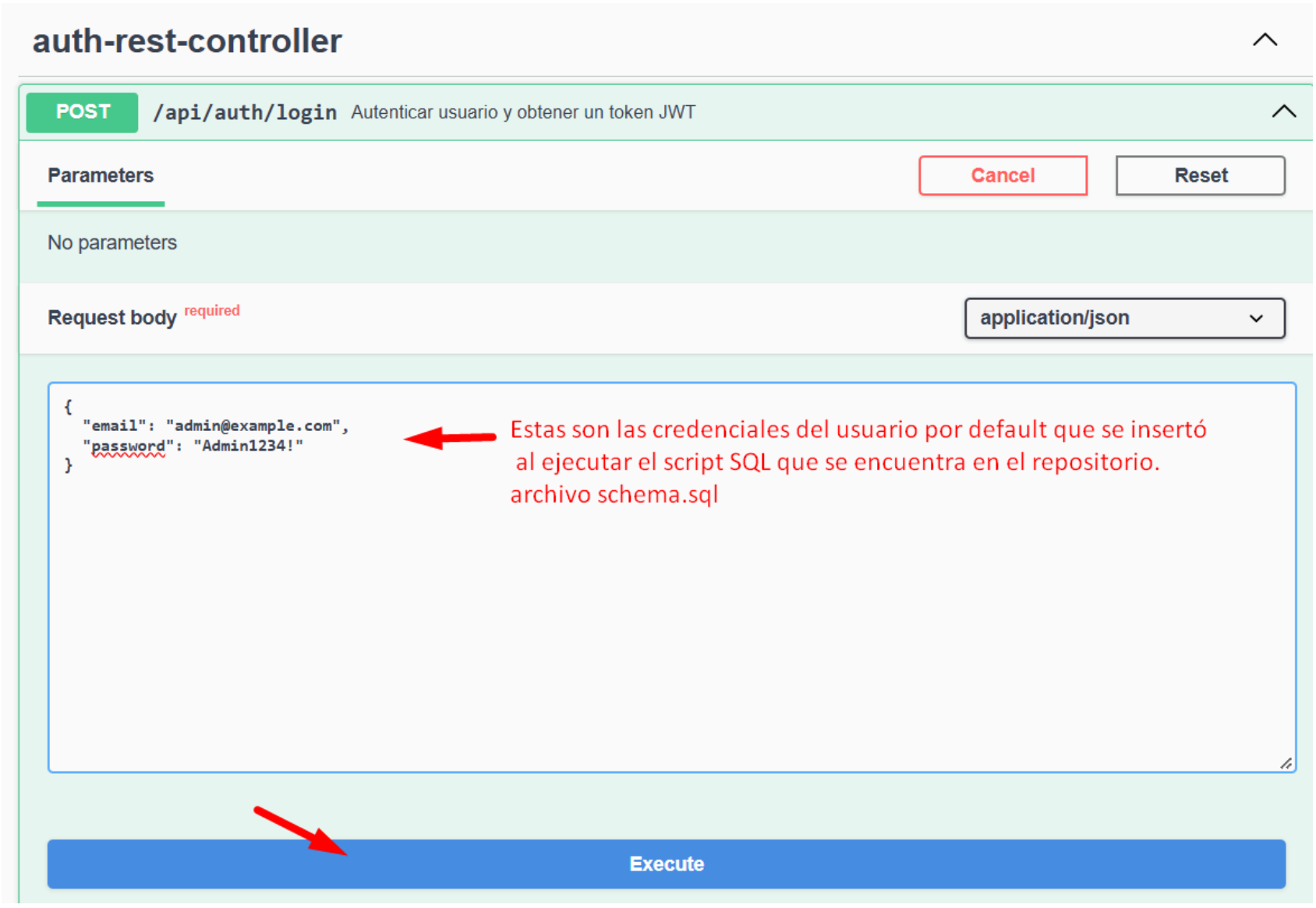


Imagen 66: Ingreso de credenciales por defecto del usuario administrador en la petición de login.

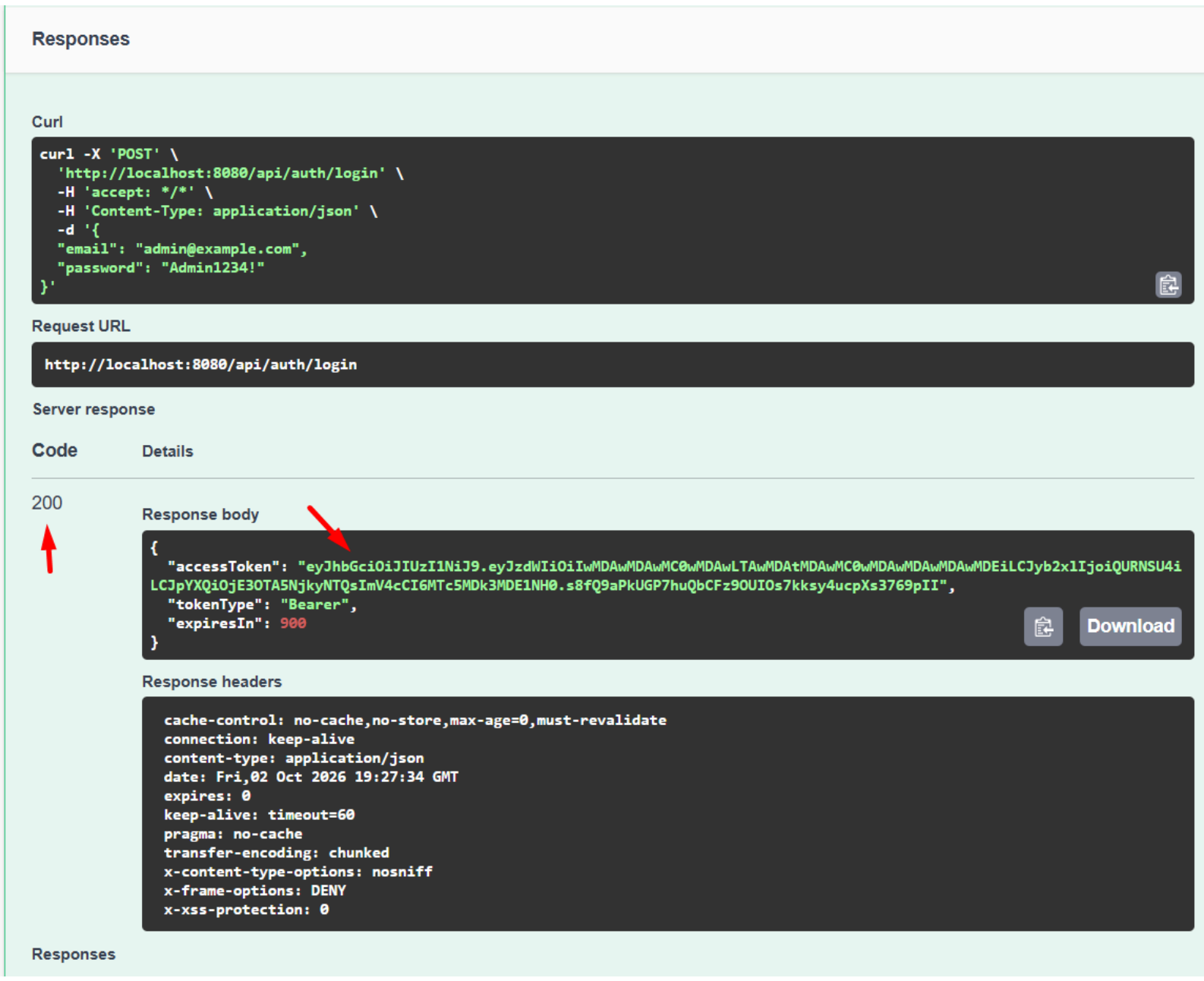
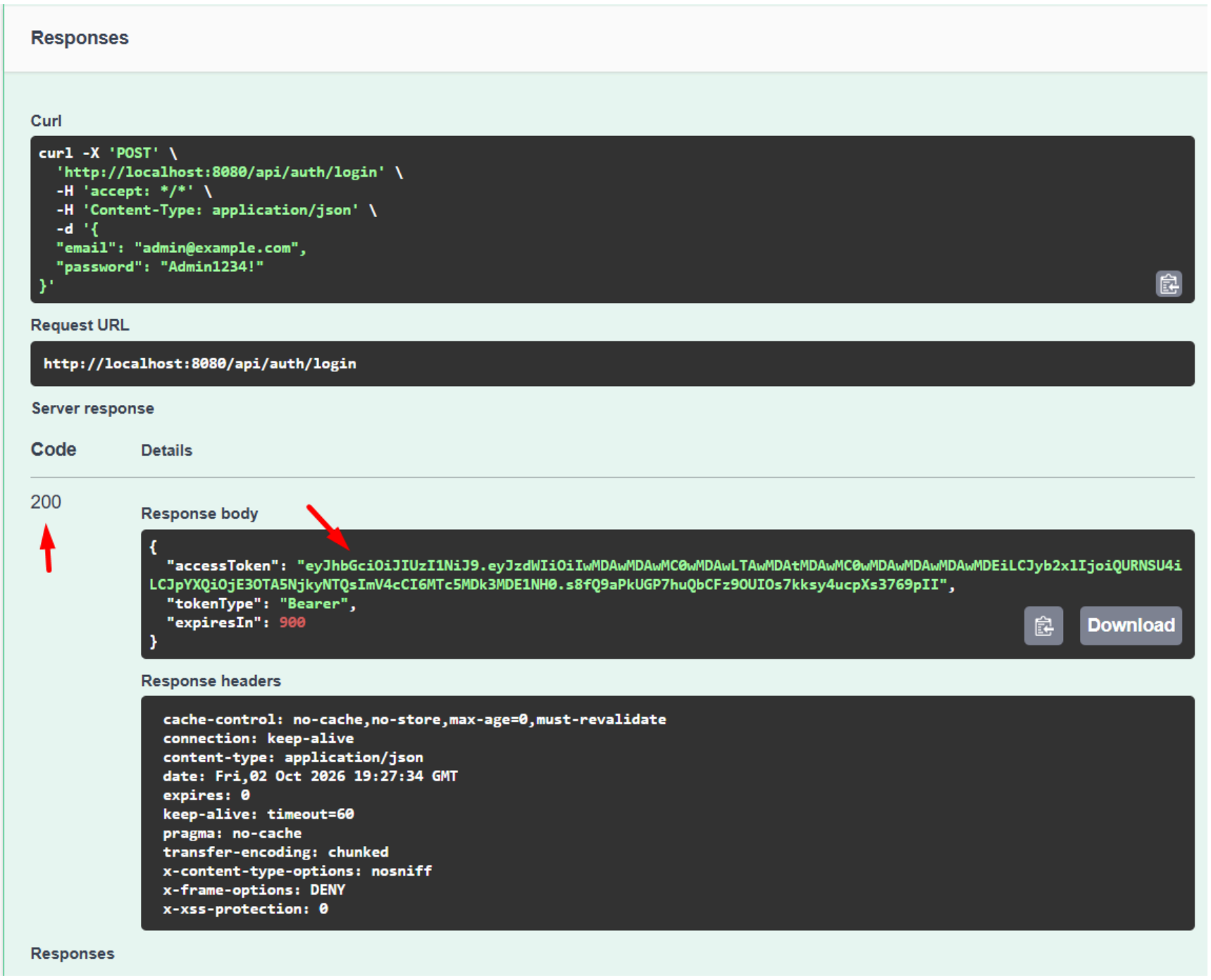


Imagen 67: Respuesta exitosa del servicio devuelvo un token de acceso JWT (accessToken).

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻



8. Pruebas de API con Clientes HTTP Externos (Boomerang / Postman)

Además de la documentación interactiva de Swagger, se recomienda utilizar un cliente de peticiones HTTP para gestionar colecciones de prueba reutilizables.
Ahora usamos una herramienta para realizar peticiones HTTP

Actualmente existen muchas opciones, las más populares son:
Postman: https://www.postman.com/downloads/ (aplicacion web o desktop)
Insomnia: https://insomnia.rest/download (aplicacion desktop)
Boomerang: https://chromewebstore.google.com/detail/boomerang-soap-rest-clien/eipdnjedkpcnlmmdfdkgfpljanehloah?hl=es&pli=1 (complemento para Google Chrome y navegadores derivados)
Rest Client: https://marketplace.visualstudio.com/items?itemName=humao.rest-client (complemento para VSC)
Http Client: https://plugins.jetbrains.com/plugin/13121-http-client (Complemento para IJIDEA)
Rest Client: https://plugins.netbeans.apache.org/catalogue/?id=81 (complemento para Netbeans)
HTTP4e: https://marketplace.eclipse.org/content/http4e-http-and-rest-client-eclipse (complemento para Sprint Tools)

Instalación de la Extensión Boomerang

Accedemos a la tienda de aplicaciones de Google Chrome (Chrome Web Store), buscamos la extensión **Boomerang - SOAP & REST Client** y presionamos **Añadir a Chrome**.
Vamos a usar Boomerang y tú pruebas con los otros.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Imagen 68: Ficha de la extensión Boomerang en la Chrome Web Store haciendo clic en "Añadir a Chrome".

Creación de un Proyecto en Boomerang

Iniciamos la extensión y creamos un nuevo proyecto de trabajo asignándole un título relacionado con el taller (por ejemplo, **Taller de MicroServicios Spring Boot Hexagonal**).

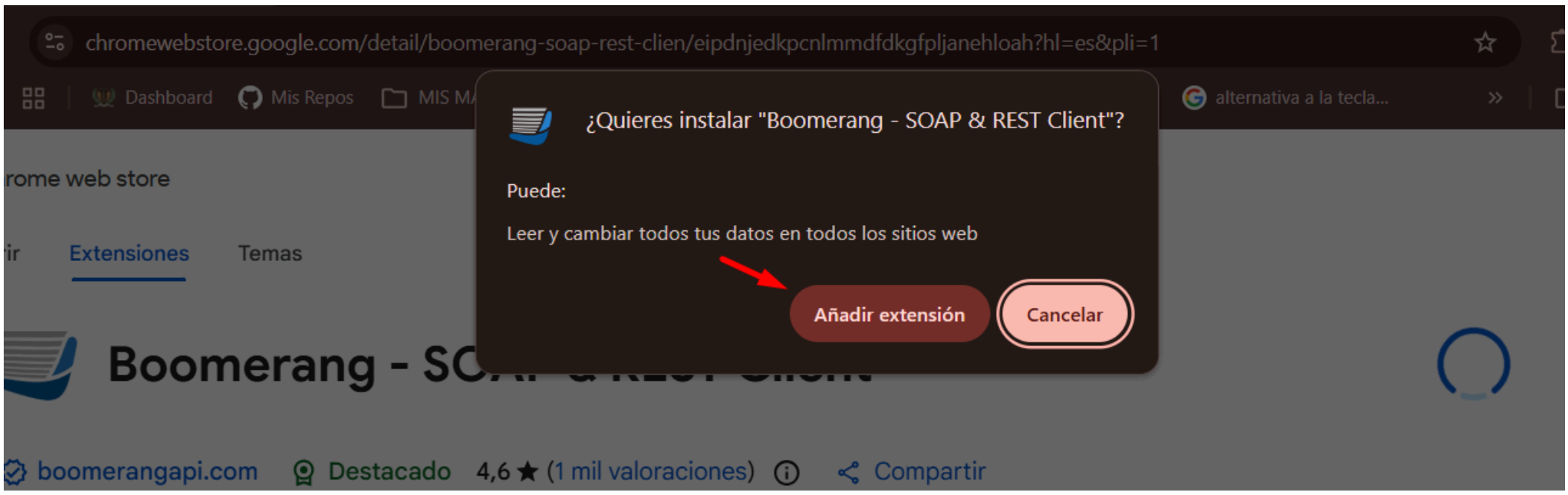


Imagen 69: Diálogo emergente confirmando la instalación de la extensión en el navegador.

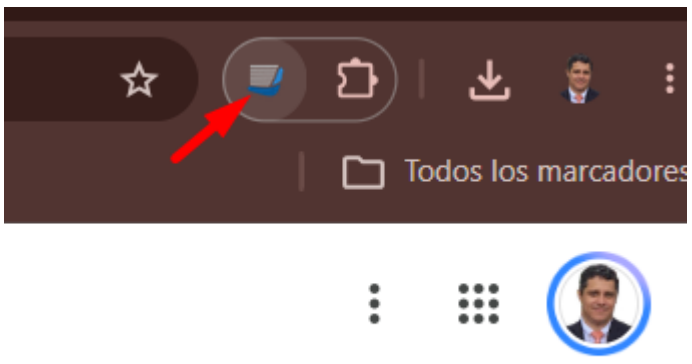


Imagen 70: Ícono de acceso directo de Boomerang fijado en la barra superior del navegador.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

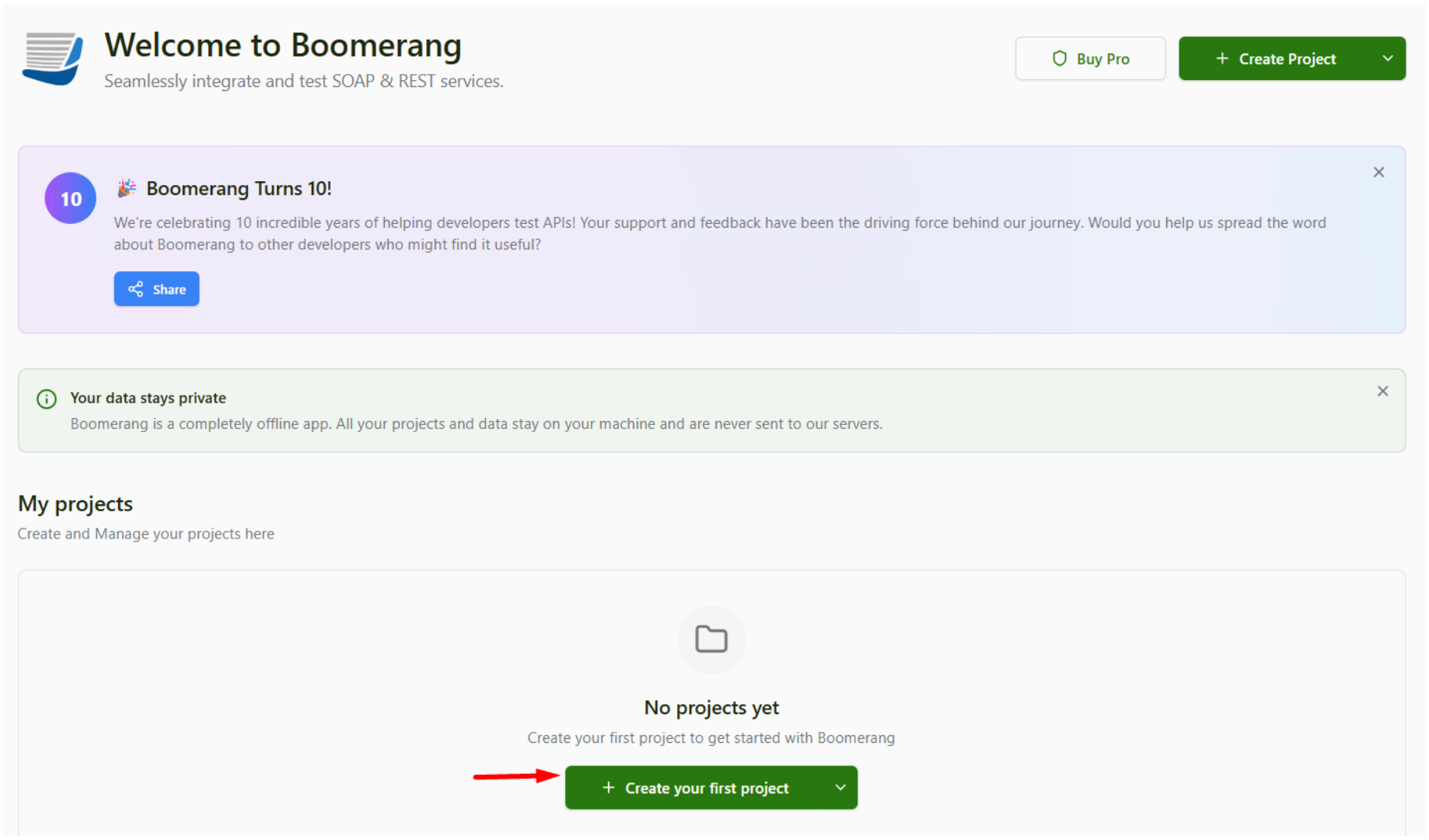


Imagen 71: Pantalla principal de inicio de la extensión Boomerang para crear proyectos de prueba.

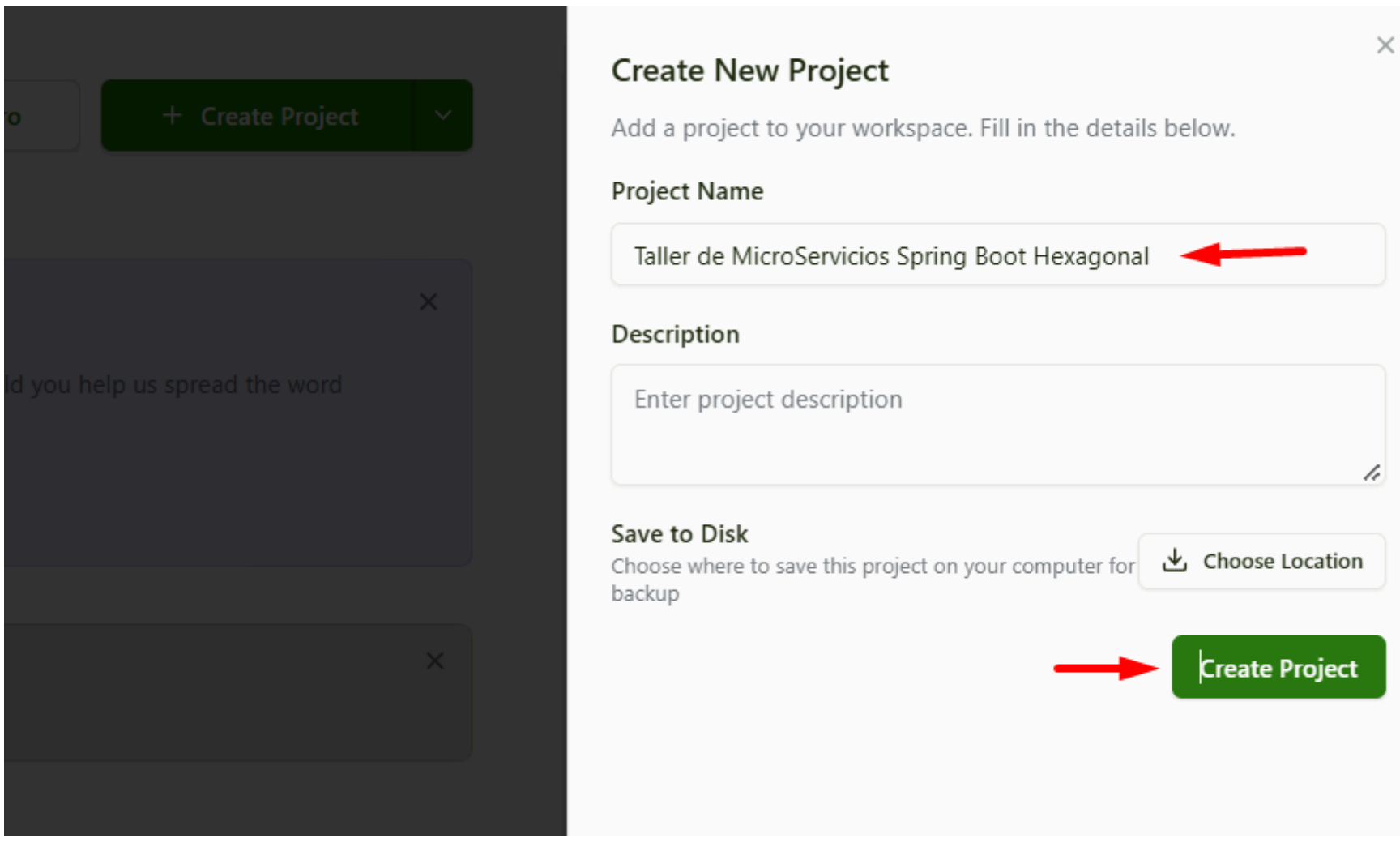


Imagen 72: Ventana emergente "Create New Project" en Boomerang asignando el nombre del taller.

configuración de Petición HTTP GET

Añadimos una nueva solicitud seleccionando el método **GET** e ingresando la URL del recurso expuesto: **http://localhost:8080/api/users**.

CC-BY.CC **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

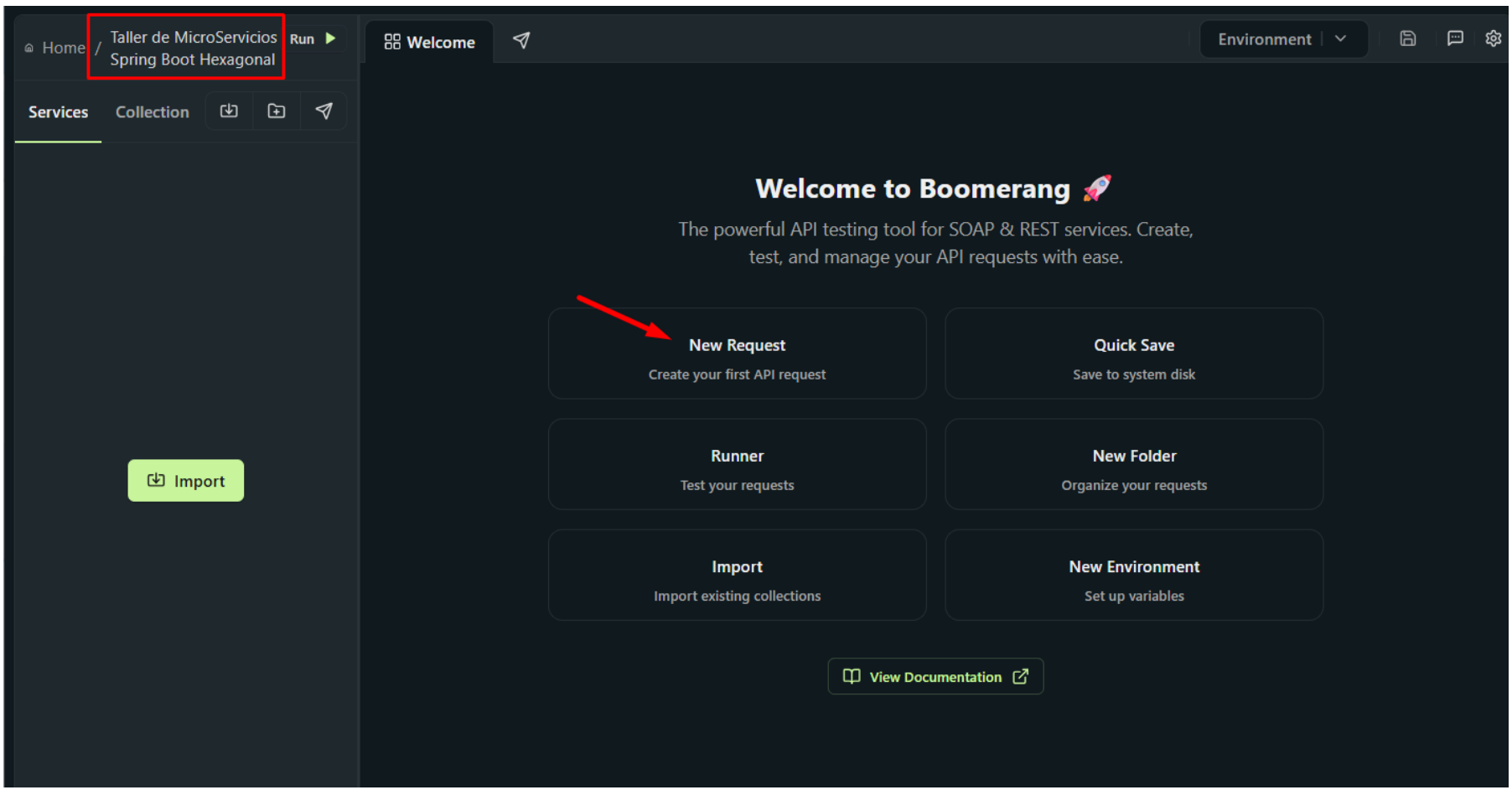


Imagen 73: Panel de control del proyecto recién creado listo para agregar nuevas solicitudes ("New Request").

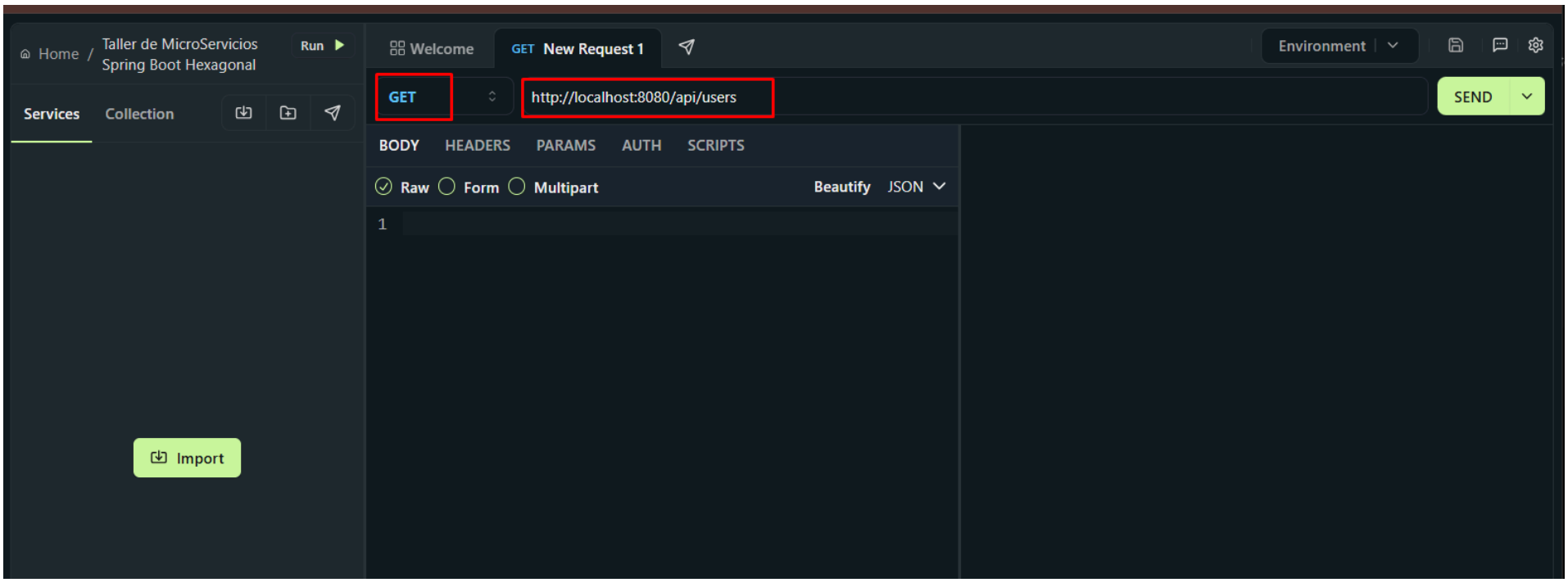


Imagen 74: configuración de la petición HTTP introduciendo el método GET y la URL del endpoint `/api/users`.

Inclusión de la Cabecera de Autorización JWT

En la pestaña **Headers**, agregamos la clave **Authorization** y asignamos como valor el prefijo **Bearer** seguido del token JWT copiado en el paso de inicio de sesión.

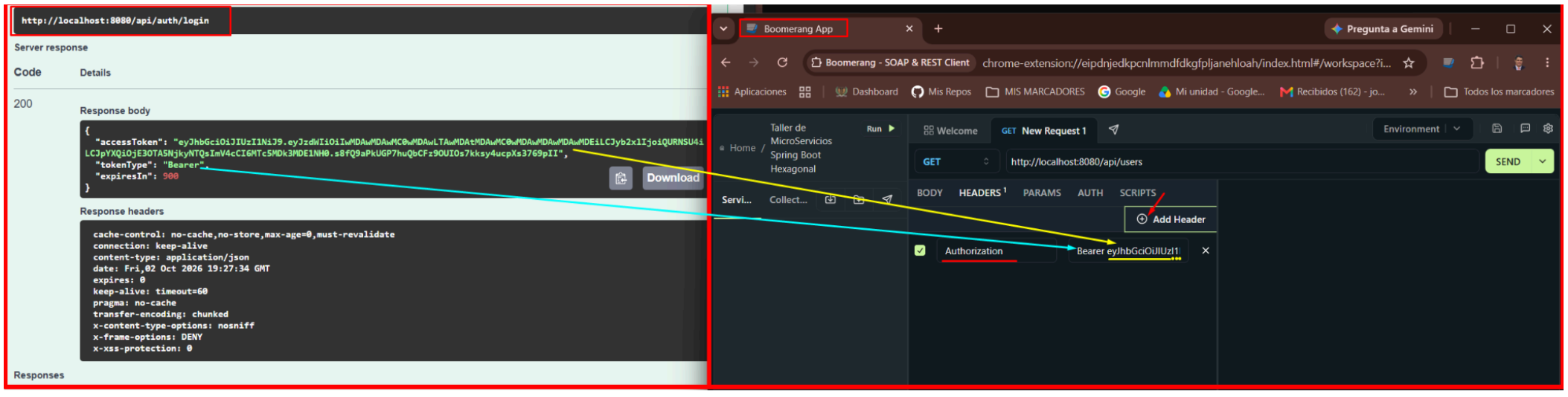


Imagen 75: Vinculación del token Bearer JWT copiado desde Swagger UI dentro del campo de encabezado "Authorization" en Boomerang.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

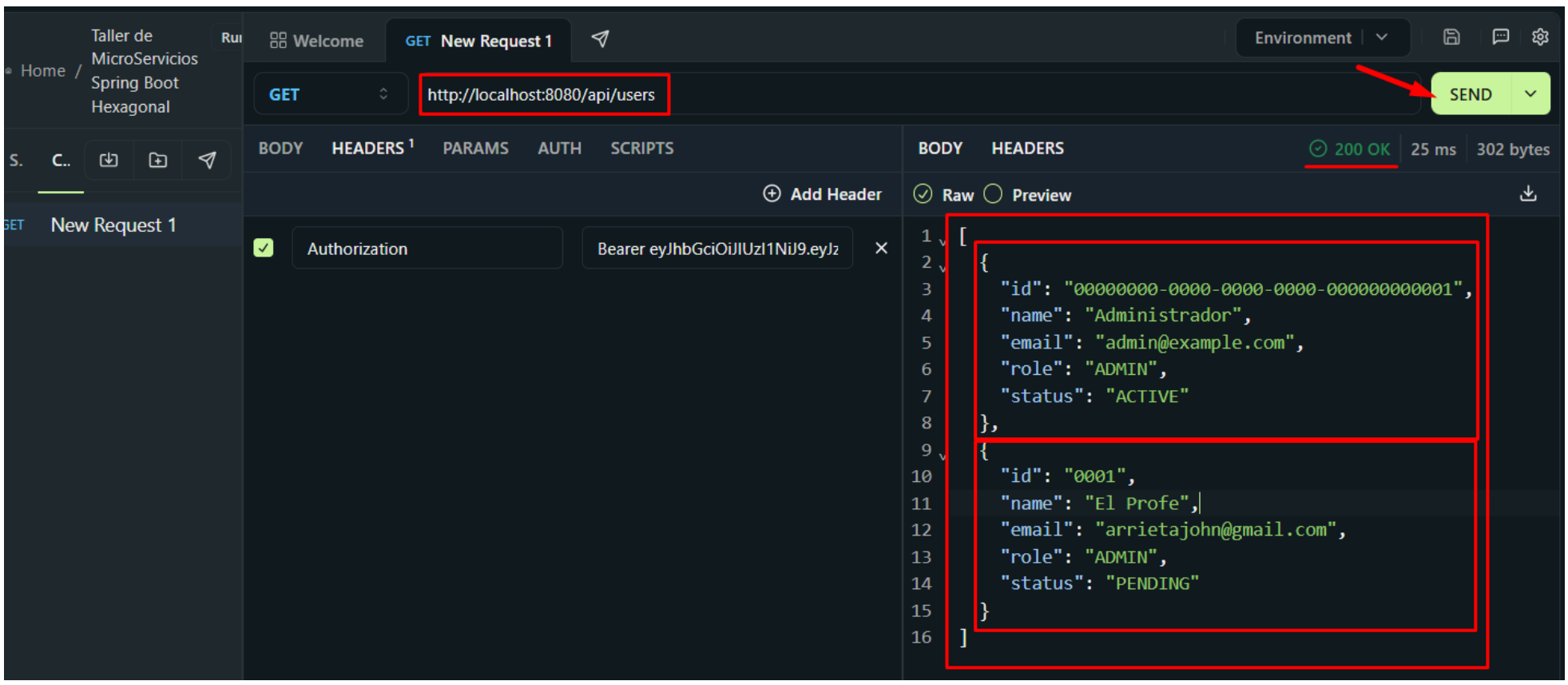


Imagen 76: Respuesta HTTP 200 OK listando la colección de usuarios en formato JSON.

Ejecución de la Petición y Lectura de la Respuesta

Presionamos el botón **Send** y verificamos que el servidor retorne un código de respuesta **200 OK** junto con la lista JSON conteniendo todos los usuarios almacenados en la base de datos.

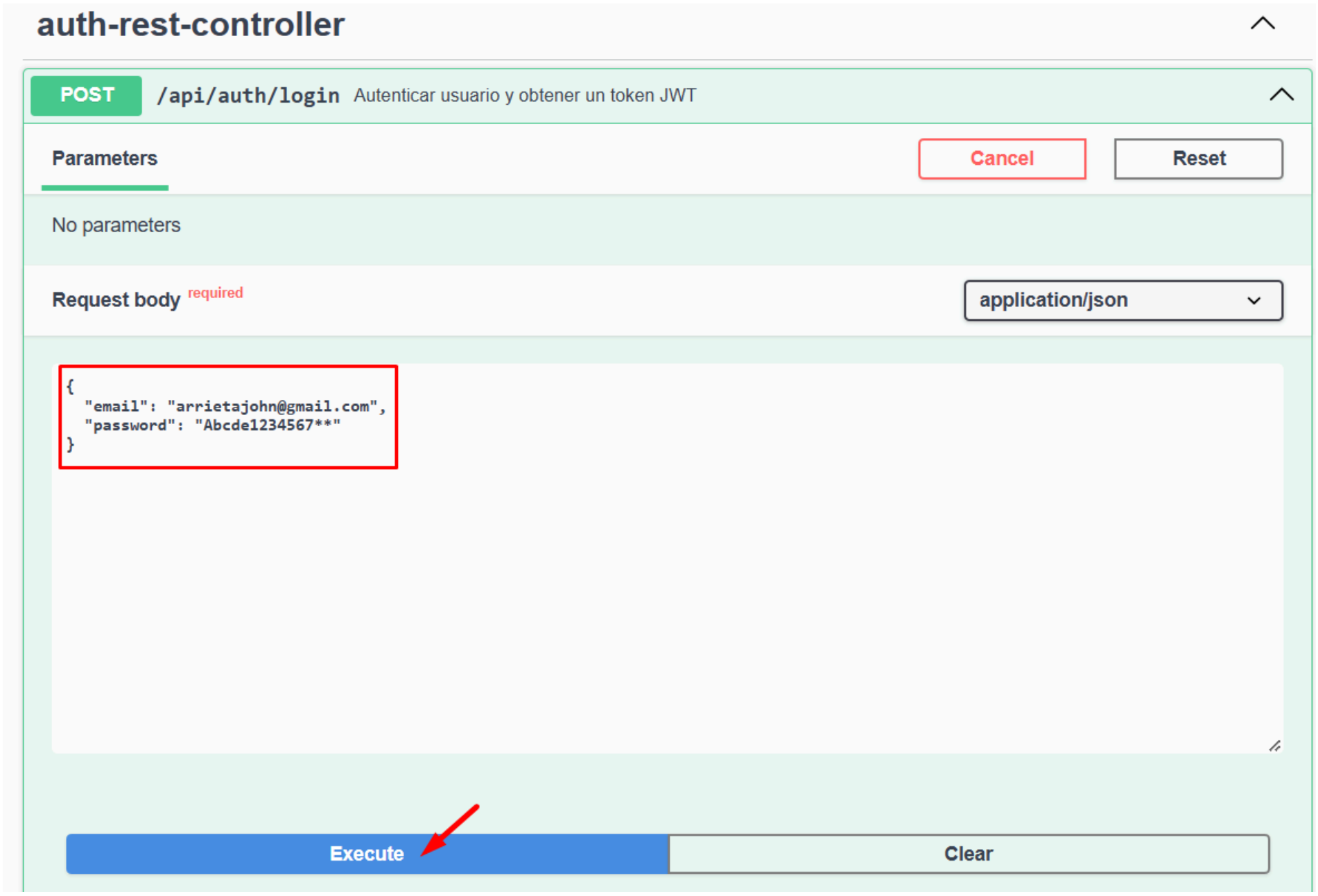


Imagen 77: Envío de credenciales de usuario registrado mediante Swagger UI para verificar inicio de sesión.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

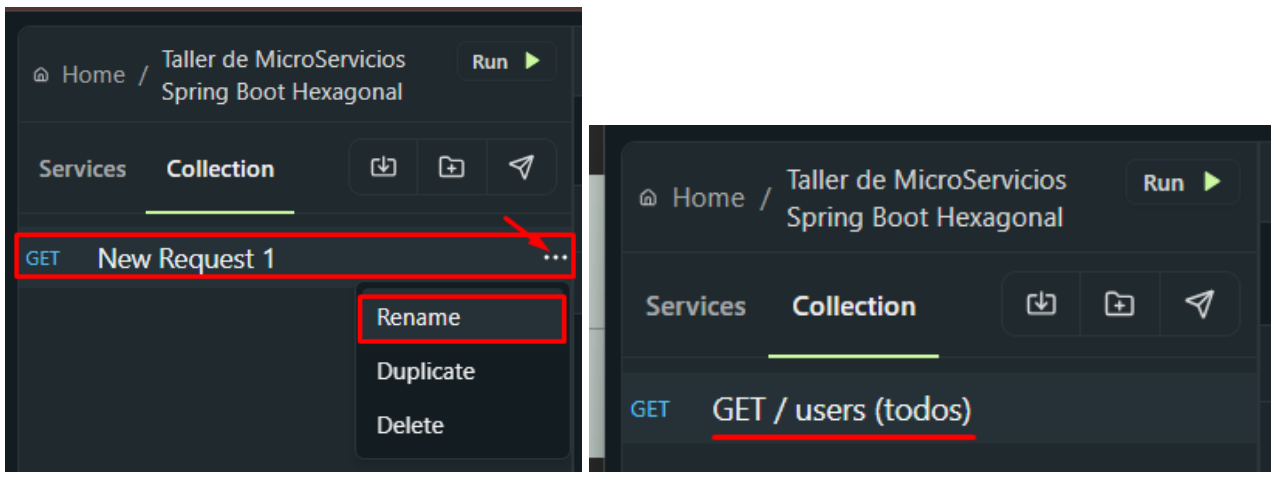


Imagen 78: Menú desplegable en Boomerang permitiendo renombrar solicitudes guardadas en la colección.

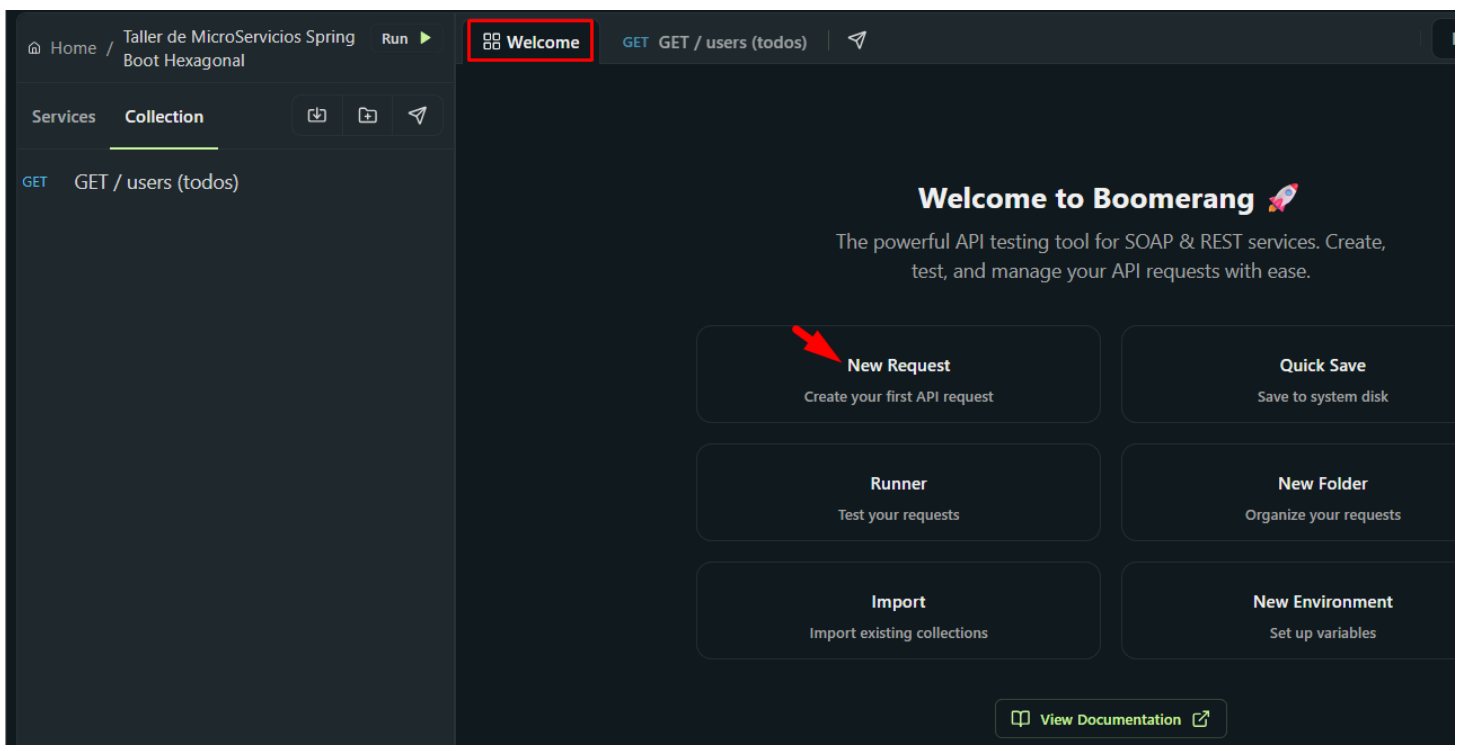


Imagen 79: Vista del proyecto organizado listando las peticiones configuradas en el panel lateral.

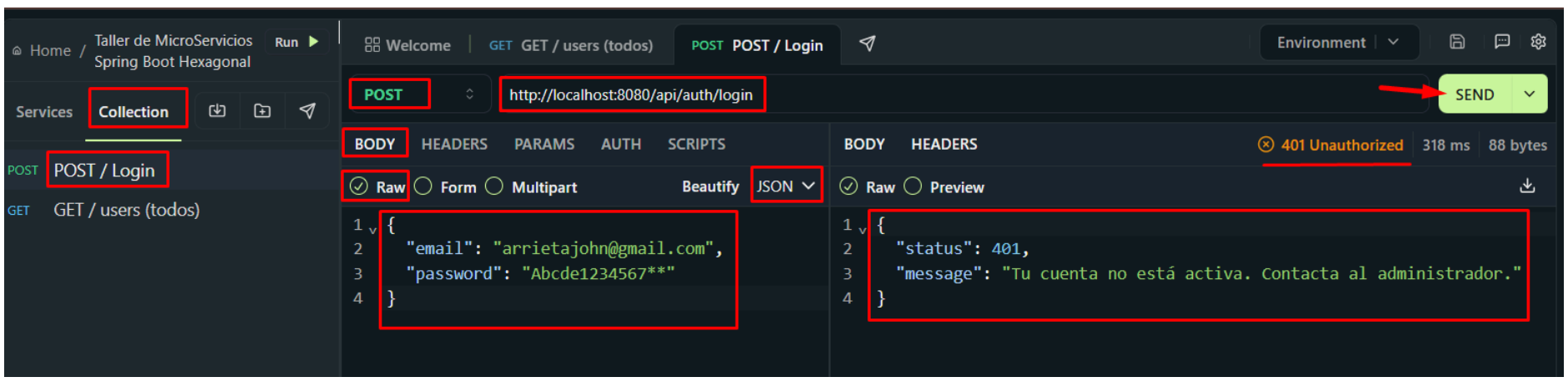


Imagen 80: Prueba de intento de inicio de sesión con usuario inactivo retornando error 401 Unauthorized.

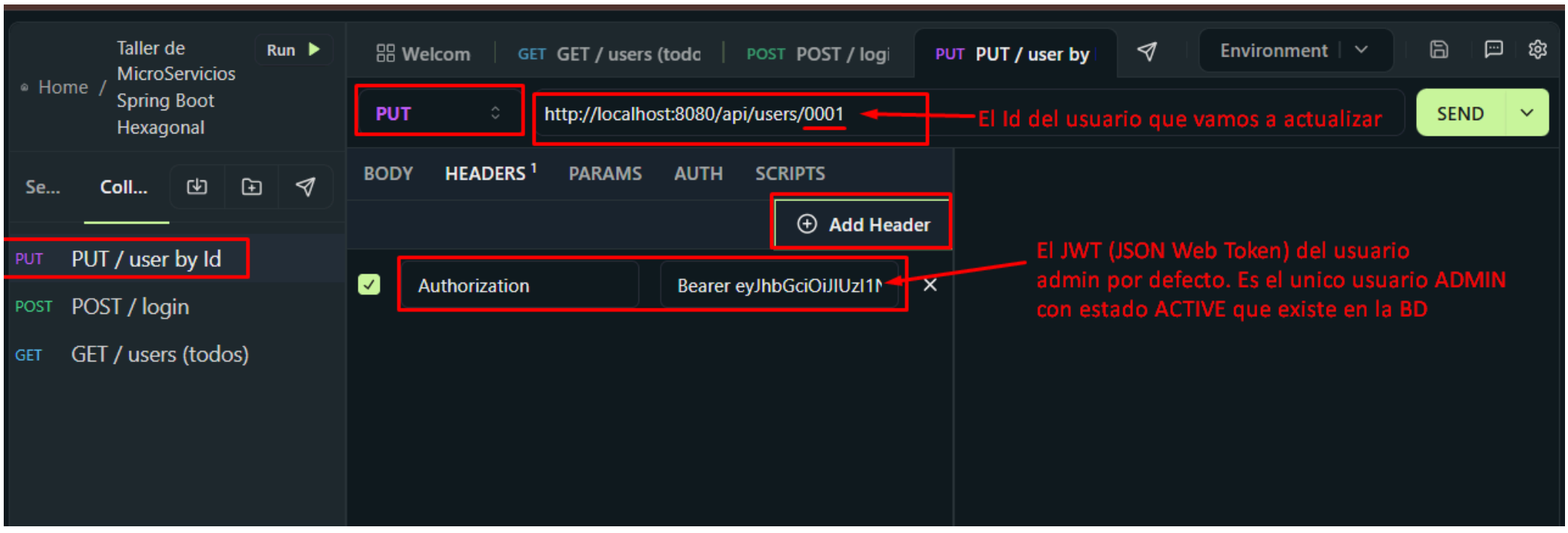


Imagen 81: configuración de la solicitud PUT para actualizar la información de un usuario específico por su ID.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻



Imagen 82: Respuesta HTTP 200 OK confirmando la actualización de los datos del usuario en la base de datos.

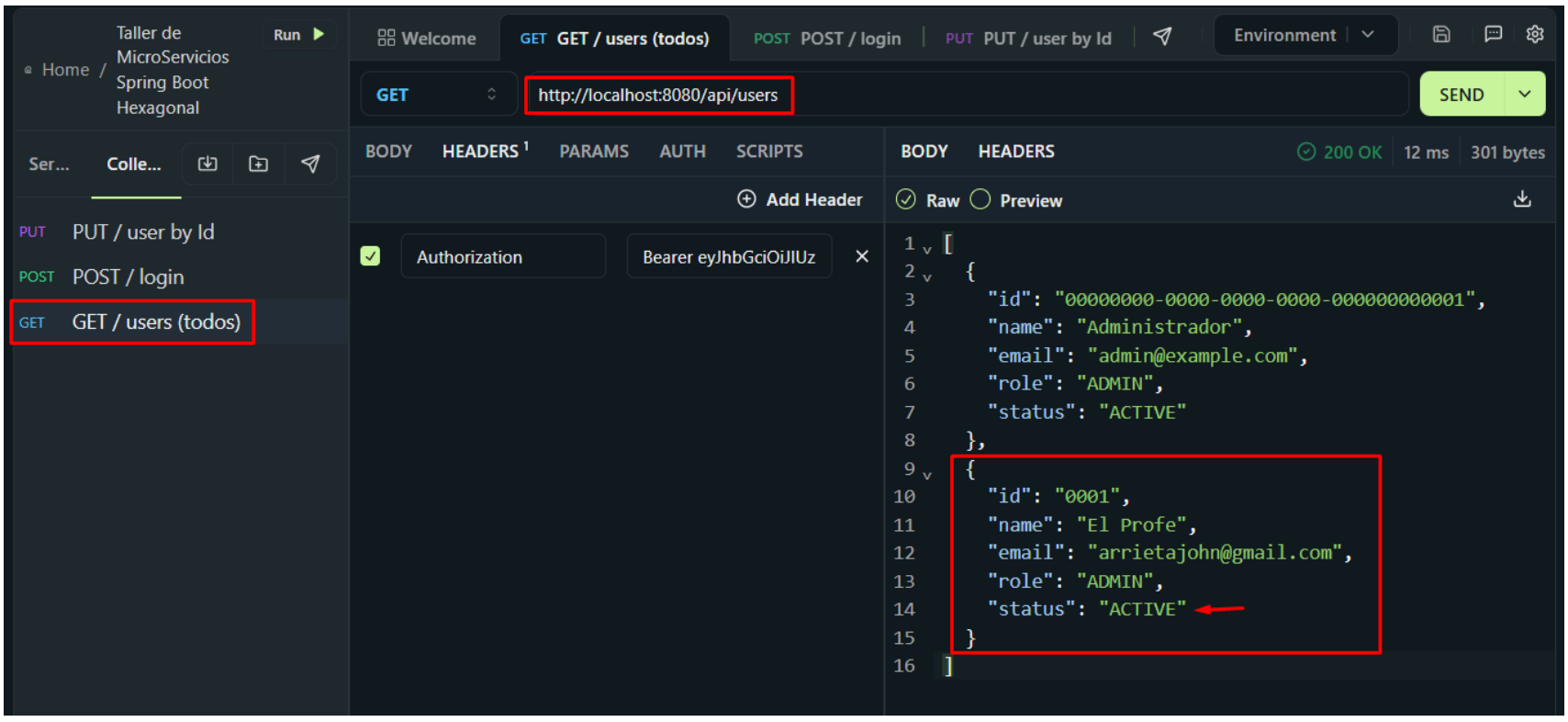
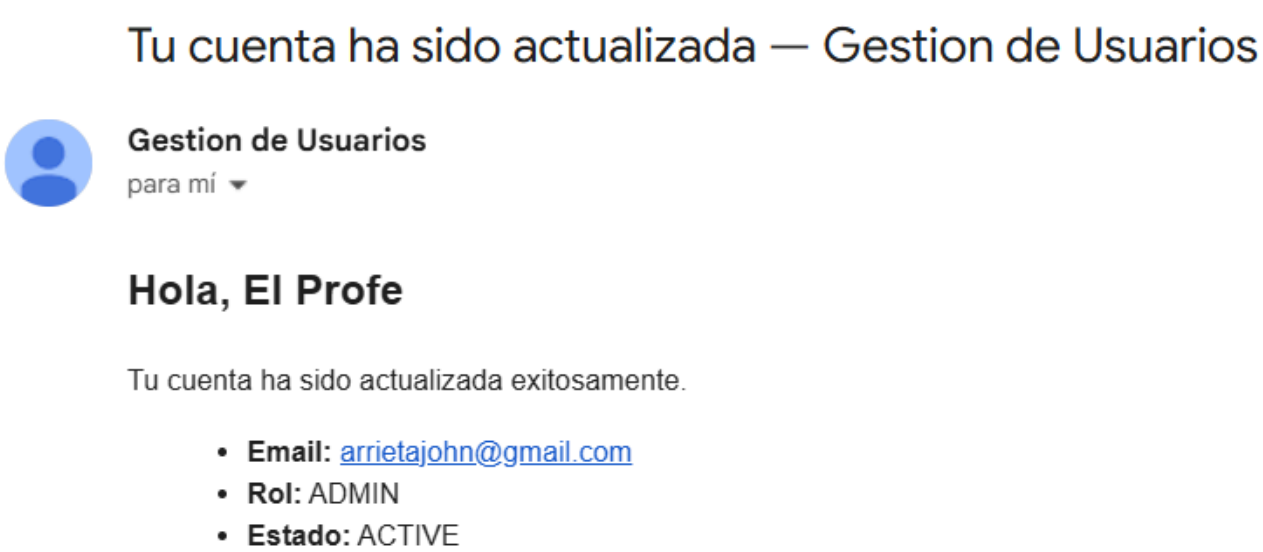


Imagen 83: Petición GET devuelvo la lista actualizada reflejando el cambio de estado del usuario a "ACTIVE".

Validación del correo de notificación de cambio de estado





Guia 1 - Clonar el repositorio y levantar el microservicio en local

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

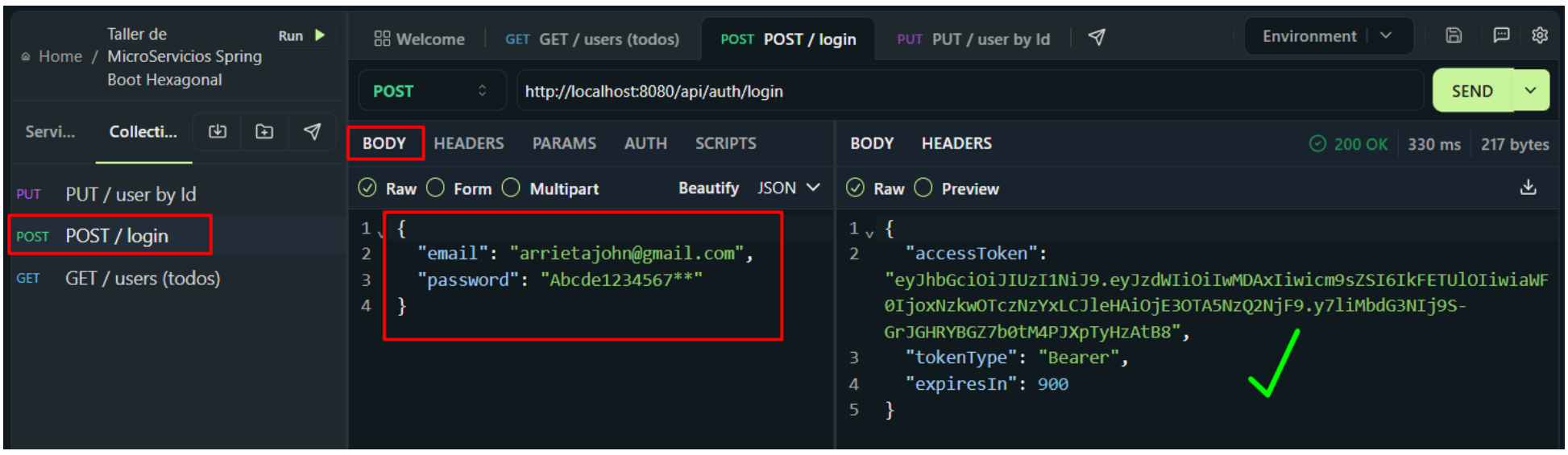
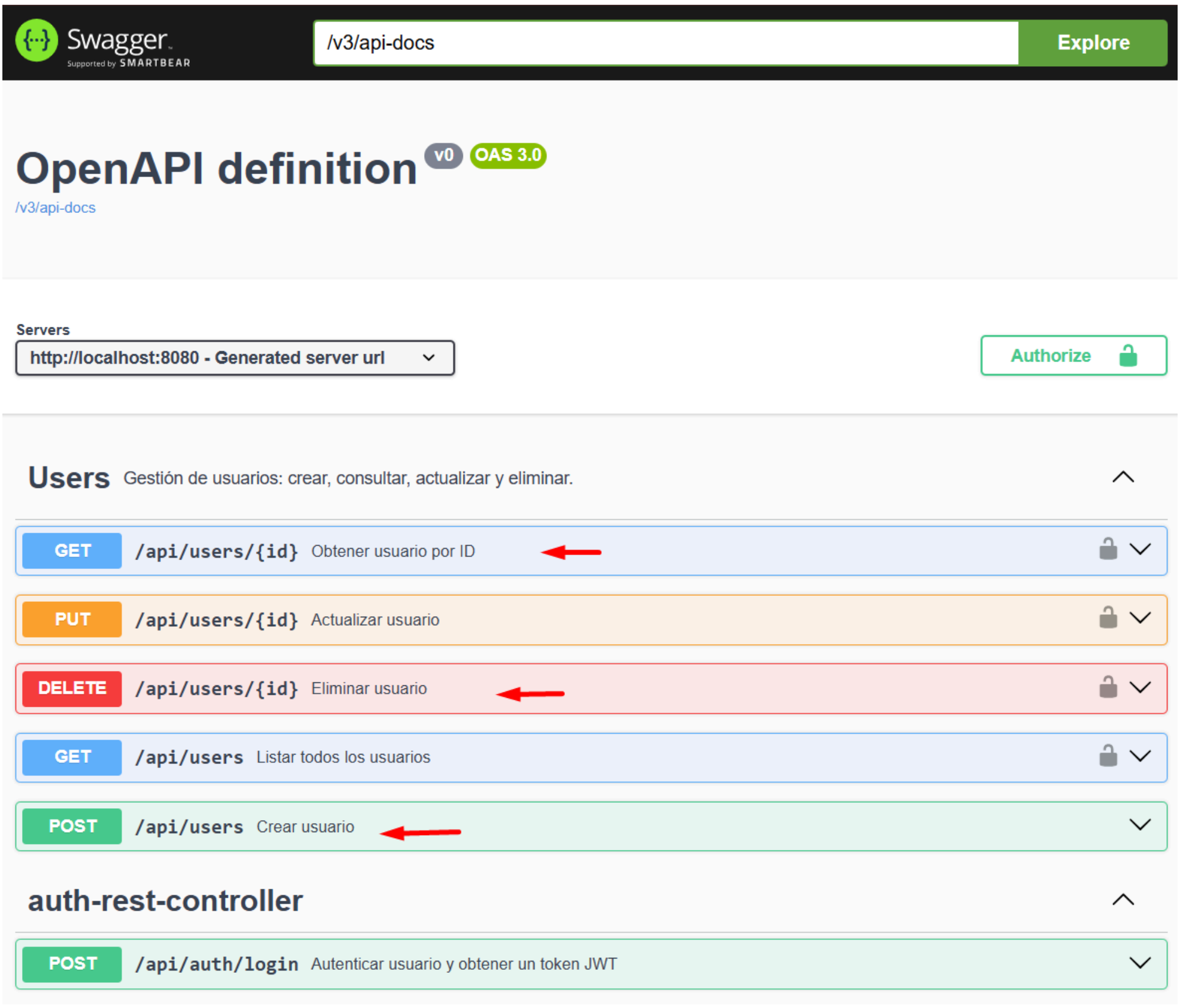


Imagen 84: Verificación final de autenticación exitosa obteniendo un token JWT tras la activación de la cuenta.

A modo de ejercicio, agrega en Boomerang las requests (peticiones) que faltan (siempre guitare de la documentación de Swagger)

- POST / users (crear usuario)
- GET / users by Id (Obtener usuario por su Id
- Delete / users by id (Eliminar usuario por su Id)



CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Exportación de la Colección de Peticiones

Una vez finalizadas las pruebas de los endpoints (GET, POST, PUT, DELETE), guardamos la colección y la exportamos en formato JSON. Este archivo permitirá respaldar las peticiones o importarlas en otros entornos de desarrollo como Postman o Insomnia.

Por último guarda todos las request para que las puedas importar en esta u otras herramientas similares (postman, insomnia, los complementos de los IDE)

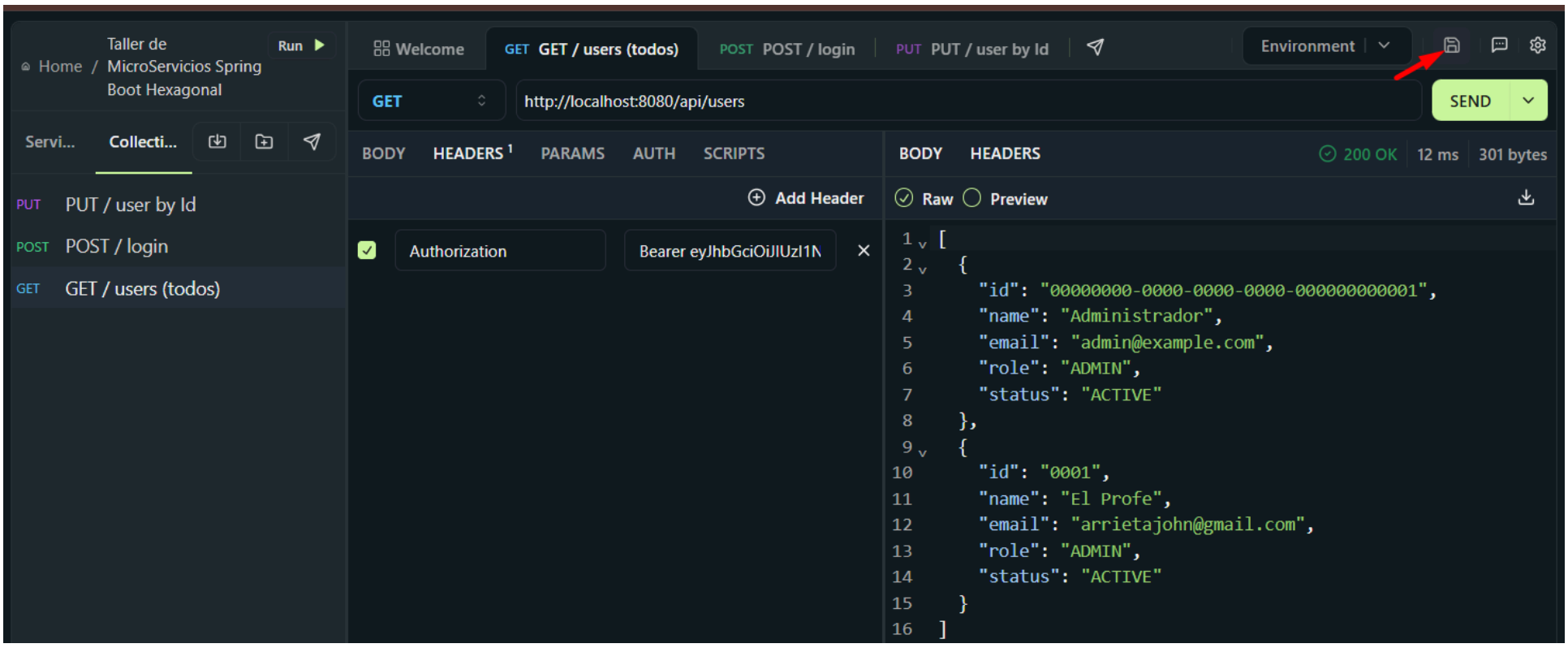


Imagen 85: Opción de guardado en Boomerang para conservar los cambios en la colección de peticiones.

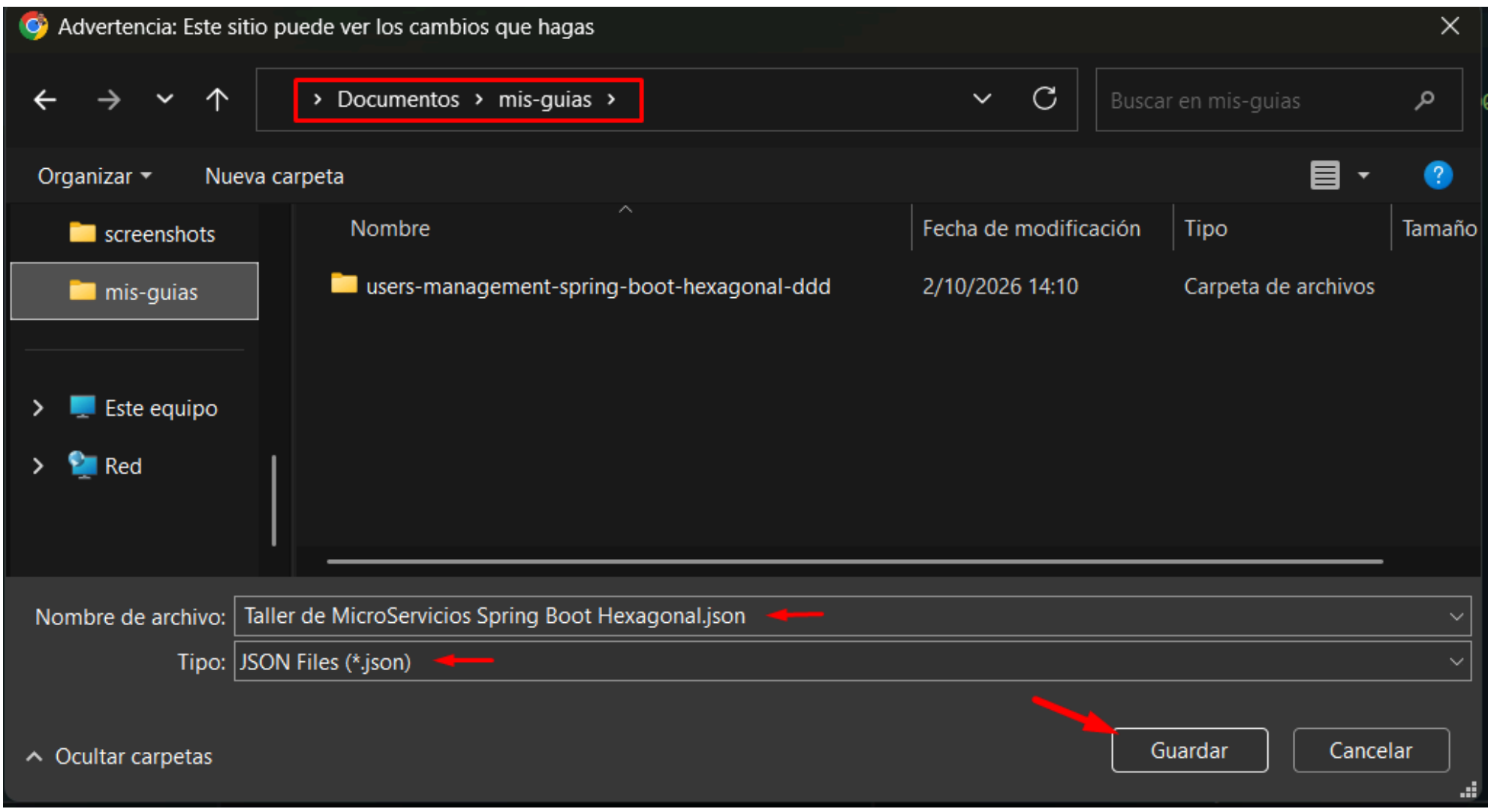


Imagen 86: Ventana para guardar el archivo JSON de exportación con todas las peticiones del taller preparadas.